

Introduction & Module Details

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Staff

▣ Lecturers:

▣ Dr. David LILLIS



▣ Assoc. Prof. Rem COLLIER



▣ Head Teaching Assistant:

▣ WU Zhenghao



Virtual Learning Environment (VLE)

- We will use **Brightspace** as the VLE for this module.
- This will be used for:
 - Making learning materials available.
 - Discussions on the discussion forum.
 - Announcements.
- Log in to <https://brightspace.ucd.ie> using your UCD Connect username and password.
 - If you are correctly registered, you should automatically have access (<https://brightspace.ucd.ie/d2l/home/302586>).





COMP3009J-Information Retrieval-2024/25 Spring

Calendar

Sunday, 16 February 2025



Upcoming events



There are no events to display. [Create an event](#)

Welcome to the Module

Updates

There are no current updates for
COMP3009J-Information
Retrieval-2024/25 Spring

My Announcements

There are no announcements to display.
[Show All Announcements](#)



COMP3009J-Information Retrieval-2024/25 Spring

Calendar ▾

- All announcements about this module will be made on Brightspace.
- You can configure Brightspace to emails you about announcements.
 - This can be either to a UCD email account or to a personal account.
 - I've put a link to the instructions for this on Brightspace.

private or

Welcome to the Module

Updates ▾

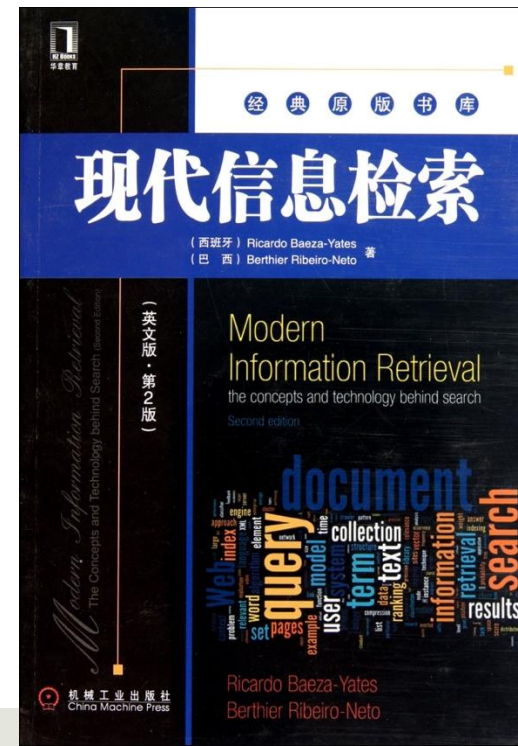
There are no current updates for
COMP3009J-Information
Retrieval-2024/25 Spring

My Announcements ▾

There are no announcements to display.
[Show All Announcements](#)

Resources

- **Modern Information Retrieval** (2nd Edition) by Ricardo Baeza-Yates and Berthier Ribeiro-Neto (ISBN: 978-7-111-33174-2): in the library.
- **An Introduction to Information Retrieval** by Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze: will be posted on Brightspace.
- Academic Papers from conference proceedings (SIGIR, ECIR, TREC etc.): will be posted on Brightspace.



Assessment

- Your final grade for this module will be based on:
- Programming Assignment (30% of final grade)
 - Before the programming assignment, we will use programming worksheets to prepare you for this task.
- Written Final Exam (70% of final grade)
 - Closed-book written examination
 - Held during exam period at end of semester (Weeks 17-18)
- Final grades are calculated according to UCD's regulations:
 - <https://www.ucd.ie/students/exams/gradingandremediation/understandinggrades/>

Module Timetable

- **Week 1 and 2 (2 lectures):**

- Lecture: Monday 09:55 in Room 511, Teaching Building 3
- Lecture: Wednesday 09:55 in Room 214, Teaching Building 4

- **Week 3 and 4 (1 lecture and 1 lab):**

- Lecture: Monday 09:55 in Room 511, Teaching Building 3
- Lab: Wednesday 09:55 in Room 214, Teaching Building 4

- **Week 5 (1 lab)**

- Lab: Wednesday 09:55 in Room 214, Teaching Building 4

- **Week 6 and later (1 lecture and 1 lab):**

- Lecture: Monday 09:55 in Room 511, Teaching Building 3
- Lab: Wednesday 09:55 in Room 214, Teaching Building 4

- **Schedule may change due to holiday arrangements/sports day/etc.**

Face to Face Lectures

- A lecture is **not** just PowerPoint slides. It also consists of:
 - Everything I say during the lecture.
 - Any work that I do on the blackboard or projector.

- Every student should bring a notebook (paper or digital) to class:
 - Take notes on things that are not in the lecture slides.
 - Write down your own understanding of key points.
 - You remember more if you write things down.
 - Relying only on typing for notes is not advised – you will need to draw diagrams too.

Labs and Attendance

- Labs are scheduled to begin in Week 3.
 - At the beginning, there will be weekly “worksheets” for you to work on.
 - Although these are not graded, they will help you to understand the lecture content better, and prepare you for the programming assignments and the final exam.
 - It is important to complete these early worksheets if you want to well on the later graded assignment.
 - During later labs, you will work on the programming assignment.
- It is expected that you attend all lectures and labs.

Discussion Forum

- Outside class times, We will use the **Brightspace Discussion Forum** for questions.
 - This is better than email because then everyone can see the answers to common questions.
 - **Anybody** can **ask** a question.
 - **Anybody** can **answer** a question.
 - If you want to, your question can be **anonymous** so that other users will not see your name (only I will see who asked the question)
 - Class representatives may **gather** questions from the class and ask them in the forum.

Discussions

[Discussions List](#)

[Subscriptions](#)

[Group and Section Restrictions](#)

[Statistics](#)

New

More Actions

Filter by: [Unread](#) [Unapproved](#)

[Collapse All Forums](#)

COMP3009J (Information Retrieval) Forum

Forum guidelines:

- All posts must be in English.
- All posts must be related to COMP3009J.
- Any student can ask a question.
- Any student can reply to any question.
- You may post anonymously without using your name.
- When posting code, please do not post photos: post the actual code in text.

If posts are made that do not follow the guidelines above, they may be removed without comment.

Getting Help

- If there is something you have difficulty with, plenty of help is available.
 - The lecturer normally has lots of time to answer questions during lab times.
 - In the lab you will have a Teaching Assistant (TA) who can answer questions that you have, or help to solve programming problems.
 - You can ask the lecturer questions during or after a lecture.
 - Outside of class time, there is a Brightspace Discussion Forum, for questions about the module content.
 - If your question is not suitable for the forum (e.g. is of a personal nature), you can email the lecturer at david.lillis@ucd.ie or rem.collier@ucd.ie (later in the semester).
 - Please make sure to say your **name** and include the **module code** in the subject in your email.
 - Please do not ask questions about module contents, assignments, exams, etc. via email – use the forum.

Programming: Python

- In this module we will use **Python 3** as our programming language.
 - Python 2 is a different language that is *not compatible* with Python 3.
- Please download and install this from <https://www.python.org/downloads/>
 - Latest version is 3.13.2.
- If you have Python already installed but you not know which version it is, run "`python --version`" on your command line.
 - Or sometimes "`python3 --version`"

Programming: Python Editor

- A good **programming editor** is recommended.
- Some suggestions include:
 - Visual Studio Code (Windows, Mac, Linux): <https://code.visualstudio.com/>
 - CotEditor (Mac): <https://coteditor.com/>
 - IDEs like PyCharm, Spyder, IDLE are **not** recommended.
- Jupyter notebooks are also not recommended.

Questions?

- If you have questions now, please ask!
- If you have questions later, please post in the Brightspace discussion forum for COMP3009J.

What is Information Retrieval (IR)?

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

What is Information Retrieval?

- For now, think of an internet search engine.
- As a user, there is some information that you want to know.
- The search engine has access to billions of web pages of information.
 - Its job is to let you know which pages contain the information that you are looking for.
 - **Efficiency and scalability are key!**
- You describe what you want using a **query**.
- We will look at a more formal definition later.

What is Information Retrieval (IR)?

- As computer and internet users, we use IR systems regularly:
 - Web Search Engines (e.g. Baidu, Bing, Google, etc.)
 - Desktop Search
 - Mobile Search
 - Library Catalogue Searches
 - Searching Individual Web Sites (e.g. newspaper archives, company document repositories, etc.)

What is Information Retrieval?

Information Retrieval (IR) deals with the **representation, storage, organization of**, and **access to** information items.

The representation and organization of the information items should provide the user with easy access to **the information in which he is interested**.

- *Baeza-Yates and Ribeiro-Neto*

What is Information Retrieval?

Information Items

- This definition provides us with a number of issues that need to be addressed when considering what constitutes an IR system:
 - **Information Item:** Historically, “information items” have been books, documents or other written material that contain information in an **unstructured form**.
 - In IR, we frequently use the word “document” to refer to these items. However, in more modern times, IR systems have developed to cover other forms of information, and may be asked to include such things as video, images, etc.

What is Information Retrieval?

Structured vs Unstructured Data

- Structured data tends to refer to information in “tables” (e.g. database, spreadsheet).

Employee	Manager	Salary
Smith	Jones	50000
Chang	Smith	60000
Ivy	Smith	50000

Typically allows numerical range and exact match (for text) queries, e.g.,

Salary < 60000 AND Manager = 'Smith'.

What is Information Retrieval?

Structured vs Unstructured Data

- Unstructured data:
 - Typically refers to **free-form text** written in **natural language**.
 - “Natural” languages are those used by humans to communicate, e.g. English, Mandarin Chinese, French, etc.
 - Allows for:
 - **Keyword queries**, including operators (AND, OR, NOT, etc.)
 - More sophisticated “**concept**” **queries**, e.g.,
 - “find all web pages dealing with *drug abuse*”.
 - In reality, most documents are **semi-structured** (e.g. we can identify the headings, titles and body text).
 - Also, it often fits the structure of the language it is written in.

What is Information Retrieval?: Representation

- **Representation:** In order to allow users to perform searches, documents must be **represented** in some way.
 - Searching through the raw text of millions of documents is a **very slow** process, so some **mathematical representation** of the information is typically used.
 - **Why?**
 - Computers are much quicker at doing mathematical calculations than they are at text processing.
 - We try to represent textual documents in some mathematical form so retrieval is faster (e.g. vectors, sets, bit-strings).

What is Information Retrieval?

Storage, Organisation and Access

- **Storage and Organisation:** There are important considerations in terms of storing documents also. They must be organised so as to be **quickly accessible**. This is particularly important with large document collections.
- **Access:** The key attribute of an IR system is to allow users to access the information in question. This should happen in a **timely** and **efficient** manner.

What is Information Retrieval?

Information Need

The representation and organization of the information items should provide the user with easy access to **the information in which he is interested**.

- ▣ Users only use IR systems when there is some information that they are interested in reading.
- ▣ We call this the "**information need**" of a user.
- ▣ This phrase was coined by Robert Taylor in 1962, and is defined as having four stages.

Information Need: 4 Stages

1. the actual, but unexpressed need for information (**visceral need**): “a vague sort of dissatisfaction ... probably inexpressible in linguistic terms”.
2. the conscious within-brain description of the need (**conscious need**): “an ambiguous and rambling statement”. i.e. “I know what I’m looking for but I don’t know how to say it”.
3. the formal statement of the question (**formalised need**)
4. the question as presented to the information system (**compromised need**)

Expressing an Information Need

- The third stage, the *formalised need*, is typically easy to express in a natural language:
 - What is the capital city of Cyprus?
 - Who are the 10 best golfers in the world?
 - What common arguments are made in favour of, or against, the use of nuclear power?
 - What were the effects of Joe Biden's policies as President of the USA?
- Some information needs are simple; others are much more complex.

Presenting an Information Need to an IR System: Queries

- A **query** is the expression of an information need (*compromised need*) that is provided to an IR system to explain what information is required by the user. There are several different methods of expressing queries:
 - **Keyword-Based Querying:** A keyword (or list of keywords) is supplied to the IR system. This is by far the most common form of querying in web search (but this is changing). The average query consists of 2-3 keywords.
 - **Context Queries:** These specify sequences of words that should appear close together in documents that are retrieved. Physical proximity has semantic value.

Presenting an Information Need to an IR System: Queries

■ Types of Queries (continued)

- **Boolean Queries:** The oldest method of combining keywords allows a user to specify keywords that should or should not be contained in the documents that are desired, using the Boolean operators AND, OR and NOT.
 - e.g. “information AND (retrieval OR extraction) NOT data”.

Presenting an Information Need to an IR System: Queries

■ Types of Queries (continued)

- **Natural Language Queries:** Queries are provided in their natural form. In the past, this form of querying has not been very successful, as Natural Language Processing is a difficult task.
 - For a long time, <http://www.askjeeves.com> was the most famous natural language web search engine, but it reverted to keyword-based searching in 2006.
 - Microsoft spent \$100m to acquire <http://www.powerset.com> in 2008 in a new attempt at the concept.
 - Other examples: hokia (closed in 2014), Lexxe (closed 2015)
 - Now, many search engines will try to automatically detect natural language queries, but support keyword search also.
 - Most voice-activated interfaces support natural language queries (e.g. Apple's Siri, Amazon's Alexa), although their understanding is often quite basic.
 - More recently (2023), Microsoft and OpenAI announced that they will integrate the Bing search engine with the ChatGPT language model, which will allow for more conversational search. This product is known as Microsoft Copilot.

The Role of an IR System

The Role of an IR System

An information retrieval system does not inform (i.e. change the knowledge of) the user on the subject of his inquiry. It merely informs on the **existence** (or non-existence) and **whereabouts** of documents relating to his request.

– C. J. van Rijsbergen

- **Existence:** Tells the user that a document exists that helps to satisfy the information need.
- **Whereabouts:** Tells the user where that document is.
- The user learns nothing unless they then read the document.

The Role of an IR System

- This helps us to define the role of an IR system:
 - The principal function of an IR system is to give users a **list of documents** that are **relevant** to the user's information need.
 - It makes no attempt to take a subset of the information contained in a document and present that to the user: it is the task of the searcher to do this.
 - With most systems, this list is ranked, with the document the system believes to be of most relevance at the top of the list.



information retrieval

Query

Search

[Advanced Search](#)

Web [+ Show options...](#)

Results 1 - 10 of about 6,510,000 for **information retrieval**. (0.17 seconds)

[Information retrieval - Wikipedia, the free encyclopedia](#)

Information retrieval (IR) is the science of searching ~~for~~ documents, for information within documents, and for metadata about documents, as well as that of ...

[History](#) - [Overview](#) - [Performance measures](#) - [Model types](#)

[en.wikipedia.org/wiki/Information_retrieval](#) - [Cached](#) - [Similar](#)

[Information Retrieval](#)

An online book by CJ van Rijsbergen, University of Glasgow.

[www.dcs.gla.ac.uk/Keith/Preface.html](#) - [Cached](#) - [Similar](#)

[Information Retrieval](#)

Information Retrieval Group Department of Computing Science University of Glasgow Glasgow G12 8QQ SCOTLAND. [keith@dcs.gla.ac.uk](#) ...

[www.dcs.gla.ac.uk/~iain/keith/](#) - [Cached](#)

[Introduction to Information Retrieval](#)

The book aims to provide a modern approach to **information retrieval** from a computer science perspective. It is based on a course we have been teaching in ...

[www.csli.stanford.edu/.../information-retrieval-book.html](#) - [Cached](#) - [Similar](#)

[Information Retrieval](#)

Computer Science - Database Management & **Information Retrieval** ... The Journal of **Information Retrieval** is an international forum for theory, ...

[www.springer.com/.../information+retrieval/journal/10791](#) - [Cached](#) - [Similar](#)

[Information Retrieval](#)

[www.springerlink.com/link.asp?id=103814](#) - [Similar](#)

[Information retrieval: algorithms and heuristics - Google Books Result](#)

David A. Grossman, Ophir Frieder - 2004 - Computers - 332 pages

The authors answer these and other key information retrieval design and implementation questions. This book is not yet another high level text.

[books.google.com/books?isbn=1402030037...](#)

[Modern Information Retrieval](#)

A recent IR book, covering algorithms, implementation, query languages, user interfaces, and multimedia and web **retrieval**.

[people.ischool.berkeley.edu/~hears/irbook/](#) - [Cached](#) - [Similar](#)

[Information retrieval: uncertainty and logics: advanced models - Google Books Result](#)

Links to Documents
Containing Relevant
Information

Why is Information Retrieval Important?

- The term “Information Overload” coined by Alvin Toffler as far back as 1970.
- This describes the situation where there is so much information being made available to us that it is impossible for people to absorb all of it.
- This problem has become much worse as a result of the Internet, with millions of people publishing material every day.
- IR systems are vital in providing users with access to this information
- Reliable sources on how many queries search engines like Baidu and Google handle per day are difficult to find. But most sources agree that they handle **billions** of search queries daily. Approximately 15% of Google queries have never been seen before.

Google Searches over Time

Year	Number of Searches (approx)
1998	1,000,000,000 (1 billion)
2000	14,000,000,000 (14 billion)
2001-2003	Over 55,000,000,000 (55 billion)
2004-2008	73,000,000,000 (73 billion)
2009	Over 365,000,000,000 (365 billion)
2012-2015	1,200,000,000,000 (1.2 trillion)
2016	Over 2,000,000,000,000 (2 trillion)

Source: <https://searchengineland.com/google-now-handles-2-999-trillion-searches-per-year-250247>

Relevance

- This concept of “**relevance**” is very important in IR:
 - A document is “relevant” if its content satisfies (or helps to satisfy) the user’s information need.
 - The system alerts the user to the existence and location of a document: it’s up to the user to read the document.
 - It is important to note that this is, in theory, a **subjective** concept, that is entirely up to the user to judge.
 - Remember that a query is just an expression of an information need.
 - For example, if a user searches for “jaguar”, documents that discuss luxury cars may **appear to be relevant**, but will be of no use to a user who is researching big cats.

How do we know if the system is good?

- An IR system has been successful if the documents it returns satisfy the information need.
- The only person qualified to judge this is the user, which is a problem when trying to evaluate IR systems.
- Laboratory experiments use “test collections” for which **standard queries** have been created.
- Documents in these collections have been **judged** by human judges as to whether or not they are relevant to each query.
- This method becomes more difficult as document collections grow.

Related Research

There are a number of other research areas that are not traditionally considered to be “Information Retrieval”, but are closely related.

Question Answering

- **Question Answering:** Here, the system is designed to provide a full answer to a question, e.g.:
 - Question: What is the capital city of France?
 - Answer: Paris is the capital city of France.
- Many mainstream search engines include some element of question answering as well as their Information Retrieval system.
- Over time, as search engines have gained more functionality, the distinction between IR and related tasks has become much more unclear.
- Many Large Language Models (e.g. ChatGPT) attempt to do Question Answering (i.e. giving answers directly).

Question Answering

The image shows a Google search results page for the query "what is the capital city of france". The search bar at the top contains the query, and the results show "About 69,300,000 results (0.24 seconds)".

On the left side, there is a sidebar with navigation links: "Everything", "Images", "Videos", "News", "Shopping", and "More". Below these are "All results", "Timeline", and "More search tools".

Two green boxes highlight specific search results:

- The first box highlights the top result: "France Capital city — Paris - Feedback". Below this, it says "According to wikipedia.org, placesinfrance.com, mapsofworld.com and 7 others -" and includes a link to "Show sources".
- The second box highlights a result from Wikipedia: "Paris - Wikipedia, the free encyclopedia". It provides coordinates and a brief description: "Coordinates: 48°51'24"N 2°21'03"E / 48.856667°N 2.350833°E / 48.856667; 2.350833. Paris is the capital and largest city in France, situated on the river ...". It also includes links for "History", "Etymology", "Geography", "Cityscape", and "en.wikipedia.org/wiki/Paris".

Two annotations are present:

- A green arrow points from the text "This is not Information Retrieval" to the first highlighted result box.
- A red arrow points from the text "This is" to the second highlighted result box.

The search results also include other links such as "Paris the Capital City of France and Facts", "Travel Guide To Paris Capital City Of France", "France Capital , Capital of France - Paris", and "Answers.com - What is the capital of France".

Information Extraction

- An **Information Extraction** system is designed to take unstructured text and try to create structured data from it.
- **Input:**
 - “Kate Hudson was born in Los Angeles, California, the daughter of Academy Award-winning actress Goldie Hawn and Bill Hudson, an actor, comedian, and musician. Her parents divorced eighteen months after her birth; she and her brother, actor Oliver Hudson, were raised in Colorado by her mother and her mother’s long-time boyfriend, actor Kurt Russell.”
- **Output (examples):** mother(Kate Hudson, Goldie Hawn), brother(Kate Hudson, Oliver Hudson), gender(Kate Hudson, female)

Summary

- We have seen:
 - What is Information Retrieval (definition)?
 - Information Items (unstructured: representation, storage, organization and access)
 - Information Need (expressed as queries of various types)
 - Relevance
 - What is not Information Retrieval?
 - Question Answering
 - Information Extraction

An Information Retrieval Example: Boolean Queries

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

See Chapter 1 of Manning, Raghavan and Schütze book.

Example

- Aims:

- See a simple example of an Information Retrieval task using the **Boolean Model** (i.e. based on whether a document does or does not contain particular words, or combinations of words).
- Begin to explore why **representation** is important (i.e. what sort of mathematical structures do we use to represent documents and queries).

Example: the Boolean style query

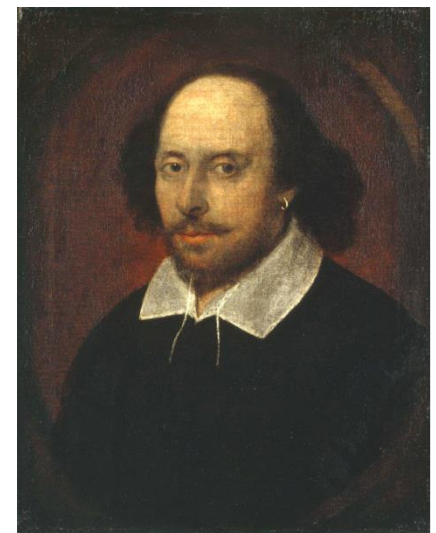
- William Shakespeare was a famous English playwright. Suppose we want to search for characters in his plays.
- Which plays of Shakespeare contain the words **Brutus** AND **Caesar** but NOT **Calpurnia**?
- As a first attempt, we could read the plays line-by-line to find those that contain **Brutus** and **Caesar** and then remove those that contain **Calpurnia**.



Example: the Boolean style query

- “... we could read the plays line-by-line to find those that contain **Brutus** and **Caesar** and then remove those that contain **Calpurnia**.”
- Why is that not the answer?
 - Slow (for large corpora)
 - Other operations (e.g., find the word **Romans** near **countrymen**) not feasible.
 - Ranked retrieval not possible (best documents shown first)
 - Later lectures!

A **corpus** is a collection of documents. The plural is **corpora**.



Example: The Boolean style query

Term-document incidence matrix

- Instead, we might store the information we need in some sort of data structure. Here is a **term-document incidence matrix** showing some of the words contained in some of Shakespeare's plays.
- Or course this is a small simple example. Shakespeare's plays contain 1,223 characters in total.

		Plays (Documents)					
		Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Words (Terms)	Antony	1	1	0	0	0	1
	Brutus	1	1	0	1	0	0
	Caesar	1	1	0	1	1	1
	Calpurnia	0	1	0	0		
	Cleopatra	1	0	0	0		
	mercy	1	0	1	1		
	worser	1	0	1	1		

1 if the play contains the word, 0 otherwise.

Example: The Boolean style query

Incidence vectors

- So we have an **incidence vector** for each word.
 - It consists of 1s (for the plays it appears in) and 0s (for those it does not appear in).
 - e.g.
 - **Brutus:** 110100
 - **Caesar:** 110111
 - **Calpurnia:** 010000

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

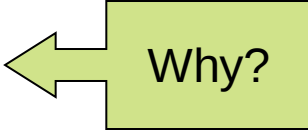
Example: The Boolean style query Operators

- Our query is: **Brutus AND Caesar NOT Calpurnia**
- To get “**NOT Calpurnia**”, we get the **complement** of the incidence vector for **Calpurnia** (using the bitwise NOT operator, which changes all 1s to 0s and all 0s to 1s).
 - $001000 \rightarrow 110111$
- Now we can use the bitwise AND operator to combine our three vectors:
 - $110100 \text{ AND } 110111 \text{ AND } 101111 = 100100$
- Referring back to the term-document incidence matrix, we see that the answer is: **Antony and Cleopatra, Hamlet**

Bigger collections

- This approach can be effective, but how well does it **scale** to larger collections?
 - Consider $N = 1,000,000$ documents, each with about 1,000 words.
 - Average 6 bytes per word including spaces and punctuation.
 - 6GB of data in the documents.
 - Say there are $M = 500,000$ *distinct* terms among these.

Can't build the matrix

- 500,000 x 1,000,000 matrix has **half-a-trillion** 0s and 1s.
- But it has no more than one **billion** 1's.  Why?
 - matrix is extremely **sparse**.
- What's a better representation?
 - We only record the 1 positions, and not the 0s.
 - We will look at this in a later lecture.

Another problem: Ambiguous Queries

- Sometimes it is difficult to figure out what the information need was if we can only see the query: some queries are **ambiguous**.
- For example, if a user searches for “jaguar”, documents that discuss luxury cars may appear to be relevant, but will be of no use to a user who is researching big cats.
- Similarly, a search for “bank” could be:
 - A river bank.
 - A financial institution.
 - A manoeuvre made by an aeroplane.

Questions

Indexing

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

See Chapter 1 of Manning, Raghavan and Schütze book.
Some slides adapted from P. Nayak and P. Raghavan

Indexing

- Aim:
 - Continue to explore the Boolean Model – in particular more effective ways of **representation**.
 - We will get the same (correct) answer, but using more efficient approaches.

Introduction: Indexing

- So that we can search a document collection, we must represent the documents in some type of appropriate data structure.
- We frequently refer to this data structure as an **index**, and the process of creating this data structure is called **indexing**.
- As we saw previously, an incidence matrix is one possibility, but does not scale well for larger document collections.

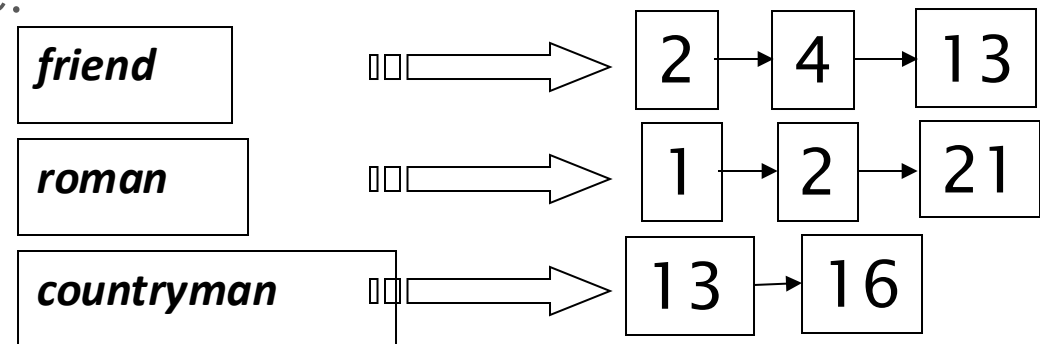
	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Inverted Index

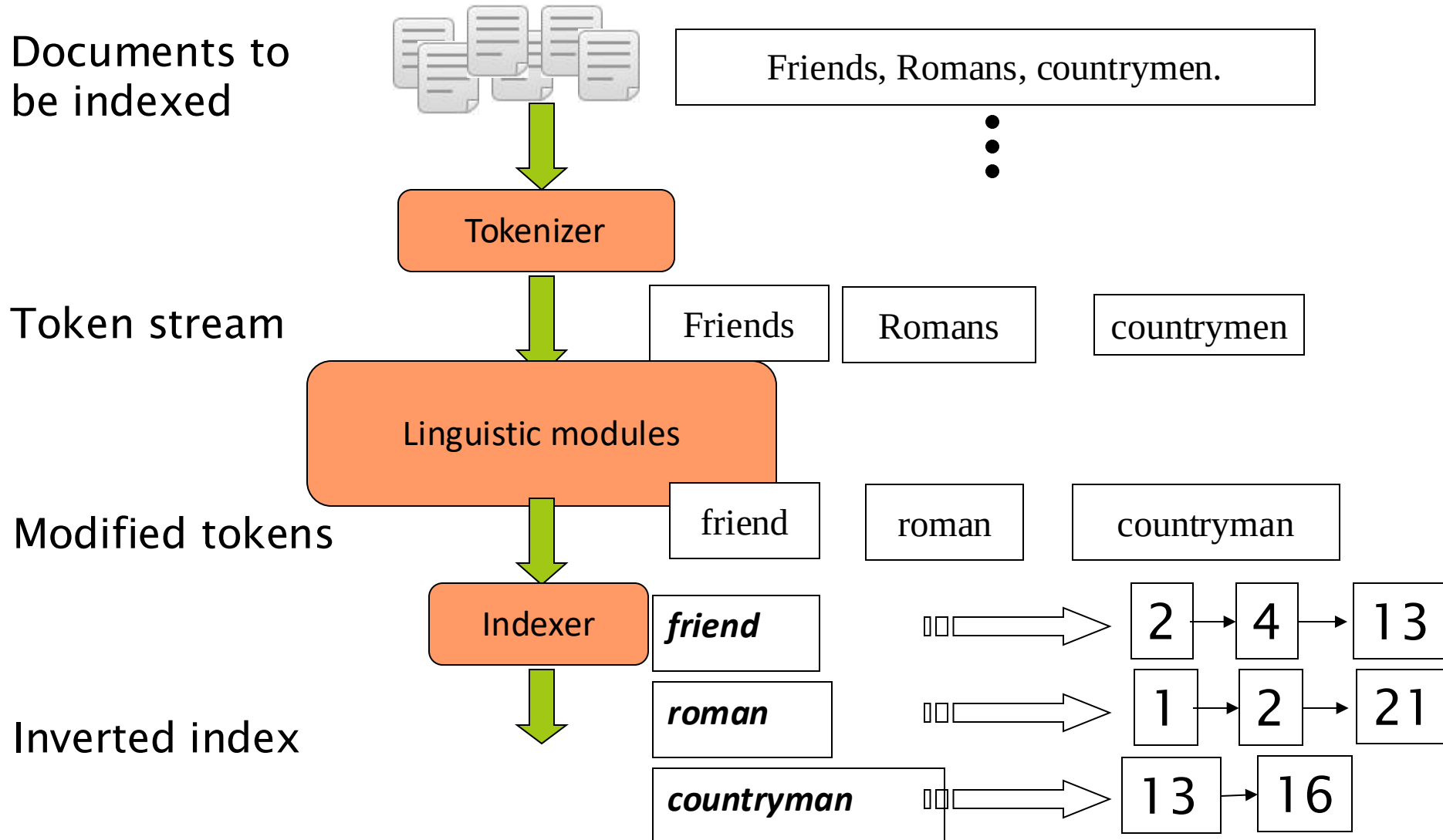
- An **inverted index** is a type of data structure where each term t is mapped to a list of documents that contain t .
 - We identify each document using a **docID**: a unique identifier for each document.
- We need variable-sized list for each term, called a **postings list**. A linked list would be suitable.
 - Each docID in the list is a **posting**.
- Each list should be **ordered** by the **docID**.
 - We will see the reason for this later.

What is a **term**?

For now, we can think of a term as being a word. Later we will see that they are slightly different.



Construction of an Inverted Index



1. Split the text into **tokens** (generally words)

2. Tokens must be processed further, according to the language the text is in (more on this later).

Here, we notice that they are converted to lowercase. Why?

3. Then, we create the inverted index.

Indexer step 1: Token sequence

- Sequence (list) of pairs: (token, docID).
- As part of this process, punctuation is generally removed.
- In this example, we also convert to lowercase.

Doc 1

I did enact Julius
Caesar I was killed
i' the Capitol;
Brutus killed me.

Doc 2

So let it be with
Caesar. The noble
Brutus hath told you
Caesar was ambitious

Term	docID
i	1
did	1
enact	1
julius	1
caesar	1
i	1
was	1
killed	1
i	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Indexer step 2: Sort

- Sort this sequence by the terms (alphabetically) and then by the docIDs.

Term	docID
i	1
did	1
enact	1
julius	1
caesar	1
i	1
was	1
killed	1
i	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2



Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
i	1
i	1
i	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Indexer step 3: Dictionary & Postings

- If a term appears more than once in a document, these entries are merged (i.e. a term is recorded only once for each document).
- Split into a **dictionary** (alphabetical **order** of terms) and **postings** (**ordered** list of documents each term appears in).
 - Why is ordering important?
- We also record the **document frequency** of each term (the number of documents it appears in).
 - This information will be useful later.

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
i	1
i	1
i	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2



term	doc. freq.	→	postings lists
ambitious	1	→	[2]
be	1	→	[2]
brutus	2	→	[1] → [2]
capitol	1	→	[1]
caesar	2	→	[1] → [2]
did	1	→	[1]
enact	1	→	[1]
hath	1	→	[2]
i	1	→	[1]
it	1	→	[2]
julius	1	→	[1]
killed	1	→	[1]
let	1	→	[2]
me	1	→	[1]
noble	1	→	[2]
so	1	→	[2]
the	2	→	[1] → [2]
told	1	→	[2]
you	1	→	[2]
was	2	→	[1] → [2]
with	1	→	[2]

Using the Inverted Index: How do we process a query?

- Consider the following query:

- **Brutus** AND **Caesar**

1. Locate **Brutus** in the dictionary and retrieve its postings.

- What is the time complexity for finding **Brutus** in our dictionary?

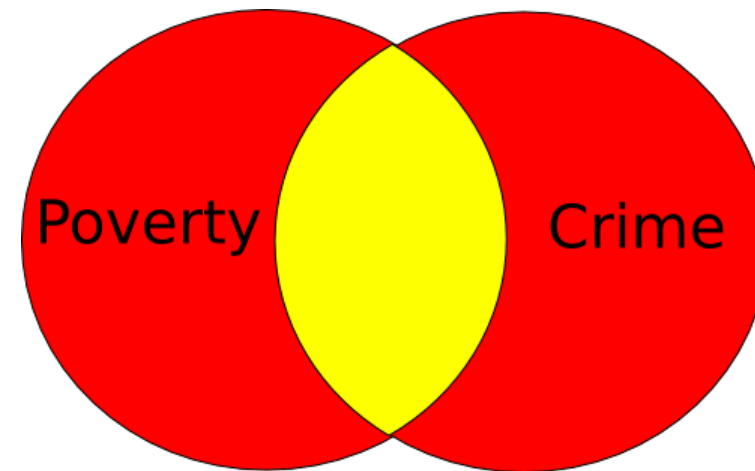
2. Locate **Caesar** in the dictionary and retrieve its postings.

3. Merge the two postings (find the **intersection**) of the document sets.

- Each of our boolean operators (AND, OR, NOT) is equivalent to a set operation.

Set Theoretic Representation of AND

If P is the set of documents that contain the term “**poverty**” and C is the set of documents that contain the term “**crime**”, the query “**poverty AND crime**” can be calculated by $P \cap C$ (i.e. the area in yellow).

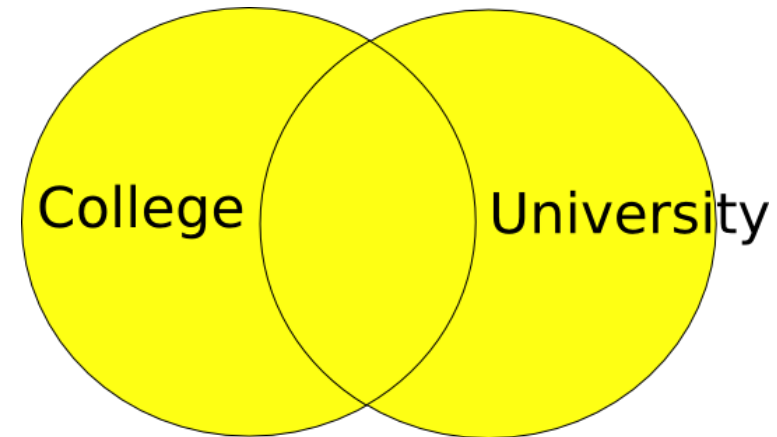


Boolean Operators: AND

- **AND** is used to **narrow** a search by ensuring that all the search terms should appear in the results.
- It is commonly used to search for relationships between two concepts or terms.
- The more terms or concepts combined in a search with AND, the fewer records that will be retrieved.

Set Theoretic Representation of OR

If C is the set of documents that contain the term “**college**” and U is the set of documents that contain the term “**university**”, the query “**college OR university**” can be calculated by $C \cup U$. (i.e. the area in yellow)

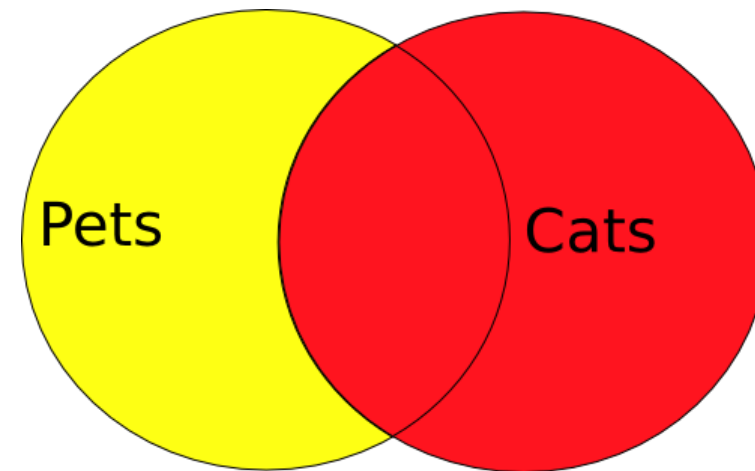


Boolean Operators: OR

- **OR** is used to **broaden** a search by retrieving any, some or all of the keywords used in the search statement.
- It is commonly used to search for synonymous terms or concepts.
- Note that the final result is not the same as the sum of the individual results: documents containing both terms (e.g. “University College Dublin”) will only be counted once.
- The effect of using OR is that documents containing **any number of the terms specified** will be returned.

Set Theoretic Representation of NOT

If P is the set of documents that contain the term “**pets**” and C is the set of documents that contain the term “**cats**”, the query “**pets NOT cats**” can be calculated by $P \setminus C$ (i.e. the area in yellow).

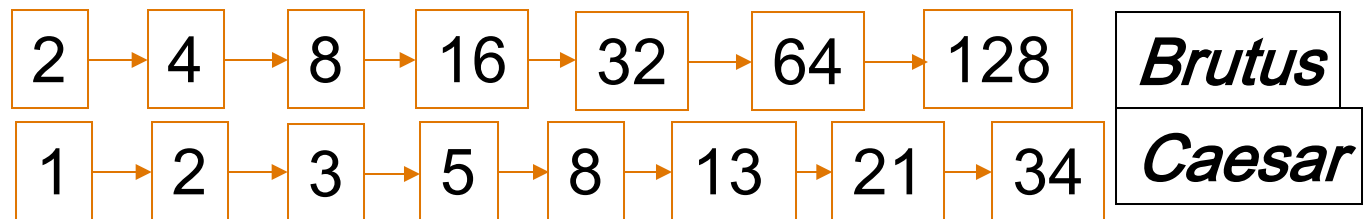


Boolean Model Operators: NOT

- **NOT** is used to **specifically exclude** a term from your search.
- One difficulty with using NOT is that a document that is highly relevant to what you're searching for may also contain the term you had attempted to avoid.
- Again, the more terms or concepts combined in a search with NOT, the fewer records that will be retrieved.

Merging postings lists

- How would you merge two postings lists into one to get their intersection (i.e. an AND query)?
 - Hint:** Why is it important that the lists are sorted?



Merging two postings lists: Intersection (AND)

INTERSECT(p_1, p_2)

```
1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then ADD(answer,  $\text{docID}(p_1)$ )
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return answer
```

- If lengths are x and y , algorithm is $O(x+y)$. **Why?**
- **Answer:**
 - Each iteration of the loop advances to the next document in at least one of the lists.
 - Worst case (two distinct lists with nothing in common):
 - x iterations to reach end of first list.
 - y iterations to reach end of second list.
 - $O(x+y)$
 - This **only** works because the lists are sorted by docID.

Exercise

- Can we adapt the merge for the following query types? How?
 - **Brutus OR Caesar**
 - Documents that contain either the term **Brutus** or the term **Caesar**, or both.
 - **Brutus AND NOT Caesar**
 - Documents that contain the term **Brutus** but do not contain the term **Caesar**.
 - **Brutus OR NOT Caesar**
 - Documents that contain **Brutus** or do not contain **Caesar**.
- If not, what else do we need?

Questions?

Query Optimisation

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

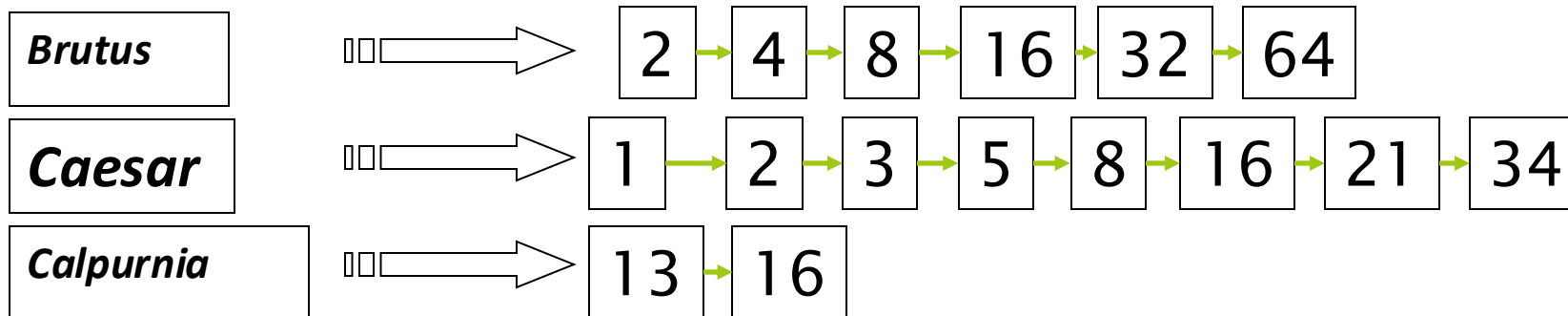
UCD School of Computer Science
Beijing Dublin International College

Query Optimisation

- Aim:
 - Continue to consider efficiency.
 - How can we change our querying strategy so that we will *likely* get faster results?

Query Optimisation

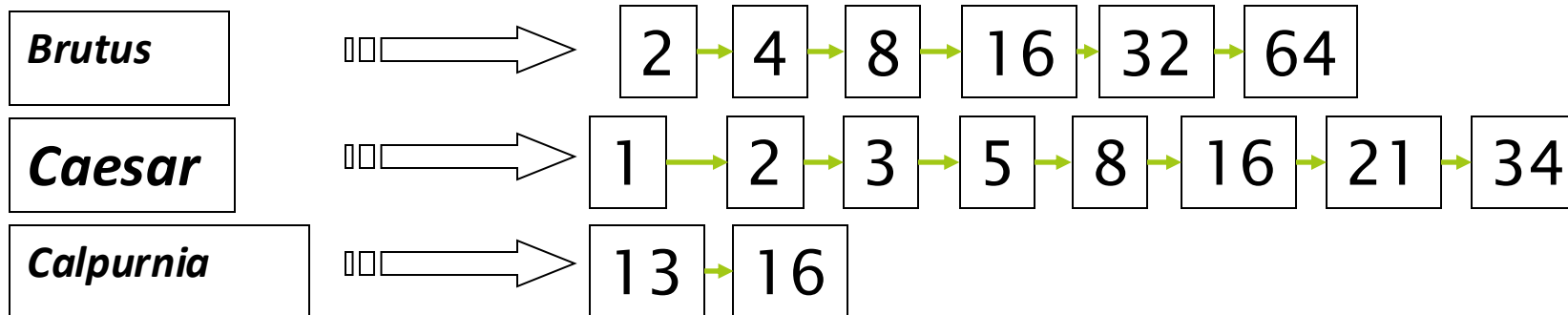
- What is the best order for query processing with postings lists?
- For a query that is an AND of several terms:
 - For each of the terms, retrieve its postings list, and merge with the others.



- Query: **Brutus** AND **Calpurnia** AND **Caesar**

Query Optimisation

- Process in order of increasing frequency.
 - Start with the smallest set, then work towards the bigger ones.
 - **Why?** If we reach the end of the shorter list, we can stop!
 - For a very short list, this can save us a lot of time.
 - This is why we record document frequency in the dictionary.



- Query becomes : (**Calpurnia AND Brutus**) AND **Caesar**

More complex optimisation

- What about a combination of *AND* and *OR* operations?
- E.g. (***madding*** *OR* ***crowd***) *AND* (***ignoble*** *OR* ***strife***)
 - Get the document frequencies for all terms.
 - Estimate the size of each *OR* operation.
 - Maximum possible size is the sum of the frequencies of the terms.
 - Process in increasing order of *OR* size.

Exercise

- Recommend a query processing order for

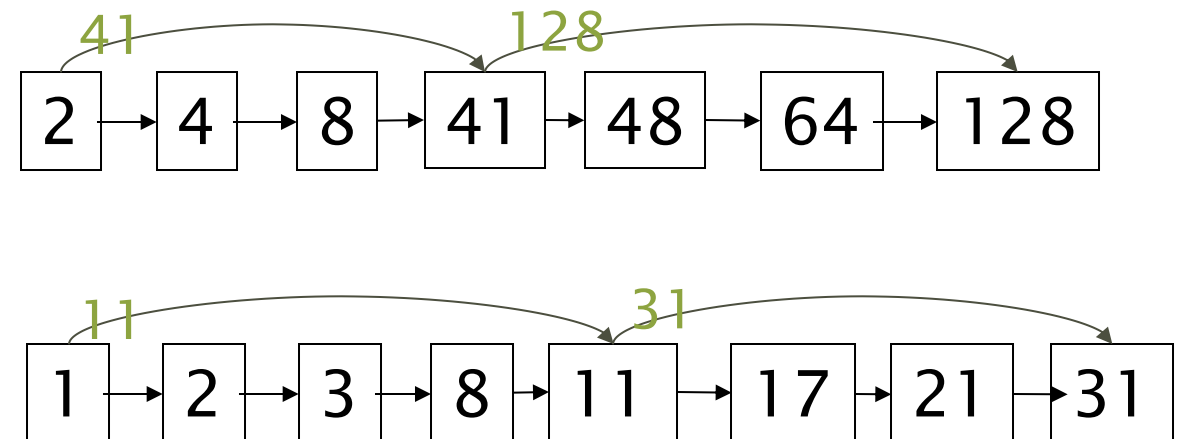
*(tangerine OR trees) AND
(marmalade OR skies) AND
(kaleidoscope OR eyes)*

Term	Freq
eyes	213312
kaleidoscope	87009
marmalade	107913
skies	271658
tangerine	46653
trees	316812

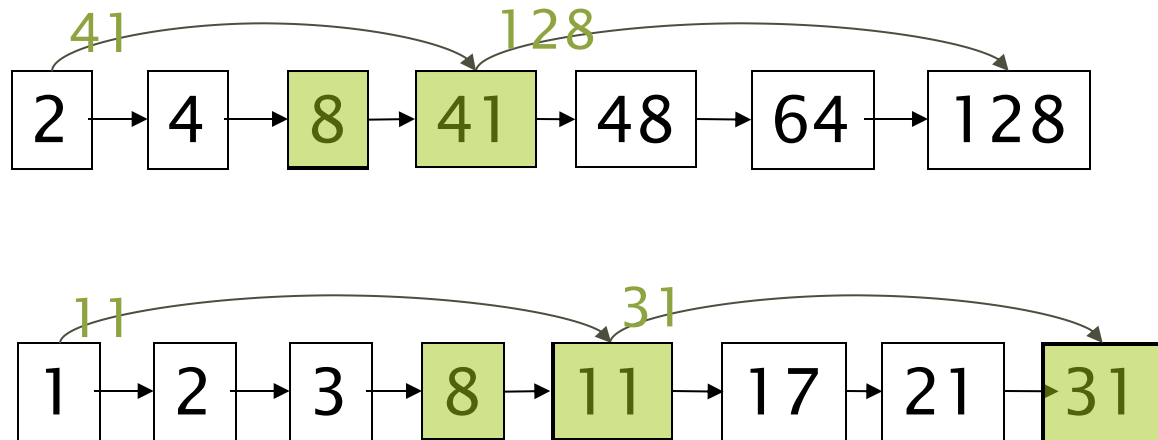
- Which two terms should we process first?

A more optimised data structure: Skip Pointers

- In a regular linked list, each node (which in this case contains a posting) has a pointer/reference to the next node in the list.
- By introducing a second type of reference/pointer, called a **skip pointer**, we can reach later parts of the list without iterating through every element.
- This helps to skip postings that we know will not be in the final search results.



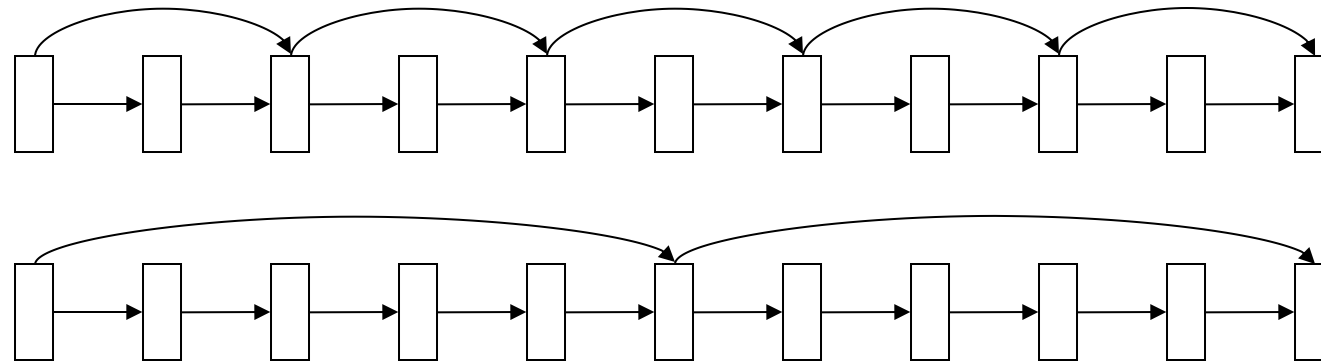
Query processing with skip pointers



- ▣ Suppose we have stepped through both lists until we process **8** on each list. We match it and advance.
- ▣ We then have **41** on the upper list and **11** on the lower. **11** is smaller.
- ▣ But the skip pointer of 11 in the lower list is 31. This is still less than 41 so we can skip everything in between, since we know they cannot be part of the answer.
- ▣ Again, this only works if the list is **sorted**.

Where do we place skips?

- A key word for this semester: “Tradeoff”:
 - More skips $\rightarrow$ shorter skip spans $\Rightarrow$ more likely to skip. But lots of comparisons to skip pointers.
 - Fewer skips $\rightarrow$ few pointer comparison, but then long skip spans $\Rightarrow$ few successful skips.



Placing skips

- Simple heuristic: for postings of length L , use $\sqrt{L}$ evenly-spaced skip pointers [Moffat and Zobel 1996]
- This ignores the distribution of query terms.
- Easy if the index is relatively static; harder if L keeps changing because of updates.
- This definitely used to help; with modern hardware it may not unless you have a memory-based index [Bahle et al. 2002]
 - The I/O cost of loading a bigger postings list can outweigh the gains from quicker in memory merging!

Questions

Preprocessing

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

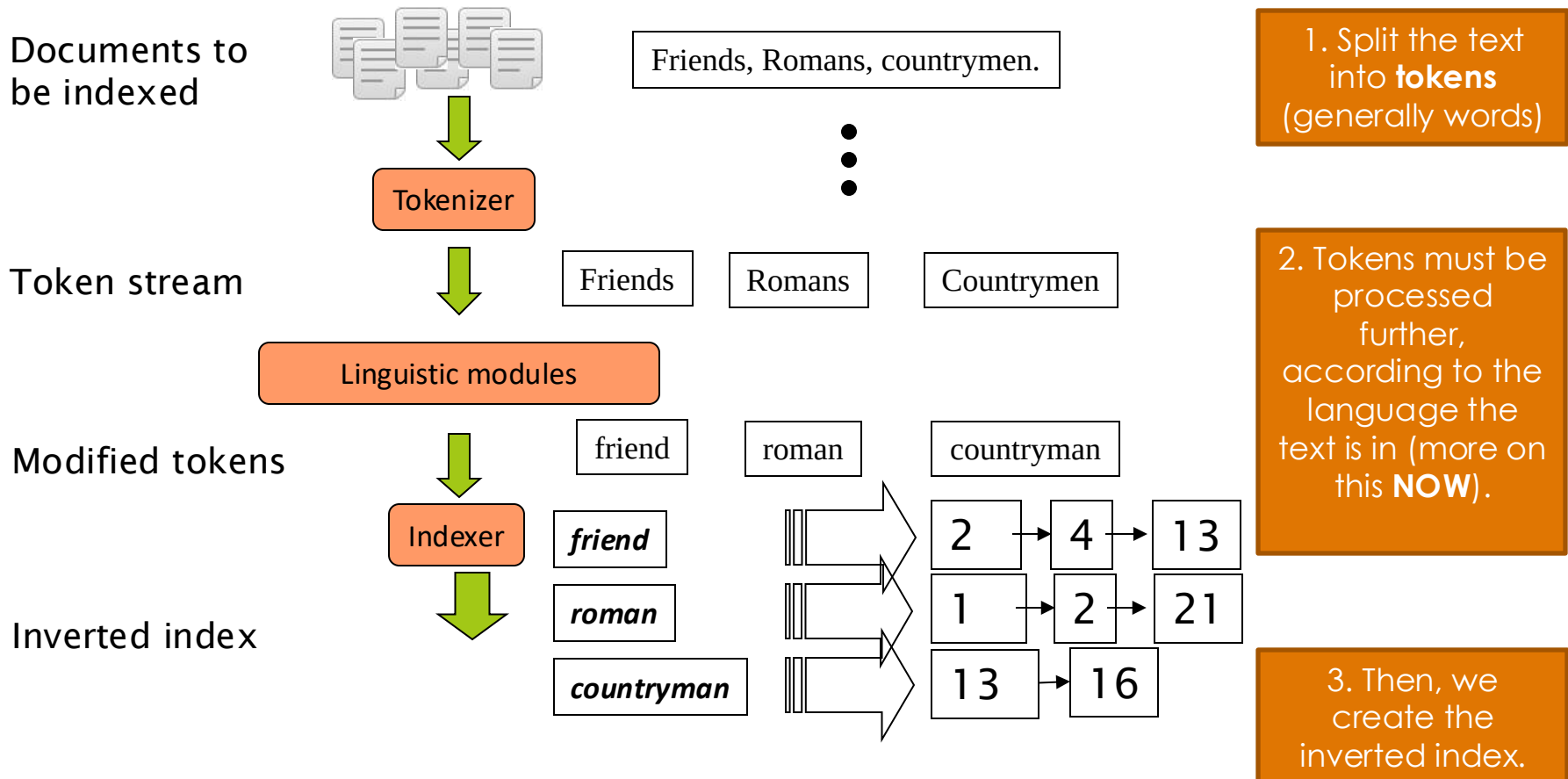
UCD School of Computer Science
Beijing Dublin International College

Preprocessing

- **Aim:**

- Investigate some of the challenges of dealing with natural language when creating index terms.

Remember: creating an Index



Tokenisation (or “Tokenization” if you’re American)

- **Input:** “Friends, Romans and Countrymen”
- **Output:** Tokens:
 - *Friends*
 - *Romans*
 - *Countrymen*
- A **token** is an instance of a sequence of characters. In the previous lecture we said they were similar to words, but they are not the same.
- Each such token is now a candidate for storing as an index entry, after **further processing**. We refer to this as **preprocessing** as it occurs before queries are processed by the system.
 - When we store a token in an index, we call it a **term**.
 - **Note:** a term is not always a real word, as we shall see later.
- How can we turn tokens into terms?
 - Libraries are available to deal with most of these situations.

Tokenisation

- Issues in tokenisation:
 - ***Finland's capital*** → *Finland* AND *s*? *Finlands*? *Finland's*?
 - ***Mercedes-Benz*** → ***Mercedes*** and ***Benz*** as two tokens?
 - ***state-of-the-art***: break up hyphenated sequence.
 - ***lowercase, lower-case, lower case*** ?
 - It can be effective to get the user to put in possible hyphens
 - ***San Francisco***: one token or two?
 - How do you decide it is one token?

Numbers

■ 3/20/91

Mar. 12, 1991

20/3/91

■ 55 B.C.

■ B-52

■ *My PGP key is 324a3df234cb23e*

■ (800) 234-2333

- Often have embedded spaces

- Older IR systems may not index numbers

- But often very useful: think about things like looking up error codes/stacktraces on the web

- Will often index “meta-data” separately

- Creation date, format, etc.

Tokenisation: language issues

- French
 - **L'ensemble** → one token or two?
 - **L ? L' ? Le ?**
 - Want **l'ensemble** to match with **un ensemble**
 - Until at least 2003, it didn't on Google
 - Internationalisation!
- German noun compounds are not segmented
 - **Lebensversicherungsgesellschaftsangestellter**
 - 'life insurance company employee'
 - German retrieval systems benefit greatly from a **compound splitter** module
 - Can give a 15% performance boost for German

Tokenisation: language issues

- Chinese and Japanese have no spaces between words:
 - 莎拉波娃现在居住在美国东南部的佛罗里达。
 - Not always guaranteed a unique tokenization.
 - e.g. (烘手)机 vs. 烘(手机)
- Further complicated in Japanese, with multiple alphabets intermingled.



フォーチュン500社は情報不足のため時間あた\$500K(約6,000万円)

Katakana Hiragana Kanji Romaji

End-user can express query entirely in hiragana!

Tokenisation: language issues

- Arabic (or Hebrew) is basically written right to left, but with certain items like numbers written left to right.
- Words are separated, but letter forms within a word form complex ligatures.

استقلت الجزائر في سنة 1962 بعد 132 عام من الاحتلال الفرنسي.

← → ← → ← start

- ‘Algeria achieved its independence in 1962 after 132 years of French occupation.’
- With Unicode, the surface presentation is complex, but the stored form is straightforward.

Normalisation

- We may need to “normalise” tokens so that they become terms in the same form.
 - e.g. we want *USA* and *U.S.A.* to match.
- Some common approaches:
 - Changing all tokens to lowercase.
 - Even if something should have uppercase letters in it, users often type in lowercase anyway.
 - Delete full stops to form terms: *USA, U.S.A.* → *usa*
 - Delete hyphens:
anti-discriminatory, antidiscriminatory → *antidiscriminatory*

Thesauri and soundex

- Do we handle synonyms and homonyms?
 - E.g., by hand-constructed equivalence classes
 - **car** = **automobile** **color** = **colour**
 - We can rewrite to form equivalence-class terms
 - When the document contains **automobile**, index it under **car-automobile** (and vice-versa)
 - Or we can expand a query
 - When the query contains **automobile**, look under **car** as well
- What about spelling mistakes?
 - One approach is Soundex, which forms equivalence classes of words based on phonetic heuristics (i.e. it indexes terms using their sounds rather than their spelling).

Conclusions

- When creating an index from a set of tokens, several challenges are presented.
- These challenges depend on the language that the document collection is written in.
- Several approaches have been proposed to address these problems.
 - For many approaches, a tradeoff is required, as they do not improve retrieval in all circumstances.

Preprocessing: Stopwords, Stemming and Lemmatisation

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Preprocessing

- **Aim:**

- Examine some common commonly-applied preprocessing techniques.
- Observe some tradeoffs – in “solving” one problem, can we sometimes create others?

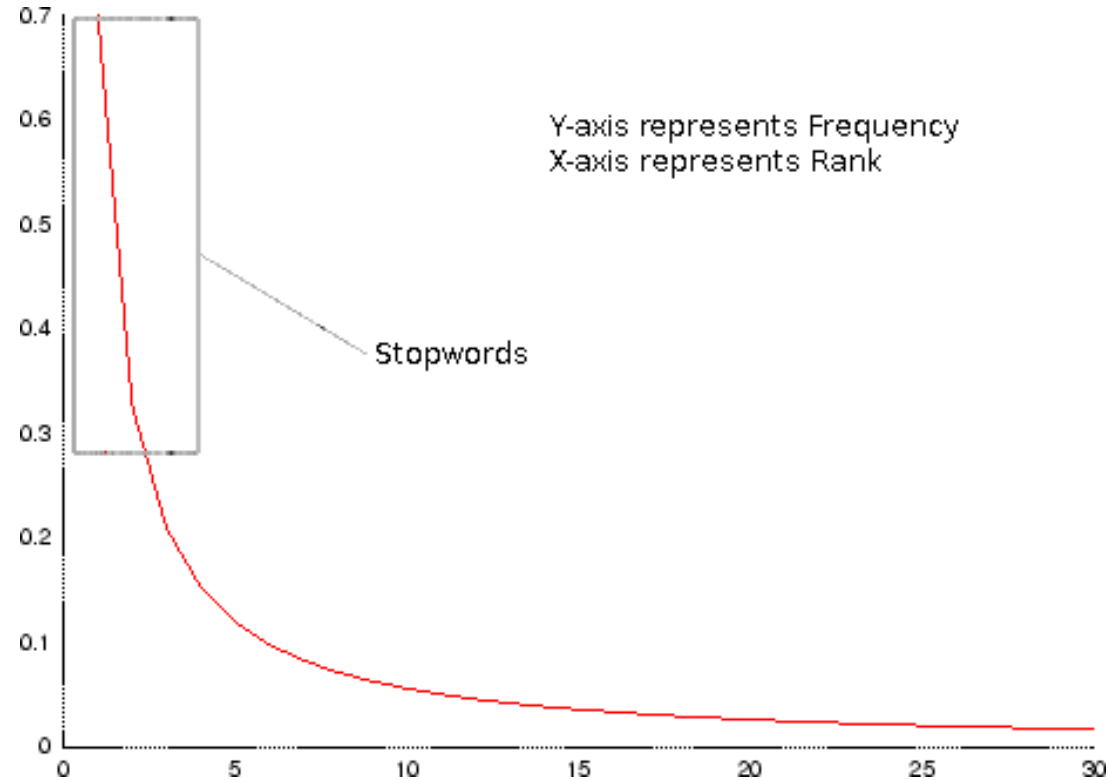
Stopword Removal

- A **stopword** is a commonly occurring term that appears in so many documents that it does not add to the meaning of the document, and appear in most English texts (except for very short ones).
- For example, in English, terms such as *the*, *and*, *of*, etc. tell a reader nothing about what the document is about.
- They are so common that they appear in almost all documents, and of little use when differentiating between documents.
- The fact that they are so common also means that more processing power is required to deal with these terms than others that are less common.
- This additional processing, combined with their relative lack of usefulness means that they are often removed from the documents at the preprocessing stage.

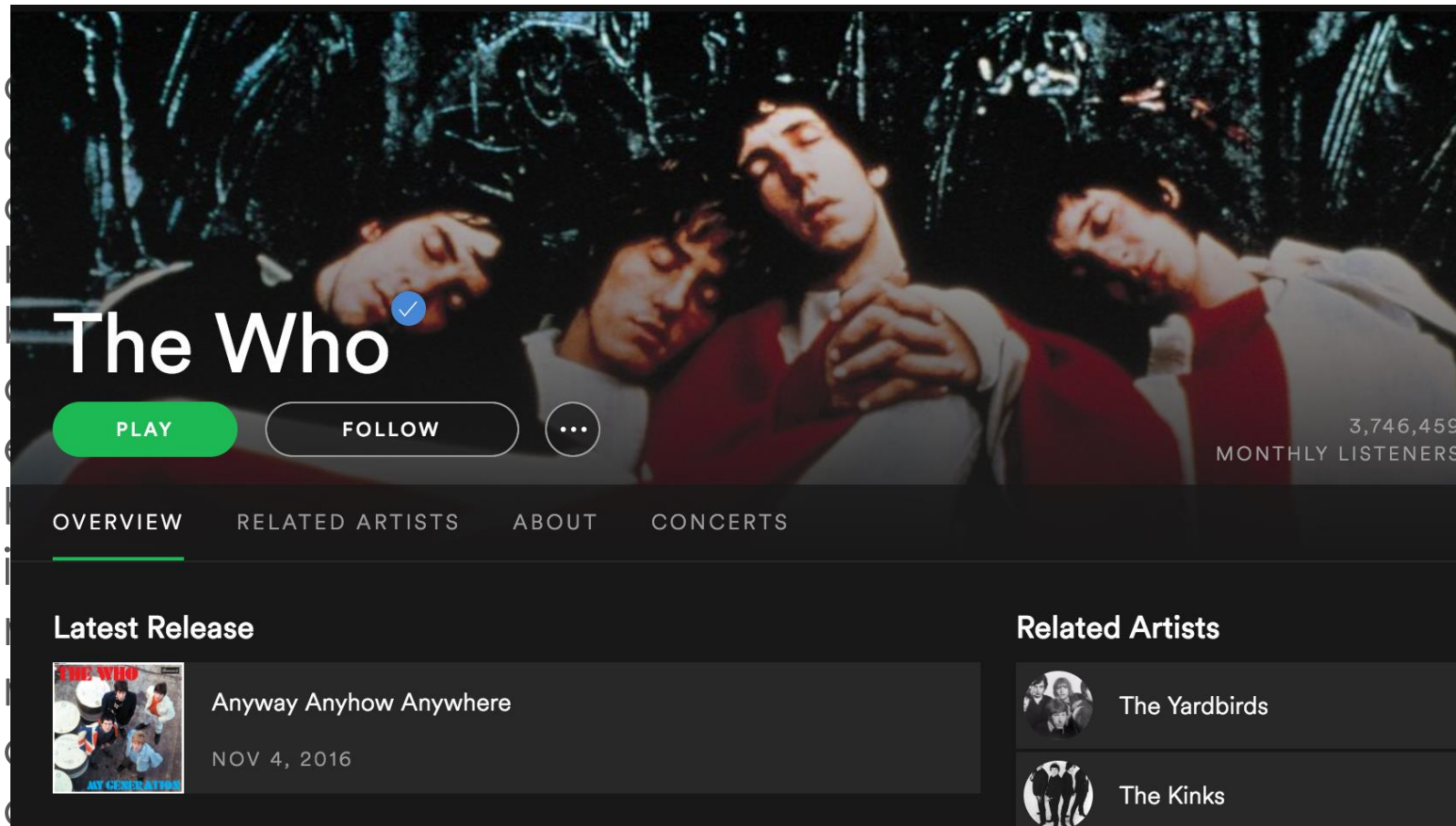
Stopword Removal

- Stopwords can be estimated using **Zipf's Law**.
- George Kingsley Zipf analysed the statistical occurrence of words (terms) in text.
- He noted that while only a few terms are used very often, more are used only rarely.
- If we rank terms from the most commonly used to the least commonly-used in a large text collection, the second term is approximately $\frac{1}{2}$ as common as the first, the third is approximately $\frac{1}{3}$ as common as the first, etc.
- We use this to identify what terms count as stopwords in various languages.

Stopword Removal: Zipf's Law



a about above across adj after again against all almost alone along
also although always am among an and another any anybody
anyone anything anywhere apart are around as aside at away be
because been before behind being below besides between beyond
both but by can cannot could deep did do does doing done down
downwards during each either else enough etc even ever every
everybody everyone except far few for forth from get gets got had
hardly has have having her here herself him himself his how however i
if in indeed instead into inward is it its itself just kept many maybe
might mine more most mostly much must myself near neither next no
nobody none nor not nothing nowhere of off often on only onto or
other others ought our ours out outside over own p per please plus pp
quite rather really said seem self selves several shall she should since so
some somebody somewhat still such than that the their theirs them
themselves then there therefore these they this thorough thoroughly
those through thus to together too toward towards under until up
upon v very was well were what whatever when whenever where
whether which while who whom whose will with within without would
yet young your yourself



The Who

PLAY FOLLOW ...

3,746,459 MONTHLY LISTENERS

OVERVIEW RELATED ARTISTS ABOUT CONCERTS

Latest Release

 Anyway Anyhow Anywhere
NOV 4, 2016

Related Artists

 The Yardbirds

 The Kinks

some somebody somewhat still such than that **the** their theirs them
themselves then there therefore these they this thorough thoroughly
those through thus to together too toward towards under until up
upon v very was well were what whatever when whenever where
whether which while **who** whom whose will with within without would
yet young your yourself

Stopword Removal

- There is a **tradeoff** required when using stopwords.
- The trend is away from removing them:
 - Good compression means that space required is small.
 - Good optimisation means processing them takes little time.
- Some queries will become impossible if all stopwords are removed.
 - Phrases: “King **of** Denmark”
 - Song titles, etc. “Let **it be**”, “**To be or not to be**”
 - Relational queries: “Flights **to** London”
- Some solutions:
 - Don't remove stopwords. Use all words as terms.
 - Recognise when combinations of stopwords are meaningful, and include these as terms in the index.

Stemming

- Many terms in natural language appear different to a computer, but represent the same concept.
- In English, terms such as “connect”, “connecting”, “connects”, “connected”, etc. represent a similar concept.
- Many IR systems believe that a search for “computing” should also include documents that contain the word “computer”, for example.
- **Stemming** is the process that maps these terms to a **common root**.
 - e.g. “computing”, “computer” → “comput”
 - Note: a stem does not need to be a real word.
- The end of a term is called a **suffix** (-er, -s, -ing, -ed, etc.)
- Hence stemming is also known as **suffix stripping**.

Stemming

- While there have been many algorithms to accomplish stemming, the most famous (for English text) is **Porter's Stemming Algorithm**.
- The rules that it follows are detailed in:
 - Martin Porter, An Algorithm for Suffix Stripping, Program, 14 no. 4, pp. 130-137, July 1980.
- Numerous implementations are available at:
 - <http://tartarus.org/~martin/PorterStemmer>
- It should be noted that Porter's stemming algorithm is not necessarily the most accurate, though it is the most widely used. Snowball and Porter2 are often considered to be more effective.

Stemming Algorithms

- Stemming Example: each of the following news headlines include words with a common stem:
 - accept: FAI decides to **accept** rescue deal.
 - acceptable: FAI deems rescue deal **acceptable**.
 - acceptance: Owens pleased after rescue deal **acceptance**.
 - accepted: The FAI has **accepted** the government rescue deal.
 - accepts: FAI **accepts** rescue deal.
- Clearly, these would be relevant to the same query.
 - The stem for all of these is “**accept**”.
- If stemming is applied to the documents, it is also applied to queries, so a user can find these documents using any term related to the stem **accept** (accept, acceptable, acceptance, accepted, accepts, etc.)

Stemming: Problems

- Sometimes suffixes can be removed from words that are **not related**, but that end up being the same afterwards.
- This is known as **overstemming**.
- Stemming is a very useful process to help words of similar meanings to be matched, but it is not a perfect process.
- **Example:** consider the words “general” and “generate”
- Both stem to “gener”, even though they are not related.
- Using a dictionary can help avoid this type of problem

Stemming: Problems

- Although not specifically a problem with stemming, there is also an issue with words that do not have the same meaning, but are spelled the same (known as **homographs**).
 - At the Battle of Waterloo, Napoleon's forces were **routed**.
 - The army was heavily defeated in battle.
 - The cars were **routed** off the motorway.
 - The cars were sent in a different direction.
- It is very difficult to tell the difference between these without using Natural Language Processing, which is a much slower process that often needs to see the word in context.

Lemmatisation

- Lemmatisation is a Natural Language Processing (NLP) technique for converting words into **lemmas** (i.e. base words found in dictionaries).
 - Unlike a **stem**, a **lemma** is always a real word.
- This is slower than stemming, but is usually more effective, because it can deal with words that stemming can't handle.
- To find the right lemma, a lemmatiser will often need to know the **part of speech** that the word is being used as (verb, noun, etc.). This requires more text analysis than stemming.

Lemmatisation vs. Stemming

Verbs

Word	Lemma	Stem
running	run	run
runs	run	run
ran	run	ran
be	be	be
am	be	am
are	be	ar
brought	bring	brought
brings	bring	bring

Nouns

Word	Lemma	Stem
wolves	wolf	wolv
wolf	wolf	wolf
church	church	church
churches	church	church
mouse	mouse	mous
mice	mouse	mice
radius	radius	radiu
radii	radius	radii

Preprocessing: Phrase Queries

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Phrase Queries

- Want to be able to answer queries such as “**stanford university**” – as a phrase.
- Thus the sentence “*I went to university at Stanford*” is not a match.
 - The concept of phrase queries has proven easily understood by users; one of the few “advanced search” ideas that works
- For this, it is not enough to store only
<term : docs> entries

A first attempt: Biword indexes

- Index every consecutive **pair** of terms in the text as a phrase.
- For example the text “Friends, Romans, Countrymen” would generate the biwords (bi- is a prefix used to mean that there are two of something).
 - ***friends romans***
 - ***romans countrymen***
- Each of these biwords is now a dictionary term.
- Two-word phrase query-processing is now immediate.

Longer phrase queries

- “*stanford university palo alto*” can be broken into the Boolean query on biwords:

stanford university AND *university palo* AND *palo alto*

Without the documents themselves, we cannot verify that the docs matching the above Boolean query do contain the exact phrase.



Can have false positives!

Issues for biword indexes

- False positives, as noted before.
- Index blowup due to bigger dictionary
 - Infeasible for more than biwords, big even for them.
- Biword indexes are not the standard solution (for all biwords) but can be part of a compound strategy.

Solution 2: Positional indexes

- In the postings, store for each **term** the position(s) in which it appears in the documents.

<**term**, number of docs containing **term**;
doc1: position1, position2 ... ;
doc2: position1, position2 ... ;
etc.>

Positional index example

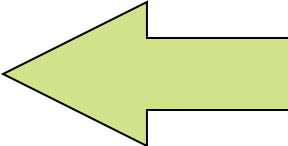
<*be*: 993427;

1: 7, 18, 33, 72, 86, 231;

2: 3, 149;

4: 17, 191, 291, 430, 434;

5: 363, 367, ...>



Which of docs *1,2,4,5*
could contain “*to be*
or not to be”?

- For phrase queries, we use a merge algorithm recursively at the **document** level.
- But we now need to deal with more than just equality: we look for terms that follow each other.

Positional index size

- You can compress position values.
- **BUT**, a positional index expands postings storage *substantially*
- Nevertheless, a positional index is now standardly used because of the power and usefulness of phrase queries.

Rules of thumb

- A positional index is 2–4 times as large as a non-positional index.
- Positional index size 35–50% of volume of original text.
- Caveat: all of this holds for “English-like” languages.
- Often, a combination of selected biwords and a positional index are used, with good results.

Vector Space Model

Introduction

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Introduction

- So far, we have seen Information Retrieval (IR) as a process where a user submits a **query** consisting of some **terms** connected with **boolean operators**.
- This is known as the “Boolean Model” of IR, and has some serious disadvantages:
 - The model predicts that each document is either **relevant** or **non-relevant** to the **query**.
 - There is no notion of a “partial” match to the query conditions and so this can lead to too few documents being retrieved.
 - Every document that matches the query is treated the same, so **no ranking** occurs.

Introduction

- One of the most common IR models in use is the **Vector Space Model**.
- This models both **documents** *and* **queries** in an **N-dimensional space**.
- Basic vector algebra is used to calculate the **most similar** documents to a query.
- The basic vector space model has been adapted for many other purposes.
- The most common are:
 - Machine Learning (e.g. K-NN algorithm)
 - Clustering
- See page 77 of Modern Information Retrieval (2nd Ed.).

Basis

- A collection contains a set of documents each of which is composed of a “bag of words” (also known as terms).
 - Each unique **term in the collection** is represented by a dimension in the vector space.
 - A document is then represented by giving a non-zero value to the dimension representing a term it contains.
 - This number is known as a **term weight**.
 - Other dimensions get a value of zero.
 - i.e. the weight of a term is 0 in documents that do not contain it.
 - Note the order of the terms within the document is not retained.
- A **similarity score** is calculated based on the proximity of two vectors in the N-dimensional space.

Partial Matching

- Unlike the Boolean Model, the Vector Space Model facilitates **partial matching** by using **non-binary** term weights.
- In models like this, each document must have a **similarity score** calculated for it, which measures how similar it is to the given query.
- This is usually shown as $sim(q, d)$ (i.e. the similarity between a query q and a document d).
- These models return a **ranked list** of documents, where the documents with the highest similarity scores are at the top of the list.
- In this way, it is hoped that the most relevant documents are at the top of the result set, so that the user can find relevant information easier.

Vector Space Model

Document and Query Representation

Document Representation

- Consider the following documents:
 1. Information Retrieval is an exciting subject
 2. Mathematics is important in Information Retrieval
- Ignoring stopwords the collection contains **6 distinct terms**: information; retrieval; exciting; subject; mathematics; important.
- The vector space will have **six dimensions** (one for each term in the document collection).
- This means that **EVERY DOCUMENT** is mathematically represented by six dimensions, regardless of how many terms it contains.

Document Representation

- For the introduction to this topic, we first assume a simple weighting scheme (we will replace this later):
 - $w_{i,j}$ refers to the weight of term i in document j .
 - if term i is contained in document j , the weight is 1 (i.e. $w_{i,j} = 1$).
 - if term i is not contained in document j , $w_{i,j} = 0$.
- In practice, the weight will be a real number where:
 - $0 \leq w_{i,j} \leq 1$
- We will see how this is calculated later.

Term	Doc 1	Doc 2
information	1	1
retrieval	1	1
exciting	1	0
subject	1	0
mathematics	0	1
important	0	1

Query Representation

- The query is represented in an identical way.
- Consider the query: *important information*.
- This is represented in the vector model as:
 - $(1, 0, 0, 0, 0, 1)$
- It is important that the **order of the terms** in the **vector** is the same as for the corpus (i.e. the first dimension relates to the term “information”, the second relates to “retrieval”, etc.).
 - If the order changes, then we are no longer comparing like with like.

Vector Space Model

- The rest of this topic is organised as follows:
 - **Cosine Similarity:** how to compare queries with documents to measure how similar they are.
 - **Term Weights:** Some words are more important in describing a document than others.
 - **Conclusions and Summary**

Vector Space Model

Cosine Similarity

COMP3009J: Information Retrieval

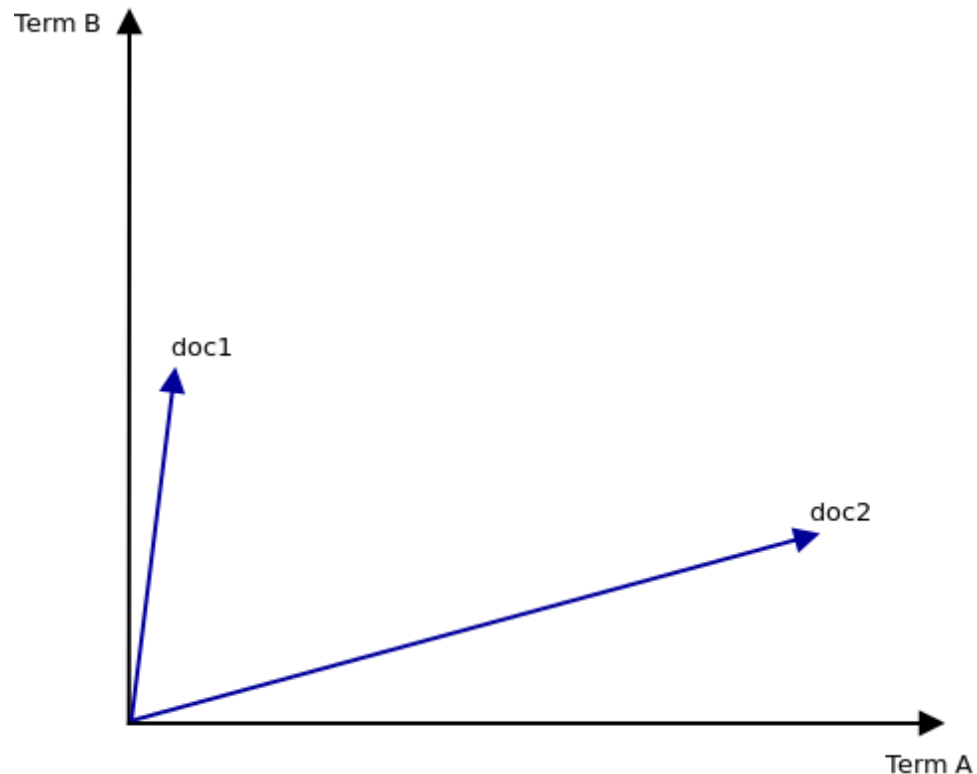
Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Retrieval

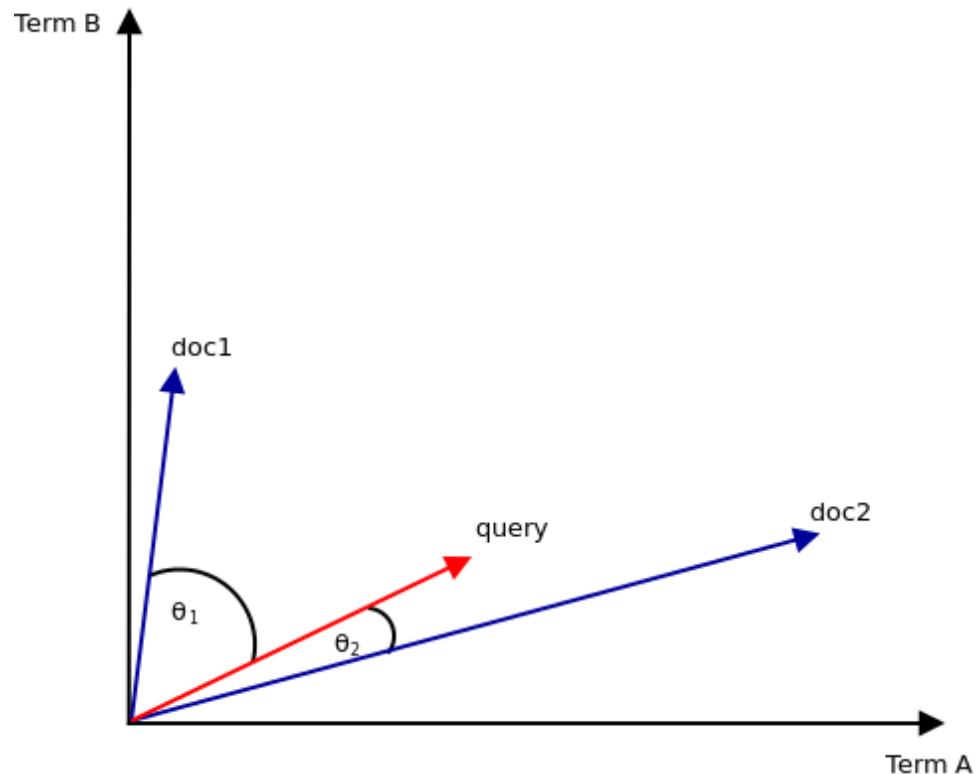
- The theory behind retrieval says that the closer the vectors are in the N-dimensional space, the more similar the items they represent are.
- In retrieval, the query vector is compared to **all of the document vectors** in the index (unless this has been filtered beforehand using postings lists).
- Vector algebra contains an operator known as the **dot product**, defined for two vectors $\vec{a}$ and $\vec{b}$ as:
 - $\vec{a} \cdot \vec{b} = \cos.\theta \times |\vec{a}| \times |\vec{b}|$
 - Here, $|\vec{a}|$ and $|\vec{b}|$ are the lengths of vectors $\vec{a}$ and $\vec{b}$ respectively.
 - θ is the angle between the two vectors.

Similarity Example



- Here, we see a VERY simple vector model with only two terms.
- In the document “doc1”, Term B is far more important than Term A.
- The opposite is the case with document “doc2”.
- (This is for illustration only: a realistic example will have many more dimensions).

Similarity Example



- We now introduce a query, **also represented by a vector**.
- The key to the Vector Space Model is the angle between two vectors (θ_1 and θ_2).
- Here, θ_2 is clearly the smaller angle, suggesting that the query is more similar to “doc2”.

Retrieval

- Consider the dot product formula again:

- $\vec{a} \cdot \vec{b} = \cos.\theta \times |\vec{a}| \times |\vec{b}|$

- This can be rewritten as:

- $\cos.\theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \times |\vec{b}|}$

- This means that if we can calculate the dot product of the two vectors, and the lengths of the two vectors, we can easily find the cosine of the angle between them.
- Why would we want to do this?

Cosine Similarity: Why?

- Principle: the smaller the angle between two vectors, the more similar they are.
- We do not use negative weights, therefore: $0^\circ \leq \theta \leq 90^\circ$
 - The angle is 90° if the vectors have nothing in common.
 - The angle is 0° if the vectors are identical (or near-identical)
- $\cos.0^\circ = 1$ and $\cos.90^\circ = 0$
 - Therefore, the cosine function gives a value between 0 (for very different documents) and 1 (for identical documents).
 - This is ideal for a measure of **similarity**.

Cosine Similarity Formulae

$$\text{sim}(d_j, q) = \frac{\vec{d_j} \cdot \vec{q}}{|\vec{d_j}| \times |\vec{q}|}$$

- $\vec{d_j}$ is a vector representing a document d_j
- $\vec{q}$ is a vector representing the query q
- $\vec{d_j} \cdot \vec{q}$ is the dot product of $\vec{d_j}$ and $\vec{q}$
- $|\vec{d_j}|$ is the length of $\vec{d_j}$
- $|\vec{q}|$ is the length of $\vec{q}$

Cosine Similarity Formulae

$$\text{sim}(d_j, q) = \frac{\vec{d_j} \cdot \vec{q}}{|\vec{d_j}| \times |\vec{q}|}$$

$$= \frac{\sum_{i=1}^T w_{i,j} \times w_{i,q}}{\sqrt{\sum_{i=1}^T w_{i,j}^2} \times \sqrt{\sum_{i=1}^T w_{i,q}^2}}$$

- ▣ T is the number of distinct terms in the entire document collection
- ▣ $w_{i,j}$ is the weight of term i in document j
- ▣ $w_{i,q}$ is the weight of term i in the query q

Vector Definitions

- The top line of this formula is the dot product in N -dimensions of $\vec{a}$ and $\vec{b}$, $\vec{a} \cdot \vec{b}$, which is defined as:
 - $\vec{a} \cdot \vec{b} = \sum_{i=1}^T (w_{i,a} \times w_{i,b})$
 - where T is the number of dimensions and $w_{i,a}$ is the weight of term i in $\vec{a}$ (i.e. the i^{th} component of the vector $\vec{a}$).
- Or in pseudocode:
 - Begin with a total of 0
 - For each dimension in the vectors:
 - Multiply the value in that dimension of vector $\vec{a}$ by the value in that dimension of vector $\vec{b}$.
 - Add this to the total.
 - The final total is the dot product.

Dot Product: Example

- Example: dot product of $(1,1,1,1,0,0)$ and $(1,0,0,0,0,1)$
- $(1 \times 1) + (1 \times 0) + (1 \times 0) + (1 \times 0) + (0 \times 0) + (0 \times 1) = 1$

Vector Definitions

- The bottom line of the similarity formula requires us to calculate the length of a vector.
- The length of a vector $\vec{a}$, given by $|\vec{a}|$ is defined as:
- $|\vec{a}| = \sqrt{\sum_{i=1}^T w_{i,a}^2}$
- Or in pseudocode:
 - Begin with a total of 0
 - For each dimension in vector $\vec{a}$
 - Take the value in that dimension of vector $\vec{a}$ and square it.
 - Add this to the total.
 - The length of vector $\vec{a}$ is the square root of the total

Vector Length: Example

■ Example: length of $(1, 1, 1, 1, 0, 0)$

■ $\sqrt{1^2 + 1^2 + 1^2 + 1^2 + 0^2 + 0^2}$

■ $= \sqrt{4} = 2$

Cosine Similarity: Example

- Returning to the documents and the query we saw earlier:

- D1: 1, 1, 1, 1, 0, 0

- D2: 1, 1, 0, 0, 1, 1

- Q: 1, 0, 0, 0, 0, 1

- The similarities between these documents and the query are calculated as:

- $$\text{sim}(d_1, q) = \frac{\vec{d}_1 \cdot \vec{q}}{|\vec{d}_1| \times |\vec{q}|} = \frac{1}{2 \times \sqrt{2}} = 0.3535$$

- $$\text{sim}(d_2, q) = \frac{\vec{d}_2 \cdot \vec{q}}{|\vec{d}_2| \times |\vec{q}|} = \frac{2}{2 \times \sqrt{2}} = 0.7071$$

- Hence document D2 is most similar to the query.
- For a larger corpus, we would return a **ranked list** starting with the document most similar to the query, and descending from there until some cutoff point (we would not return all documents)

Cosine Similarity: Some Questions

- Some questions that will help us when planning to implement this:
 - What effect on the similarity score does a term have if it does not appear in the query?
 - What effect on the similarity score does a term have if it does not appear in the document?
 - What effect on the length of a document vector does a term have if that term is not contained in the document?
- Earlier, we said:
 - ... **EVERY DOCUMENT** is mathematically represented by six dimensions (for this tiny corpus), regardless of how many terms it contains.
 - In practice, this leads to very **sparse vectors** (i.e. lots of zeros).
 - “**mathematically**” represented $\neq$ how we represent using a computer program.

Vector Space Model

Term Weighting

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Term Weighting

- In order to illustrate the model, we have used a simple binary weighting scheme.
- This gives the representation of the vectors in terms of 1s and 0s only.
- This is not commonly used in practice.
- Frequently used metric is **TF-IDF**:
 - **TF**: Term Frequency
 - **IDF**: Inverse Document Frequency

Term Weights

- We can also take the **number of times** a term occurs in a document into account (i.e. the *frequency* of the term), for example:
 - The term “Biden” will appear in a biography of Joe Biden, for obvious reasons.
 - The term “Biden” will also appear in a biography of Donald Trump, as he once won an election against Donald Trump. It will not be as common, however.
- The first document is more relevant to a search for “Biden” but without using weights, both would be treated equally.
- The greater number of occurrences of the term in the first document should result in a greater weight.

Term Weights: Why?

- Consider a document collection of 100,000 documents.
- A word that appears in every document is not very useful as an index term, as it tells us very little about which documents a user might be interested in.
- A word that appears in 5 documents considerably narrows the space of documents that might be of interest to the user.
- This is the basic rationale behind stopwords, but it has wider consequences also.

Term Weights

- When talking about term weights, we use the following notation:
- A weight $w_{i,j} > 0$ is associated with **every index term** k_i of **document** d_j .
- For an index term that does not appear in the document, $w_{i,j} = 0$
- This weight quantifies the importance of the index term for describing the document's semantic contents.
- Any time we examine a new IR model, we must firstly consider how its weighting scheme works.

Term Weights

- Index terms are usually assumed to be **independent**, though this is a simplification.
- Consider a document about computer networks. Here, the terms “computer” and “network” are clearly related, as the appearance of one attracts the appearance of the other.
- You may argue that their weights should be altered to take this dependency into account.

TF-IDF Weighting

- TF-IDF consists of two parts:
 - The **Term Frequency (TF)** assumes that a term that occurs many times in a document is more important in describing that document than one that occurs rarely. **Every term** has a separate term frequency for **every document** in the IR system.
 - The **Inverse Document Frequency (IDF)** argues that terms that only occur in very few documents are more important than those that are found in many documents. This is based on the same idea as stopwords. **Each term** has only **one** inverse document frequency within an entire document collection.

Term Frequency (TF)

- Term Frequency is calculated by using the formula below.
- $tf_{i,j}$ is the term frequency of term i in document j and is calculated as follows:
 - $tf_{i,j} = \frac{freq_{i,j}}{maxfreq_j}$
 - where $freq_{i,j}$ is the number of times term i occurs in document j and $maxfreq_j$ is the maximum number of times *any* term occurs in document j .
 - It is a *normalised* frequency.
- The reason we divide by the maximum frequency is to ensure that terms in **long documents** do not get an unfair advantage.

Inverse Document Frequency (IDF)

- A term's Inverse Document Frequency is designed to give a higher weight to terms that are rare.
- **Note** that each term only has one IDF score within an entire document collection.
- It is calculated using the following formula:
 - $idf_i = \log_2 \left(\frac{N}{n_i} \right)$
- N is the total number of documents in the collection
- n_i is the number of documents in the collection that contain term i .

TF-IDF

- Finally, the TF-IDF weight for a term i in a document j is found by multiplying its TF for that document by its IDF score, i.e.
- $w_{i,j} = tf_{i,j} \times idf_i$
- Note again that each term has a **different TF-IDF score** for **each document** in the document collection.

Vector Space Model

Conclusions and Summary

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

TF-IDF Advantages

- The advantages of the TF-IDF weighting scheme over others are:
 - Using term frequency gives more influence to a document that contains many occurrences of a particular term.
 - Using inverse document frequency lessens the impact of terms that appear very often in the collection. These terms are regarded as being less informative.

Vector Space Model: Advantages and Disadvantages

- There are three main advantages of the vector space model:
 - The term weighting schemes can improve retrieval performance.
 - Partial matching strategy allows for retrieval of documents that match part of the query.
 - The cosine similarity can be used to return a ranked list of documents .
- The main disadvantage of the model is the assumption that terms are independent, which can sometimes harm performance.

Vector Space Model: Implementation

■ The main steps in implementing the vector space model are:

1. Read in the documents.
2. Divide each document into its constituent terms.
3. Remove stopwords.
4. Stem terms.
5. Add documents to an index.
6. Calculate TF-IDF.
 - Each term has one IDF score.
 - Each term has a separate TF score in each document.
7. Receive a query.
8. Retrieve query results.

Summary

- The **Vector Space Model** of Information Retrieval is based on concepts from vector algebra.
- Documents and queries are represented as vectors.
- Cosine similarity is used to decide which documents most closely match a query.
- Terms can be weighted in multiple ways.
- A common weighting scheme is TF-IDF (Term Frequency-Inverse Document Frequency).

Information Retrieval

The BM25 Model

Dr. Seán Russell

School of Computer Science,
University College Dublin

Week 6



Table of Contents

1 Best Match Variants

2 Inverse Document
Frequency

3 Term Frequency

4 Document Length

5 The BM25 Formula

Section Contents

- 1 Best Match Variants
 - Principles

- The “BM” in BM25 has the meaning “Best Match”
- A number of different BM rankings were developed through a series of experiments
- We will focus on the BM25 model, but will also reference some earlier variants
 - Such as BM15 & BM11
- Today, BM25 is considered to be a state-of-the-art retrieval method that operates using the same principles as TF-IDF but generally performs better than the classic version we have already studied

- The development of BM25 is based on the belief that good term weighting comes from three principles
 - 1 Inverse document frequency (IDF)
 - 2 Term frequency (TF)
 - 3 Document length normalisation

Section Contents

- 2 Inverse Document Frequency
 - Formula for IDF in BM Models
 - Effect of IDF
 - Negative IDF

- Inverse document frequency is a way of ranking how important a term is in a document collection
- If the term is contained in many/most of the documents, then searching using that term will be less useful
- If the term is contained in only a small number of documents, then searching using that term will be effective

- The following formula is used to calculate inverse document frequency in BM models for a term k_i

$$\log \left(\frac{N - n_i + 0.5}{n_i + 0.5} \right)$$

- Where:
 - N is the total number of documents in the collection
 - n_i is the total number of documents that the term k_i appears in
- These values are summed for all terms in both the document and the query

$n_i \backslash N$	50	75	100	200	400	800
1	5.04	5.63	6.05	7.06	8.06	9.06
2	4.28	4.88	5.30	6.31	7.32	8.32
4	3.37	3.99	4.42	5.45	6.46	7.47
8	2.32	2.99	3.44	4.50	5.53	6.54
10	1.95	2.64	3.11	4.18	5.22	6.23
25	0.00	0.99	1.57	2.78	3.88	4.93
50	-6.66	-0.99	0.00	1.58	2.80	3.89
75		-7.24	-1.57	0.73	2.11	3.26
100			-7.65	0.00	1.58	2.80
200				-8.65	0.00	1.58

$n_i \backslash N$	50	75	100	200	400	800
10	1.95	2.64	3.11	4.18	5.22	6.23
25	0.00	0.99	1.57	2.78	3.88	4.93
50	-6.66	-0.99	0.00	1.58	2.80	3.89
75		-7.24	-1.57	0.73	2.11	3.26
100			-7.65	0.00	1.58	2.80
200				-8.65	0.00	1.58

- Note: If a term is in 50% of the documents, then IDF is 0
- If a term is in more than 50% of the documents, then IDF is negative

- If a term is actually in our query, then we don't want a negative IDF
- We have 2 options:
 - Treat all terms in 50% of the documents or more as stopwords and do not index them
 - Modify the formula, e.g. the Lucene open source IR library adjusts it to

$$\log \left(1 + \frac{N - n_i + 0.5}{n_i + 0.5} \right)$$

Section Contents

- 3 Term Frequency
 - Effect of TF
 - Partial and Complete Matches

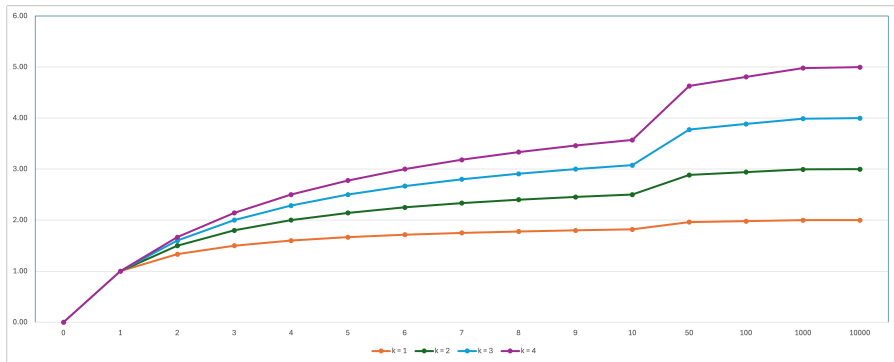
- Term frequency is usually shown as $(f_{i,j})$
 - Where i is the number of the term and j is the number of a document
- It is effectively a **count** of the number of times the term appears in the document
- BM models use a constant in a formula for normalising across different length documents

$$\frac{(k + 1)f_{i,j}}{f_{i,j} + k}$$

- This formula prevents **saturation**
 - Saturation is where a term appears so frequently in a document that it receives a higher rank than it should

$$\frac{(k + 1)f_{i,j}}{f_{i,j} + k}$$

- The constant k influences how quickly saturation is reached
- Result is between 0 and $k + 1$



- The result of the calculation increases at different rates as the frequency of the term increases

- Another nice feature of this term frequency calculation is that it rewards complete matches over partial ones
- Imagine we have the query “cat dog”
 - Assume they both have the same IDF
 - Assume that $k = 1$
- If a document contains both “cat” and “dog” once, each will add $1 * IDF$ to the ranking score
- If a document contains “cat” twice but does not contain “dog” it will add $1.33 * IDF$ for cat and $0 * IDF$ for dog
- A document that fully matches the query will be preferred over one that only partially matches

Section Contents

4

Document Length

- Example Results
- The Effect of b
- The Constants

- Comparing documents of different lengths can cause problems
 - If a very short document mentions “elephant”, it is probably about elephants
 - If a very long document mentions “elephant” once, it is probably not about elephants
- BM25 normalises the length of documents in calculations through a further modification of the term frequency calculation

$$B_{i,j} = \frac{(k + 1) f_{i,j}}{k \left((1 - b) + \frac{b \times \text{len}(d_j)}{\text{avg_doclen}} \right) + f_{i,j}}$$

$$B_{i,j} = \frac{(k + 1) f_{i,j}}{k \left((1 - b) + \frac{b \times \text{len}(d_j)}{\text{avg_doclen}} \right) + f_{i,j}}$$

- k is the same constant as before and has the same effect on term frequency
- b is a new constant that allows us to choose how important document length is
 - The range of values for b is $0 \leq b \leq 1$

$f_{i,j} \backslash d$	100	200	400	800	1600	3200	6400
1	1.43	1.29	1.08	0.82	0.55	0.33	0.18
2	1.67	1.57	1.40	1.16	0.86	0.57	0.34
3	1.76	1.69	1.56	1.35	1.06	0.75	0.47
4	1.82	1.76	1.65	1.47	1.20	0.88	0.58
5	1.85	1.80	1.71	1.55	1.31	1.00	0.67
6	1.88	1.83	1.75	1.61	1.39	1.09	0.76
7	1.89	1.85	1.78	1.66	1.45	1.16	0.83
8	1.90	1.87	1.81	1.69	1.50	1.23	0.90

- Here $k = 1$, $b = 0.75$ and $avg_doclen = 500$

$$B_{i,j} = \frac{(k+1)f_{i,j}}{k \left((1-b) + \frac{b \times \text{len}(d_j)}{\text{avg_doclen}} \right) + f_{i,j}}$$

- When $b = 0$, this becomes

$$B_{i,j} = \frac{(k+1)f_{i,j}}{f_{i,j} + k}$$

- When $b = 1$, this becomes

$$B_{i,j} = \frac{(k+1)f_{i,j}}{k \left(\frac{\text{len}(d_j)}{\text{avg_doclen}} \right) + f_{i,j}}$$

¹This is the formula used in BM15

²This is the formula used in BM11

- Not all document collections are the same
- So the ranking calculation should not be the same for every document collection
- BM25 provides two constants that we can tweak to improve performance
 - Changing the constant k changes how quickly increases in the term frequency stop causing increases in the results
 - Changing the constant b changes how important document length is in our calculation

- When we set the value of b to 0, then the calculation does not consider document length at all
- When we set the value of b to 1, then we use the full document length adjustment
- Typically, a value between these extremes is chosen (such as 0.75)

Section Contents

5

The BM25 Formula

- BM25 Example A
- BM25 Example B
- BM25 Variations
- BM25 Usefulness

$$sim_{BM25}(d_j, q) \sim \sum_{k_i \in d_j \wedge k_i \in q} \frac{(k+1) f_{i,j}}{k \left((1-b) + \frac{b \times len(d_j)}{avg_doclen} \right) + f_{i,j}} \times \log \left(\frac{N - n_i + 0.5}{n_i + 0.5} \right)$$

- This calculates the similarity score between a document d_j and a query q using BM25
- $k_i \in d_j \wedge k_i \in q$ means that this is applied to all terms k_i where that term is in both the document and the query

- **Q: “President Lincoln”** $k_0 = \text{“president”}, k_1 = \text{“lincoln”}$
- 500000 documents $N = 500000$
- “president” in 40000 documents $n_0 = 40000$
- “lincoln” in 300 documents $n_1 = 300$
- “president” occurs 15 times in d_{123} $f_{0,123} = 15$
- “lincoln” occurs 25 times in d_{123} $f_{1,123} = 25$
- d_{123} is 90% of the length of the average $\frac{\text{len}(d_{123})}{\text{avg_doclen}} = 0.9$
- $k = 1, b = 0.75$

- Calculation for k_0 (“president”) in d_{123}

$$\frac{2 \times 15}{0.25 + 0.75 \times 0.9 + 15} \times \log \left(\frac{500000 - 40000 + 0.5}{40000 + 0.5} \right)$$

$$\begin{aligned} \frac{30}{15.925} \times \log \left(\frac{460000.5}{40000.5} \right) \\ = 1.88 \times 3.52 = 6.6378 \end{aligned}$$

- Calculation for k_1 (“lincoln”) in d_{123}

$$\frac{2 \times 25}{0.25 + 0.75 \times 0.9 + 25} \times \log \left(\frac{500000 - 300 + 0.5}{300 + 0.5} \right)$$

$$\begin{aligned} \frac{50}{25.925} \times \log \left(\frac{499700.5}{300.5} \right) \\ = 1.93 \times 10.70 = 20.6355 \end{aligned}$$

- Result is sum of these values

$$6.6378 + 20.6355 = 27.2732$$

- **Q: “President Lincoln”** $k_0 = \text{“president”}, k_1 = \text{“lincoln”}$
- 500000 documents $N = 500000$
- “president” in 40000 documents $n_0 = 40000$
- “lincoln” in 300 documents $n_1 = 300$
- “president” occurs 43 times in d_7 $f_{0,7} = 43$
- “lincoln” occurs 4 times in d_7 $f_{1,7} = 4$
- d_7 is 85% of the length of the average $\frac{\text{len}(d_7)}{\text{avg_doclen}} = 0.85$
- $k = 1, b = 0.75$

- Calculation for k_0 (“president”) in d_7

$$\frac{2 \times 43}{0.25 + 0.75 \times 0.9 + 43} \times \log \left(\frac{500000 - 40000 + 0.5}{40000 + 0.5} \right)$$

$$\begin{aligned} \frac{86}{43.8875} \times \log \left(\frac{460000.5}{40000.5} \right) \\ = 1.96 \times 3.52 = 6.9046 \end{aligned}$$

- Calculation for k_1 (“lincoln”) in d_7

$$\frac{2 \times 4}{0.25 + 0.75 \times 0.9 + 4} \times \log \left(\frac{500000 - 300 + 0.5}{300 + 0.5} \right)$$

$$\begin{aligned} \frac{8}{4.8875} \times \log \left(\frac{499700.5}{300.5} \right) \\ = 1.64 \times 10.70 = 17.5132 \end{aligned}$$

- Result is sum of these values

$$6.9046 + 17.5132 = 24.4178$$

- There are a number of variations of BM25
- Two of these are particularly notable
 - BM25F: Allows different fields to be given different importance in the document
 - E.g. Title, headlines, main text
 - BM25+: Addresses the issue where short documents would be given scores that are too high

- Most IR researchers agree that it outperforms the vector model on general collections
- It takes a lot of time and many, many experiments to reach this kind of consensus
- Even though BM25 was originally proposed in the 1980s, the well-known Lucene IR library only adopted it as its default similarity score in 2016 when it replaced the Vector Space Model

Information Retrieval

Evaluation of IR

Dr. Seán Russell

School of Computer Science,
University College Dublin

Week 6



Table of Contents

1 Evaluation

2 The Cranfield Paradigm

Section Contents

- 1 Evaluation
 - Processing Evaluation
 - User Experience
 - Search
 - Evaluation of Effectiveness
 - Commonly Used Metrics

- What does it mean to evaluate something?
- In general, we are asking “how well does the system work?”
- There are many ways that this could be measured:
 - How quickly does the user receive a response?
 - How much memory is used by the system?
 - How much CPU resources are used?
 - Does the user enjoy using the system?
 - How effective is the system in satisfying the user’s information need?

- The first three questions relate to the processing done by the system
 - The algorithmic complexity of our algorithm (Big-O) will be reflected in CPU use
 - Other elements of the design (such as caches) will also be reflected in memory use
 - Both of these will combine with the actual system running the code to determine the time taken to get a response
- These are generally important questions, but they relate more to how the system is implemented than it's model

- The fourth question relates to the experience of the user
- User experience (UX) is all about how they perceive the application
 - Is it easy to use?
- Again, an important question but more about the interface than the core of the program

- The final question is about how well the system worked.
- If the system worked correctly, the returned results will contain the information we needed
- This is the most interesting question!
- If we want to compare IR systems, we need to evaluate how well the retrieval algorithms work!

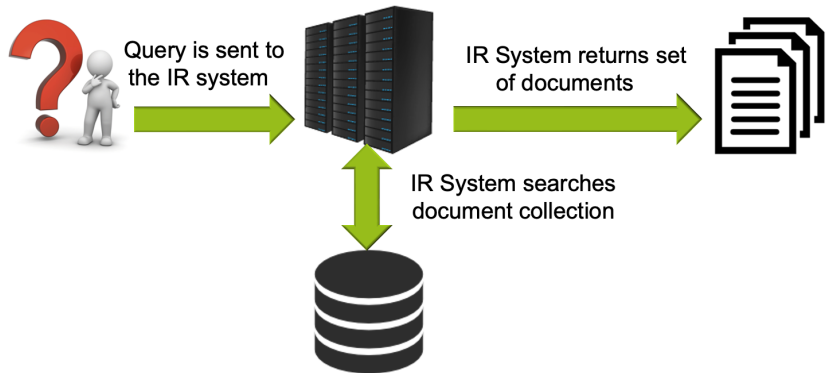
- Evaluation of the effectiveness of an IR system (particularly in the research area) is a vital topic
 - We call this **retrieval evaluation**
- There are many different techniques used in IR and there needs to be accepted ways to quantify their performance
- Many metrics exist to do this

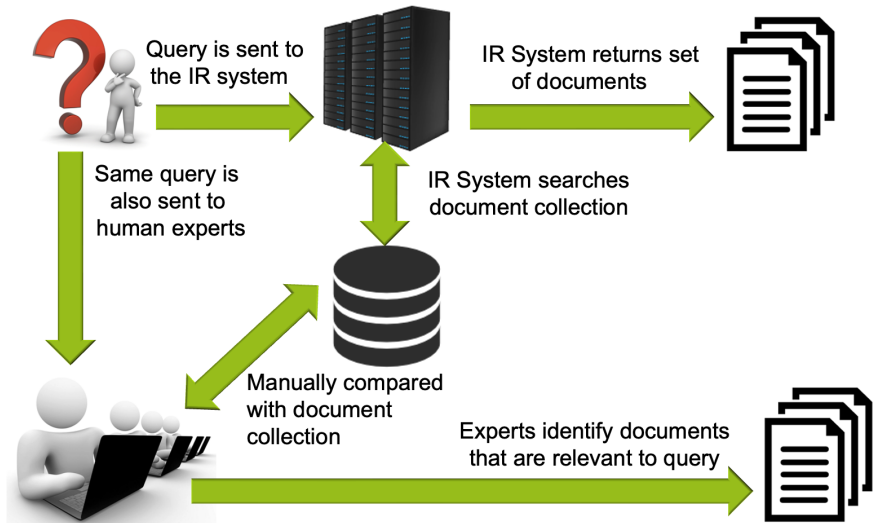
- Below are the commonly used metrics when evaluating IR systems
 - Precision/Recall
 - Precision @ n/R-precision
 - Mean Average Precision (MAP)
 - bPref
 - NDCG
- But first, we need to know how to do the evaluation!

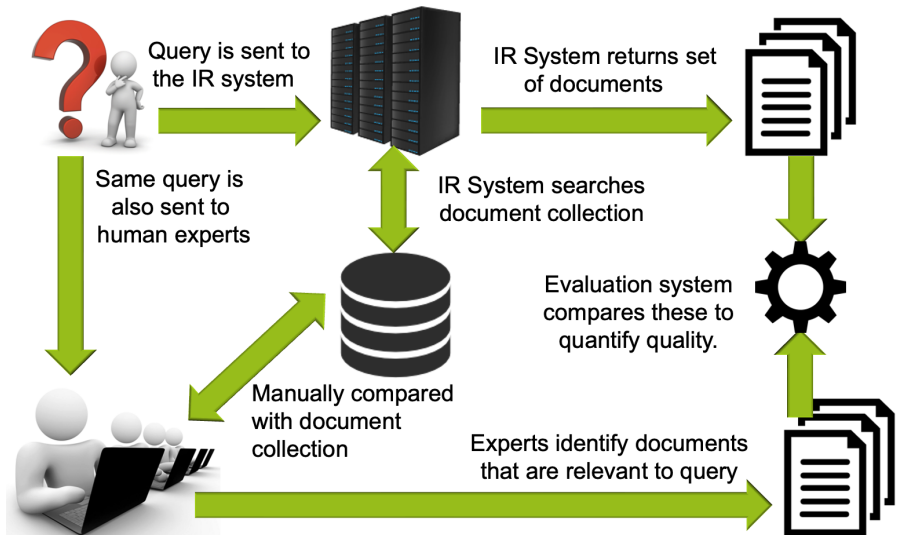
Section Contents

- 2 The Cranfield Paradigm
 - Quality of Results
 - Relevant Documents
 - Standard Test Collections
 - Document Sets
 - Ranking
 - Example

- When we query an IR system, it returns a set of documents
- But how effective was it?
- We need a way of evaluating the response numerically
 - This will allow us to compare the results of different IR systems to the same queries





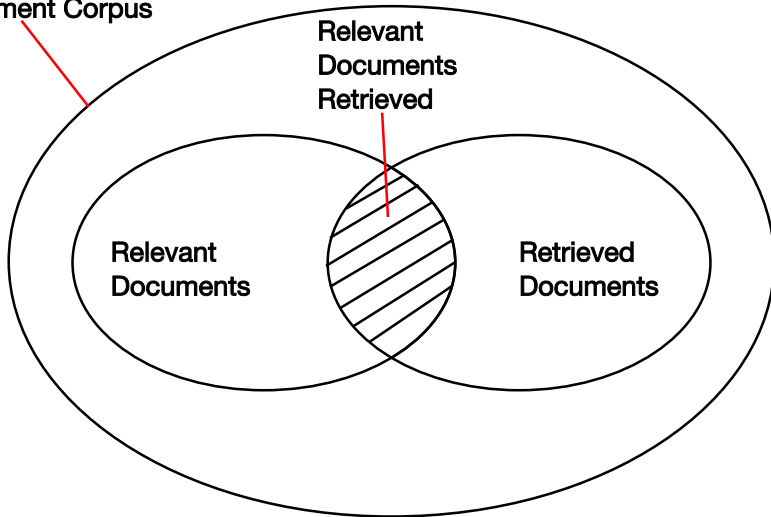


- This method of evaluation requires that we have experts identify the documents that are **relevant** to a particular query
- A **relevant document** is one that (at least partially) satisfies a user's information need
- Unfortunately, an information need is a very subjective thing
- This is the method used in the Cranfield experiments

- Evaluations carried out in this way has led to the creation of **standard test collections**
- These collections consist of
 - A **standard corpus** used for all queries
 - A set of **standard queries**
 - A set of **relevance judgements**
- I.e. For each of the standard **queries**, the human judges/experts have created a list of all of the documents in the **corpus** that are **relevant**

- When planning our evaluation, we can talk about 3 sets of documents
 - C: The corpus is the set of all available documents
 - Rel: The relevant set is the documents that have been judged as relevant by experts for that query
 - Ret: The retrieved set is the documents that the IR system has returned in response to the same query

Document Corpus



- In reality, the answer set is not really a set!
- Generally it is in the form of a ranked list
 - The boolean model is the exception to this
- When we evaluate the effectiveness of an IR technique, we need to evaluate the quality of this ranked list

- Consider a query q , on a document collection
- $Rel = \{d_3, d_5, d_9, d_{25}, d_{39}, d_{44}, d_{56}, d_{71}, d_{89}, d_{123}\}$
- Ret:

1 d_{123}
2 d_{84}
3 d_{56}
4 d_6
5 d_8

6 d_9
7 d_{511}
8 d_{129}
9 d_{187}
10 d_{25}

11 d_{38}
12 d_{48}
13 d_{250}
14 d_{113}
15 d_3

Information Retrieval

Precision and Recall

Dr. Seán Russell

School of Computer Science,
University College Dublin

Week 6



Table of Contents

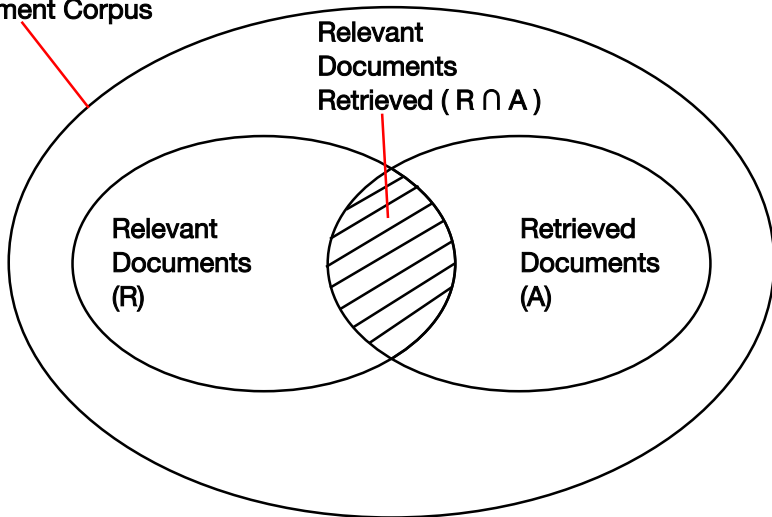
1 Precision / Recall

Section Contents

- 1 Precision / Recall
 - Example
 - Precision vs. Recall
 - High Precision Example
 - High Recall Example
 - Modern Use of Precision / Recall
 - Problems with Precision

- Precision and Recall are the two most basic (and most widely used) evaluation metrics in IR, upon which many others are based
- Precision is the fraction of the set of retrieved documents (Ret) that are relevant (i.e. that are also in Rel)
- Recall is the fraction of the relevant documents (Rel) that have been retrieved (i.e. they are also in Ret).

Document Corpus



$$\text{Precision} = \frac{|R \cap A|}{|A|}$$

$$\text{Recall} = \frac{|R \cap A|}{|R|}$$

- $Rel = \{d_3, d_5, d_9, d_{25}, d_{39}, d_{44}, d_{56}, d_{71}, d_{89}, d_{123}\}$

- Ret:

1 d_{123}

2 d_{84}

3 d_{56}

4 d_6

5 d_8

6 d_9

7 d_{511}

8 d_{129}

9 d_{187}

10 d_{25}

11 d_{38}

12 d_{48}

13 d_{250}

14 d_{113}

15 d_3

- Number of relevant documents: $|Rel| = 10$
- Number of retrieved documents: $|Ret| = 15$
- Number of relevant documents retrieved: $|Rel \cap Ret| = 5$
- Precision = $\frac{|Rel \cap Ret|}{|Ret|} = \frac{5}{15} = 0.33$
- Recall = $\frac{|Rel \cap Ret|}{|Rel|} = \frac{5}{10} = 0.5$

- The two metrics of precision and recall are often **inversely** related: as one increases the other decreases
- Precision will be high whenever a system is good at avoiding non-relevant documents
 - A system can achieve very high precision by retrieving very few documents
- Recall will be high whenever a system finds many relevant documents
 - A system can achieve 100% recall simply by retrieving all the documents in the collection

- Which is most important?
 - That is task-dependent!
- Ideally every IR system would have both recall and precision of 100% (the answer set is equal to the relevant set)

- Users searching the web want **high precision** (i.e. they want the returned documents to be relevant)
- Because of the size of the web, there are very many documents that will help satisfy the information need, so the user does not need to examine all of them
- Instead, the user wishes to avoid wasting time looking at non-relevant documents

- A patent lawyer researching a patent must ensure that they get all of the relevant documents and hence they want **high recall**
- If any relevant documents are missed, this may have serious consequences, so it is essential that all relevant documents are returned
- They will tolerate lower precision to facilitate this (i.e. they are more likely to be willing to read through some non-relevant documents)

- For older, smaller document collections, the calculation of precision and recall was easy
- Nowadays, because document collections are so huge, it is very difficult to identify all the relevant documents for every query
- This makes it often impossible to calculate recall accurately
- Thus, precision tends to be preferred, in addition to other metrics based on precision

- Basic Precision is a set-based, unranked retrieval metric (as is recall)
- It is a single-value metric based on the entire list of results that was returned by an IR system
- It does not assess the way the results are ranked
- Most IR systems return ranked lists, not sets of documents, so users are most likely to look at the top of the list first

Rank	IR 1	IR 2
1	N	R
2	N	R
3	N	N
4	R	N
5	R	N

- N is non-relevant, R is relevant
- Precision for both systems is 40%
- But clearly IR system 2 is better!

Information Retrieval

Evaluation

Dr. Seán Russell

School of Computer Science,
University College Dublin

Week 7



Table of Contents

- 1 Precision / Recall
- 2 Single Value Metrics
- 3 Precision at n ($P@n$)
- 4 R-Precision
- 5 Mean Average Precision (MAP)
- 6 Complete and Incomplete Judgements
- 7 Binary Preference
- 8 Normalised Discounted Cumulated Gain (NDCG)
- 9 Which metric should I use?

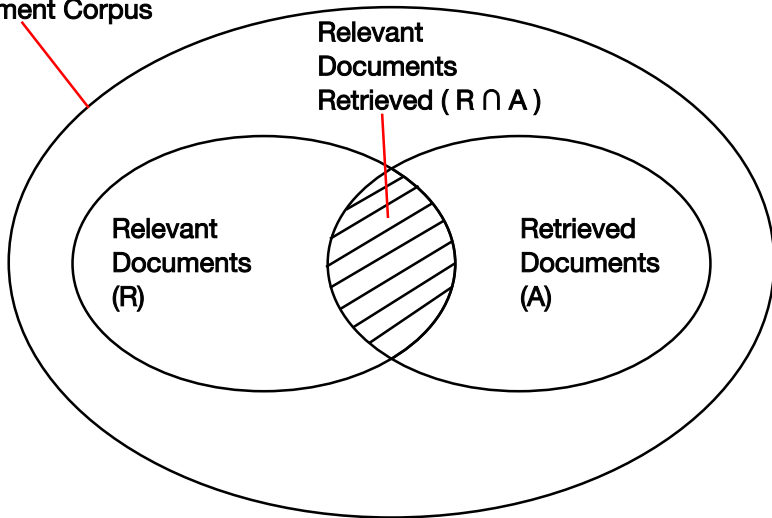
- Below are the commonly used metrics when evaluating IR systems
 - Precision/Recall
 - Precision @ n/R-precision
 - Mean Average Precision (MAP)
 - bPref
 - NDCG

Section Contents

- 1 Precision / Recall
 - Example
 - Precision vs. Recall
 - High Precision Example
 - High Recall Example
 - Modern Use of Precision / Recall
 - Problems with Precision

- Precision and Recall are the two most basic (and most widely used) evaluation metrics in IR, upon which many others are based
- Precision is the fraction of the set of retrieved documents (Ret) that are relevant (i.e. that are also in Rel)
- Recall is the fraction of the relevant documents (Rel) that have been retrieved (i.e. they are also in Ret).

Document Corpus



$$\text{Precision} = \frac{|R \cap A|}{|A|}$$

$$\text{Recall} = \frac{|R \cap A|}{|R|}$$

- $Rel = \{d_3, d_5, d_9, d_{25}, d_{39}, d_{44}, d_{56}, d_{71}, d_{89}, d_{123}\}$

- Ret:

1 d_{123}

2 d_{84}

3 d_{56}

4 d_6

5 d_8

6 d_9

7 d_{511}

8 d_{129}

9 d_{187}

10 d_{25}

11 d_{38}

12 d_{48}

13 d_{250}

14 d_{113}

15 d_3

- Number of relevant documents: $|Rel| = 10$
- Number of retrieved documents: $|Ret| = 15$
- Number of relevant documents retrieved: $|Rel \cap Ret| = 5$
- Precision = $\frac{|Rel \cap Ret|}{|Ret|} = \frac{5}{15} = 0.33$
- Recall = $\frac{|Rel \cap Ret|}{|Rel|} = \frac{5}{10} = 0.5$

- The two metrics of precision and recall are often **inversely** related: as one increases the other decreases
- Precision will be high whenever a system is good at avoiding non-relevant documents
 - A system can achieve very high precision by retrieving very few documents
- Recall will be high whenever a system finds many relevant documents
 - A system can achieve 100% recall simply by retrieving all the documents in the collection

- Which is most important?
 - That is task-dependent!
- Ideally every IR system would have both recall and precision of 100% (the answer set is equal to the relevant set)

- Users searching the web want **high precision** (i.e. they want the returned documents to be relevant)
- Because of the size of the web, there are very many documents that will help satisfy the information need, so the user does not need to examine all of them
- Instead, the user wishes to avoid wasting time looking at non-relevant documents

- A patent lawyer researching a patent must ensure that they get all of the relevant documents and hence they want **high recall**
- If any relevant documents are missed, this may have serious consequences, so it is essential that all relevant documents are returned
- They will tolerate lower precision to facilitate this (i.e. they are more likely to be willing to read through some non-relevant documents)

- For older, smaller document collections, the calculation of precision and recall was easy
- Nowadays, because document collections are so huge, it is very difficult to identify all the relevant documents for every query
- This makes it often impossible to calculate recall accurately
- Thus, precision tends to be preferred, in addition to other metrics based on precision
 - Note: precision can be calculated without needing to know all of the relevant documents, only how many of the returned documents are relevant

- Basic Precision is a set-based, unranked retrieval metric (as is recall)
- It is a single-value metric based on the entire list of results that was returned by an IR system
- It does not assess the way the results are ranked
- Most IR systems return ranked lists, not sets of documents, so users are most likely to look at the top of the list first

Rank	IR 1	IR 2
1	N	R
2	N	R
3	N	N
4	R	N
5	R	N

- N is non-relevant, R is relevant
- Precision for both systems is 40%
- But clearly IR system 2 is better!

Section Contents

2 Single Value Metrics

- The rest of the metrics that we will look at use only a single metric to calculate performance
- These metrics calculate a score for a set of results for just one query
- Of course, when evaluating an IR system, we should use several queries for evaluation
- In this situation, we must first calculate the score for each query, and then get the average over all the queries

Section Contents

3 Precision at n ($P@n$)

- What users want
- Precision at n ($P@n$)
- Problems

- When users make use of an IR system, it pays to consider what they actually want
- Particularly for web search systems where people typically look no further than the first few results
- For example, we may be interested in the precision after 10 documents have been retrieved
 - Known as “Precision at 10”, or “@10”
- This is a measure of the quality of the results that a user is likely to look at

- $$P@n = \frac{|\text{Relevant in top } n \text{ results}|}{n}$$

1 d_{123}

2 d_{84}

3 d_{56}

4 d_6

5 d_8

6 d_9

7 d_{511}

8 d_{129}

9 d_{187}

10 d_{25}

11 d_{38}

12 d_{48}

13 d_{250}

14 d_{113}

15 d_3

- $$P@3 = \frac{2}{3} = 0.67$$

- $$P@6 = \frac{3}{6} = 0.5$$

- $$P@10 = \frac{4}{10} = 0.4$$

- One problem with using $P@n$ is that its performance is affected by the number of relevant documents available
- For a query that has only 10 relevant documents, a good $P@10$ score is difficult
- If there are 1,000 relevant documents, a good $P@10$ score is easy

Section Contents

4

R-Precision

- Example
- Another Example

- A similar metric is R-precision, where we are interested in the precision after the top R documents are returned
 - where R is the number of relevant documents for that query
- This is similar to $P@n$ except that n is not fixed for all queries
 - because the number of relevant documents will be different for each query

- $$R - Precision = \frac{|Relevant\ in\ top\ r\ results|}{r}$$

- where r is the number of relevant documents that are available for this query

1 d_{123}

4 d_6

7 d_{511}

10 d_{25}

13 d_{250}

2 d_{84}

5 d_8

8 d_{129}

11 d_{38}

14 d_{113}

3 d_{56}

6 d_9

9 d_{187}

12 d_{48}

15 d_3

- $$R - Precision = \frac{4}{10} = 0.4$$

- $Rel = \{d_3, d_{56}, d_{129}\}$

1	d_{123}	4	d_6	7	d_{511}	10	d_{25}	13	d_{250}
2	d_{84}	5	d_8	8	d_{129}	11	d_{38}	14	d_{113}
3	d_{56}	6	d_9	9	d_{187}	12	d_{48}	15	d_3

- $R - Precision = \frac{1}{3} = 0.33$

- There are only 3 relevant documents
- There is only one relevant document in the first 3 results

Section Contents

5 Mean Average Precision (MAP)

- Calculating MAP
- Example - Average Precision
- Mean Average Precision

- Mean Average Precision (MAP) has for many years been the most commonly used metric in IR literature to evaluate the performance of systems
- It is a single-value metric based on precision
- Unlike simple precision, it rewards systems that rank relevant documents at the beginning of the results returned
- Unlike P@10, it continues to examine the later stages of the ranked list, although with lesser weight

• Calculating Mean Average Precision

- Firstly, we must calculate the precision at each **recall point** (at each rank where a relevant document is found).
- The **Average Precision** for this query is found by dividing the sum of these precision calculations by the total number of relevant documents
- As with most metrics, for multiple queries the same procedure must be performed for each. We must calculate the mean of the queries' average precision values, giving us **Mean Average Precision**.

- Average Precision = $\frac{\sum_{k=1}^n P@k \times rel@k}{r}$
 - n is the number of retrieved documents
 - $P@k$ is precision at rank k
 - $rel@k$ is 1 if the document at rank k is relevant, or 0 if not

① d_{123} ④ d_6 ⑦ d_{511} ⑩ d_{25} ⑬ d_{250} ② d_{84} ⑤ d_8 ⑧ d_{129} ⑪ d_{38} ⑭ d_{113} ③ d_{56} ⑥ d_9 ⑨ d_{187} ⑫ d_{48} ⑮ d_3 ① $P@1 = 1.00$ ③ $P@6 = 0.50$ ⑤ $P@15 = 0.33$ ② $P@3 = 0.67$ ④ $P@10 = 0.40$

- Average Precision (for this query):

$$\frac{1.0 + 0.67 + 0.5 + 0.4 + 0.33}{10} = 0.29$$

- Average Precision (for this query):
$$\frac{1.0+0.67+0.5+0.4+0.33}{10} = 0.29$$
- This is the AP for one query
- To get the Mean Average Precision (MAP), we calculate the average over all queries

Section Contents

6

Complete and Incomplete Judgements

- Complete Judgements
- Incomplete Judgements
- How do incomplete relevance judgments happen?
- Pooling

- All of the evaluation techniques we have mentioned so far are based on the Cranfield Paradigm
- In this, test collections and queries are created that have a known set of relevant documents associated with them
- The point is that for each query, every document in the collection is judged to be “relevant” or “non-relevant”
- Where this occurs, we say that we have “complete relevance judgments”

- With smaller collections, this Cranfield Paradigm is perfect
 - i.e. complete relevance judgments exist
- However, as document collections have become larger, complete judgments have become less common and we have **incomplete judgments**
- This means that some documents have not been judged so they may or may not be relevant to test queries
- With large-scale IR collections (such as those based on the web), this complete judgment is impossible to achieve, with potentially billions of documents to be judged for relevance against hundreds of queries

- When a corpus is very large, it is not feasible to judge the relevance of every document to every query
- The Text REtrieval Conference (TREC) sets IR tasks each year (called “tracks”)
- For ad hoc retrieval tasks (the type of IR we are learning about), they release a corpus and some standard queries (called topics)
- Participating groups are invited to use their systems to search the corpus for each topic, and submit a set of results to the organisers
- The organisers use a system called pooling to decide which documents to judge

- In its basic form, the top n documents returned in each set of results for every query are gathered into a pool
- Human assessors then judge the relevance of only the documents in the pool for each query
- This is based on the assumption that different groups using a variety of state-of-the-art retrieval techniques should, collectively, find all the relevant documents that were available in the corpus
- In practice, this is not guaranteed – there may still be relevant documents that were available, but that no system actually retrieved in response to the query

Section Contents

- 7 Binary Preference
 - bpref
 - Example
 - Binary Preference
 - Calculating bpref
 - bPref-10
 - Effect of bPref

- For the evaluation metrics we have seen so far, they are simplified by **assuming** that unjudged documents are non-relevant
- It was noticed that many unjudged documents had the effect of lowering evaluation score
 - **NOTE:** This does not mean that the retrieval was worse: whether a document is relevant or not is not affected by whether somebody has judged it
- This is a problem because evaluation scores no longer accurately reflect the effectiveness of retrieval
- For example, what would the effect on the Average Precision score be in our example of nobody had judged document d_9 ?

Rank	Document	d_9 judged	d_9 unjudged
1	d_{123}	$P = \frac{1}{1} = 1.00$	$P = \frac{1}{1} = 1.00$
2	d_{84}		
3	d_{56}	$P = \frac{2}{3} = 0.67$	$P = \frac{2}{3} = 0.67$
4	d_6		
5	d_8		
6	d_9	$P = \frac{3}{6} = 0.5$	
7	d_{511}		
8	d_{129}		
9	d_{187}		
10	d_{25}	$P = \frac{4}{10} = 0.4$	$P = \frac{3}{10} = 0.3$
11	d_{38}		
12	d_{48}		
13	d_{250}		
14	d_{113}		
15	d_3	$P = \frac{5}{15} = 0.33$	$P = \frac{4}{15} = 0.27$

	d_9 judged	d_9 unjudged
Average Precision	$\frac{1+0.67+0.5+0.4+0.33}{10} = 0.29$	$\frac{1+0.67+0.3+0.27}{9} = 0.25$

- The idea behind bpref is that these unjudged documents should not impact so largely on the evaluation score
 - From the paper: Buckley & Voorhees, “Retrieval Evaluation with Incomplete Information”, SIGIR 2004
- bpref ignores documents that have not been judged
- The only documents considered are those that were judged relevant or judged non-relevant
- This is an accepted metric for large-scale IR systems where complete relevance judgments are impossible to achieve

- bpref, for a query with R relevant documents is calculated as

- $$B = \frac{1}{R} \sum_{r \in R} 1 - \frac{|n \text{ ranked higher than } r|}{R}$$

- Where n is a member of the first R judged non-relevant documents

- In other words:

- For each relevant document in that was retrieved in the answer set:

- Count the number of non-relevant documents above it in the result set (this is $|n \text{ ranked higher than } r|$)

- This cannot be greater than the total number of relevant documents (R)

- The score for that document is $1 - \frac{|n \text{ ranked higher than } r|}{R}$

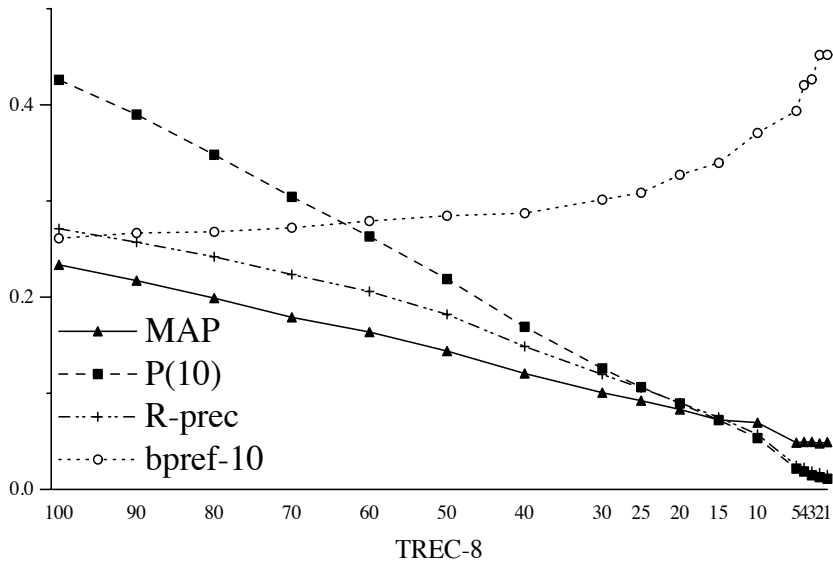
- Average this score over all the judged relevant documents

- Key:
 - R: Relevant
 - N: Non-Relevant
 - U: Unjudged
- Assume a total of 4 available judged relevant documents (R=4)
- bpref = $(0.75 + 0.75 + 0 + 0) / 4 = 0.375$

Document	bPref Contribution
N	
R	$1 - \frac{1}{4} = 0.75$
U	
R	$1 - \frac{1}{4} = 0.75$
U	
N	
N	
N	
R	$1 - \frac{4}{4} = 0$
N	
R	$1 - \frac{4}{4} = 0$

- This works well in most situations
- However, when R is very small (i.e. there are only one or two relevant documents) it fails
- To overcome this problem, we can instead use bpref-10 , which is given by
 - $$\text{bpref10} = \frac{1}{R} \sum_{r \in R} 1 - \frac{|n \text{ ranked higher than } r|}{10+R}$$
 - where n is a member of the first $10+R$ judged non-relevant documents

- From Buckley and Voorhees 2004 it can be seen that when complete judgments are available, there is no noticeable difference between MAP and bpref-10
- As the judgments become less complete, bpref is more stable than the others
 - y-axis: evaluation metric score
 - x-axis: percentage of judgments used (they kept removing judged documents and re-evaluating)



Section Contents

8

Normalised Discounted Cumulated Gain (NDCG)

- Motivation
- Graded Relevance
- Example
- Cumulated Gain
- Discounted Cumulated Gain
- DCG: Analysis
- Normalised DCG (NDCG)
- NDCG: Ideal DCG
- NDCG Calculation
- Features of NDCG

- The metrics so far make use of **binary** relevance judgments
 - Documents are judged to be **relevant** or **non-relevant**
 - Any document that helps to satisfy an information need in any way is considered to be **relevant**
- **BUT**: In reality, some documents are more relevant than others
 - Normalised Discounted Cumulated Gain (NDCG) supports graded relevance judgments

- NDCG rewards systems that
 - Rank highly-relevant documents ahead of mildly relevant ones
 - Position relevant documents in early positions in the ranking

- The first thing that is required is a set of graded relevance judgments
 - Let us assume that a 0-3 scale where 3 is a highly relevant document and 0 is a non-relevant document
- $R = \{[d_3, 3], [d_5, 3], [d_9, 3], [d_{25}, 2], [d_{39}, 2], [d_{44}, 2], [d_{56}, 1], [d_{71}, 1], [d_{89}, 1], [d_{123}, 1]\}$
 - d_3 , d_5 and d_9 are highly relevant documents
 - d_{25} , d_{39} , and d_{44} are relevant documents
 - d_{56} , d_{71} , d_{89} , and d_{123} are somewhat relevant documents
 - Everything else is non-relevant.

- Let's revisit the example from before
- This time, relevant documents are also shown with the degree of relevance attached
- We create a gain vector that records the relevance level at each rank
- $G = (1, 0, 1, 0, 0, 3, 0, 0, 0, 2, 0, 0, 0, 0, 3)$
- This is the starting point for our calculations

Rank	Doc	G
1	d_{123} (r: 1)	1
2	d_{84}	0
3	d_{56} (r: 1)	1
4	d_6	0
5	d_8	0
6	d_9 (r: 3)	3
7	d_{511}	0
8	d_{129}	0
9	d_{187}	0
10	d_{25} (r: 2)	2
11	d_{38}	0
12	d_{48}	0
13	d_{250}	0
14	d_{113}	0
15	d_3 (r: 3)	3

- Next, we calculated a cumulated gain vector (also sometimes called a “cumulative gain vector”).
- Each rank i has its own CG value

$$CG[i] = \begin{cases} G[1] & i = 1 \\ G[i] + CG[i - 1] & i < 1 \end{cases}$$
- e.g. at rank 10:
 - $CG[10] = G[10] + CG[9] = 2 + 5 = 7$

Rank	Doc	G	CG
1	d_{123} (r: 1)	1	1
2	d_{84}	0	1
3	d_{56} (r: 1)	1	2
4	d_6	0	2
5	d_8	0	2
6	d_9 (r: 3)	3	5
7	d_{511}	0	5
8	d_{129}	0	5
9	d_{187}	0	5
10	d_{25} (r : 2)	2	7
11	d_{38}	0	7
12	d_{48}	0	7
13	d_{250}	0	7
14	d_{113}	0	7
15	d_3 (r: 3)	3	10

- The idea is that each relevant document we find should add to the gain (i.e. the overall usefulness of the list)
- More highly relevant documents add more to the gain than less relevant documents
- BUT: documents late in the list can add as much to the gain as documents early in the list
 - e.g. at rank 15, a highly-relevant document adds 3 to the cumulated gain: the same as at rank 6

- This motivates us to create a vector for Discounted Cumulated Gain (DCG)
- Here, later documents add less to the gain than earlier ones
- It is calculated in the same way as CG, except that the gain at each rank is discounted by dividing it by the log of the rank (except for the first rank, which is unaffected).
- Each rank i has its own DCG value

$$DCG[i] = \begin{cases} G[1] & i = 1 \\ \frac{G[i]}{\log_2(i)} + CG[i - 1] & i < 1 \end{cases}$$

Rank	Doc	G	CG	Calculation	DCG
1	d_{123} (r: 1)	1	1	1	1
2	d_{84}	0	1		1
3	d_{56} (r: 1)	1	2	$\frac{1}{\log_2(3)} + 1$	1.6
4	d_6	0	2		1.6
5	d_8	0	2		1.6
6	d_9 (r: 3)	3	5	$\frac{3}{\log_2(6)} + 1.6$	2.8
7	d_{511}	0	5		2.8
8	d_{129}	0	5		2.8
9	d_{187}	0	5		2.8
10	d_{25} (r : 2)	2	7	$\frac{2}{\log_2(10)} + 2.8$	3.4
11	d_{38}	0	7		3.4
12	d_{48}	0	7		3.4
13	d_{250}	0	7		3.4
14	d_{113}	0	7		3.4
15	d_3 (r: 3)	3	10	$\frac{3}{\log_2(15)} + 3.4$	4.2

DCG=(1.0, 1.0, 1.6, 1.6, 1.6, 2.8, 2.8, 2.8, 2.8, 2.8, 3.4, 3.4, 3.4, 3.4, 4.2)

- We have now calculated a Discounted Cumulated Gain vector for a query:
 $DCG = (1.0, 1.0, 1.6, 1.6, 1.6, 2.8, 2.8, 2.8, 2.8, 3.4, 3.4, 3.4, 3.4, 4.2)$
- This shows how finding relevant documents increases the quality of the results to that point
 - More relevant documents contribute more to gain
 - Relevant documents found earlier in the ranked list also contribute more to gain

- We have now calculated a Discounted Cumulated Gain vector for a query:

DCG=(1.0, 1.0, 1.6, 1.6, 1.6, 2.8, 2.8, 2.8, 2.8, 3.4, 3.4, 3.4, 3.4, 4.2)

- BUT: By itself, this is not easy to compare with others
 - Is 4.2 a good score for this query?
 - Which figure(s) do we choose to compare?
- Most evaluation metrics give a single value that is in the range between 0 and 1.
- **Normalised** DCG allows us to achieve this

- Normalised Discounted Cumulated Gain is calculated by comparing the DCG vector against an Ideal DCG vector
- The Ideal DCG vector is the DCG vector that we would see if the IR system had perfect retrieval
 - i.e. it begins with all the documents of relevance level 3
 - then it includes all the documents of relevance level 2
 - then it includes all the documents of relevance level 1
- We calculate an Ideal DCG vector to be the same length as the DCG vector (with 0 relevance values inserted at the end if there are not enough relevant documents)

Rank	IG	Calculation	IDCG
1	3	3	3
2	3	$\frac{3}{\log_2(2)} + 3.0$	6.0
3	3	$\frac{3}{\log_2(3)} + 6.0$	7.9
4	2	$\frac{2}{\log_2(4)} + 7.9$	8.9
5	2	$\frac{2}{\log_2(5)} + 8.9$	9.8
6	2	$\frac{2}{\log_2(6)} + 9.8$	10.5
7	1	$\frac{1}{\log_2(7)} + 10.5$	10.9
8	1	$\frac{1}{\log_2(8)} + 10.9$	11.2
9	1	$\frac{1}{\log_2(9)} + 11.2$	11.5
10	1	$\frac{1}{\log_2(10)} + 11.5$	11.8
11	0		11.8
12	0		11.8
13	0		11.8
14	0		11.8
15	0		11.8

- The IDCG vector represents the best possible DCG scores that a perfect IR system would achieve
- $IDCG = (3.0, 6.0, 7.9, 8.9, 9.8, 10.5, 10.9, 11.2, 11.5, 11.8, 11.8, 11.8, 11.8, 11.8, 11.8)$
- We can normalise the DCG by dividing the score that was actually achieved at each rank by the ideal score, to yield a score between 0 and 1

Rank	1	2	3	4	5	6	7	8	9	10
DCG	1.0	1.0	1.6	1.6	1.6	2.8	2.8	2.8	2.8	3.4
IDGC	3.0	6.0	7.9	8.9	9.8	10.5	10.9	11.2	11.5	11.8
NDCG	0.33	0.17	0.20	0.18	0.16	0.27	0.26	0.25	0.24	0.29

Rank	11	12	13	14	15
DCG	3.4	3.4	3.4	3.4	4.2
IDGC	11.8	11.8	11.8	11.8	11.8
NDCG	0.29	0.29	0.29	0.29	0.36

- We still have the problem that we have 15 different scores for the evaluation
- This is solved in a similar way to Precision@n: we choose a rank to measure NDCG at
- NDCG@10 is a very commonly used metric: here it is 0.29

- Combines document ranks and graded relevance judgments
- Single measure of quality at any rank, without needing to know recall
- $NDCG@n$ only considers relevant documents found to that point: not affected by many relevant documents being found very late
- Gives less weight to relevant documents found late in the ranking

Section Contents

9 Which metric should I use?

- We have looked at numerous different metrics for IR evaluation
- All have their advantages and disadvantages
- We need to use an appropriate metric in order to evaluate an IR system's performance
- How “performance” is defined is dependent on the final use of the system:
 - web search;
 - intranet search;
 - research environment;
 - desktop search;
 - legal search;
 - etc...

- A general rule is that if complete judgments are available any metric can be used
- For each of these, a single value is normally reported, which is the average value from all the queries used in the evaluation
- MAP gives a good indication of performance within a single metric as it is averaging the results over multiple queries
- For collections that do not have complete judgments, bPref is a more suitable metric

- For tasks such as web search (due to user behaviour), metrics like $P@10$ might be used
- If graded relevance judgments are available, NDCG is preferred: this has continually gained in popularity in the last few years
- In reality, most evaluations use multiple metrics

IR Pipelines and Modern IR

COMP3009J: Information Retrieval

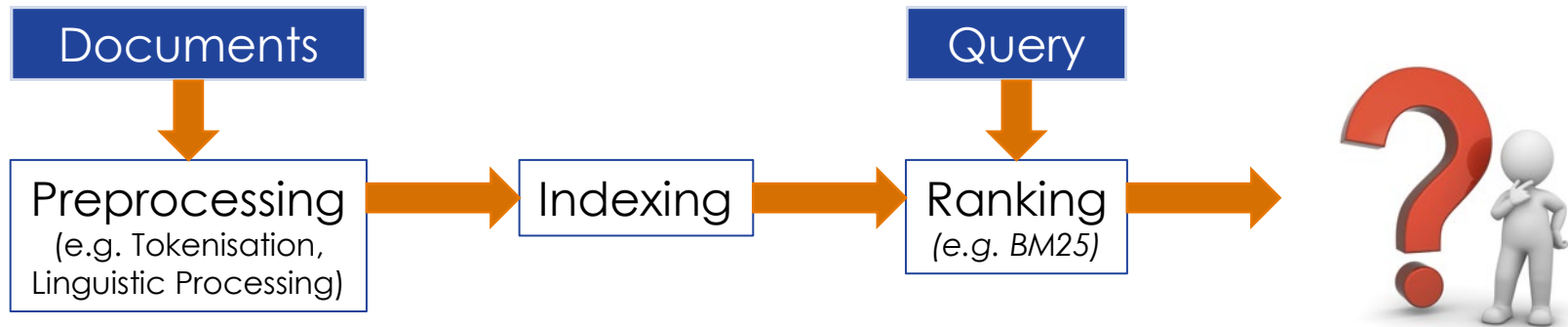
Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Recap: where are we now?

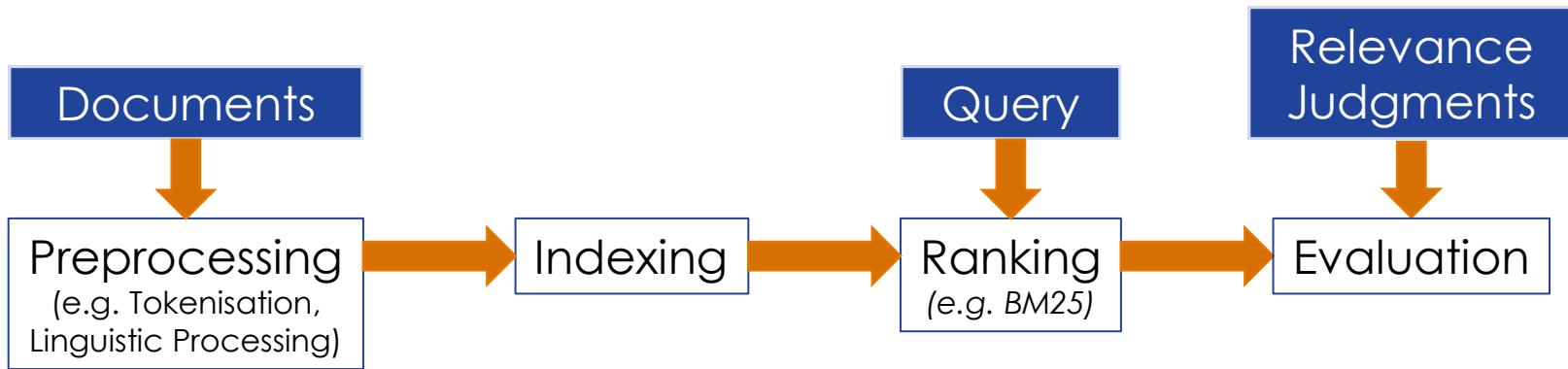
- So far we have looked at **classic** Information Retrieval, much of which is still relevant today. We know...
 - What IR is.
 - How an index can be built.
 - How several models can perform retrieval.
 - How evaluation can be done.
- Thus all the basic components are in place.
- Next we will look at how this process can be improved.

Classical IR Pipeline



1. Documents are **preprocessed** to prepare them for use in the IR system.
2. The **indexer** creates a suitable data structure so that retrieval can happen.
3. A **ranking** method matches queries against the index to produce a **ranked list of results** for the user (or to an evaluation step).

Classical IR Pipeline (Experimental)



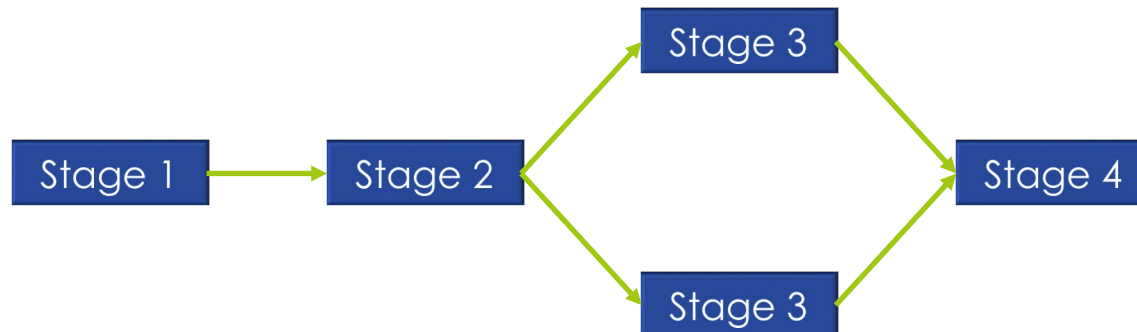
- Alternative pipeline with Evaluation component.
 - Used in experimentation – to evaluate the performance of the system that has been designed.
 - (this is evaluation in terms of the system's **effectiveness**)

Aside: Pipeline Design Pattern

- We call this design pattern a **pipeline** because the output of one process becomes the input of the next stage in the process.
- This is similar to a **linear pipeline**, where each stage happens sequentially.



- An alternative is a **non-linear** pipeline, where some stages can happen in parallel.

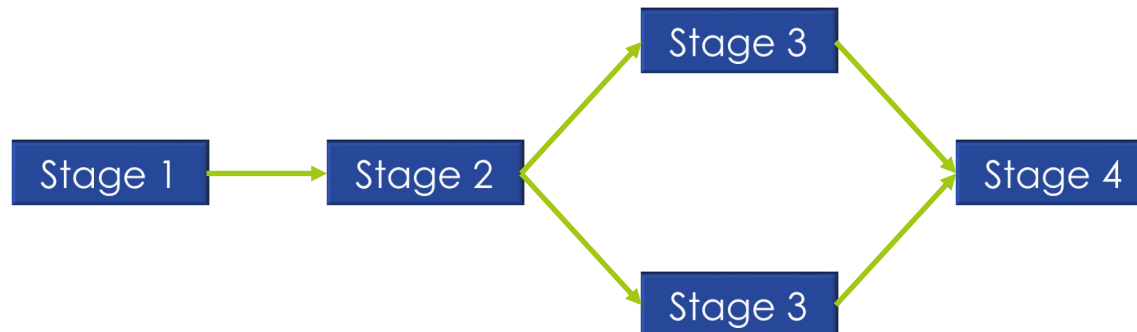


Aside: Pipeline Design Pattern

- We call this design pattern a **pipeline** because the output of one process becomes the input of the next stage in the process.

Pipelines are basically **Directed Acyclic Graphs (DAGs)** where nodes are steps and edges are dependencies.

- An alternative is a **non-linear** pipeline, where some stages can happen in parallel.



Aside: Pipeline Design Pattern

- ▣ We call this design pattern a **pipeline** because the output of one process becomes the input of the next stage in the process.

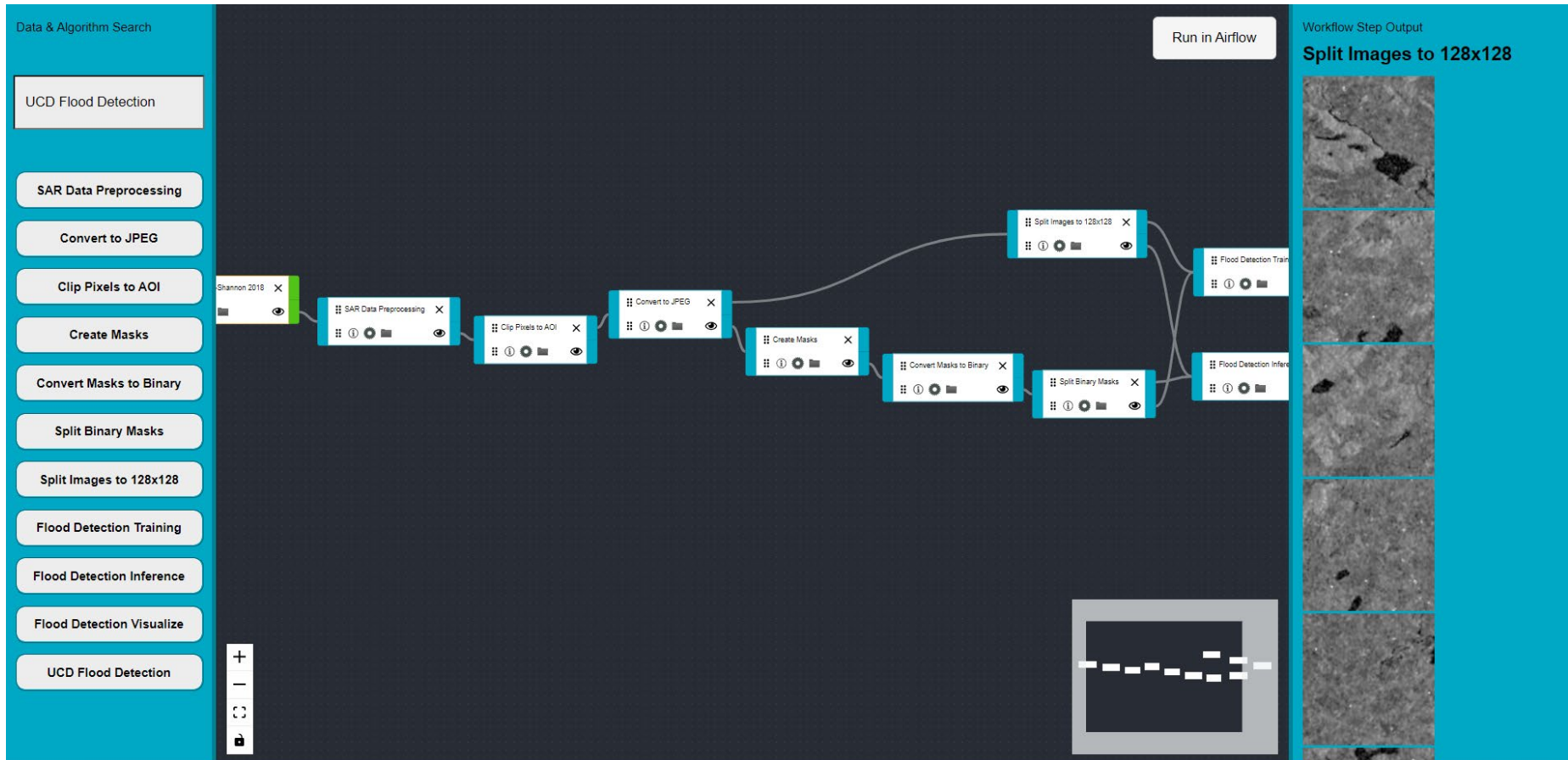
Pipelines are basically **Directed Acyclic Graphs (DAGs)** where nodes are steps and edges are dependencies.

- ▣ An alternative is a **non-linear** pipeline, where some stages can happen in parallel.

Pipelines are widely used in software practice:

CI/CD uses pipelines to compile, test, package and deploy code;
ML Ops applies the same principles to machine learning;
Agentic Workflows are communication pipelines for LLM agents

Aside: Pipeline Design Pattern



Issues with Classical Pipeline (so far)

- **Vocabulary mismatch** is a problem. The words chosen by the user may not match the words used in a relevant document.
- **polysemy**: the same word can have different meanings (e.g. “bank”).
- **synonymy**: two different words can have the same meaning (e.g. plane/airplane/aeroplane/aircraft).
- How can we be confident that our choice of **retrieval model** is the best for each query?
 - Could we use a combination of ranking models instead of just one?

Issues with Classical Pipeline (so far)

- Only the **document contents** are taken into account.
 - Particularly on the web, there is a lot more information available.
 - The **link structure** of the web:
 - Where does this page link to?
 - What other pages link to this page?
 - What text do other pages use in their links to this page (in their **anchor text**)?
 - The **URL** may contain useful information?
 - For a large search engines, they have **query logs** to record previous user's behaviour when they give the same query.

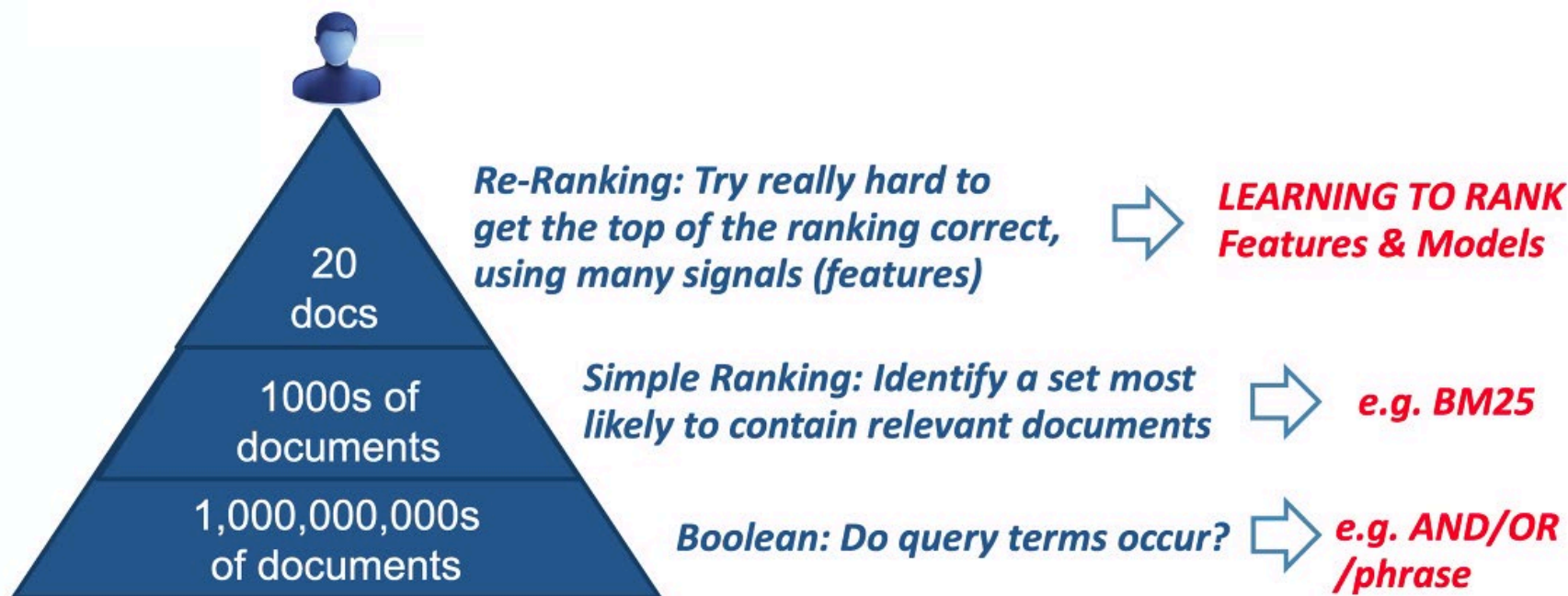
Issues with Classical Pipeline

- ❑ Doesn't make use of recent advances in **machine learning** (especially **deep learning**) and **Natural Language Processing** (NLP) and **Natural Language Understanding** (NLU).
- ❑ Machine Learning has transformed most data-processing and data-analysis tasks.
- ❑ Large Language Models (LLMs) such as **BERT** (Google, 2018), **T5** (Google, 2020), **GPT-4** (OpenAI, 2023) have made huge progress in a variety of NLP, NLU and Machine Translation tasks.
- ❑ **BUT...** Language models are *sloooooooooow*...

Speed is of the essence...

■ **Speed is essential.** According to Microsoft:

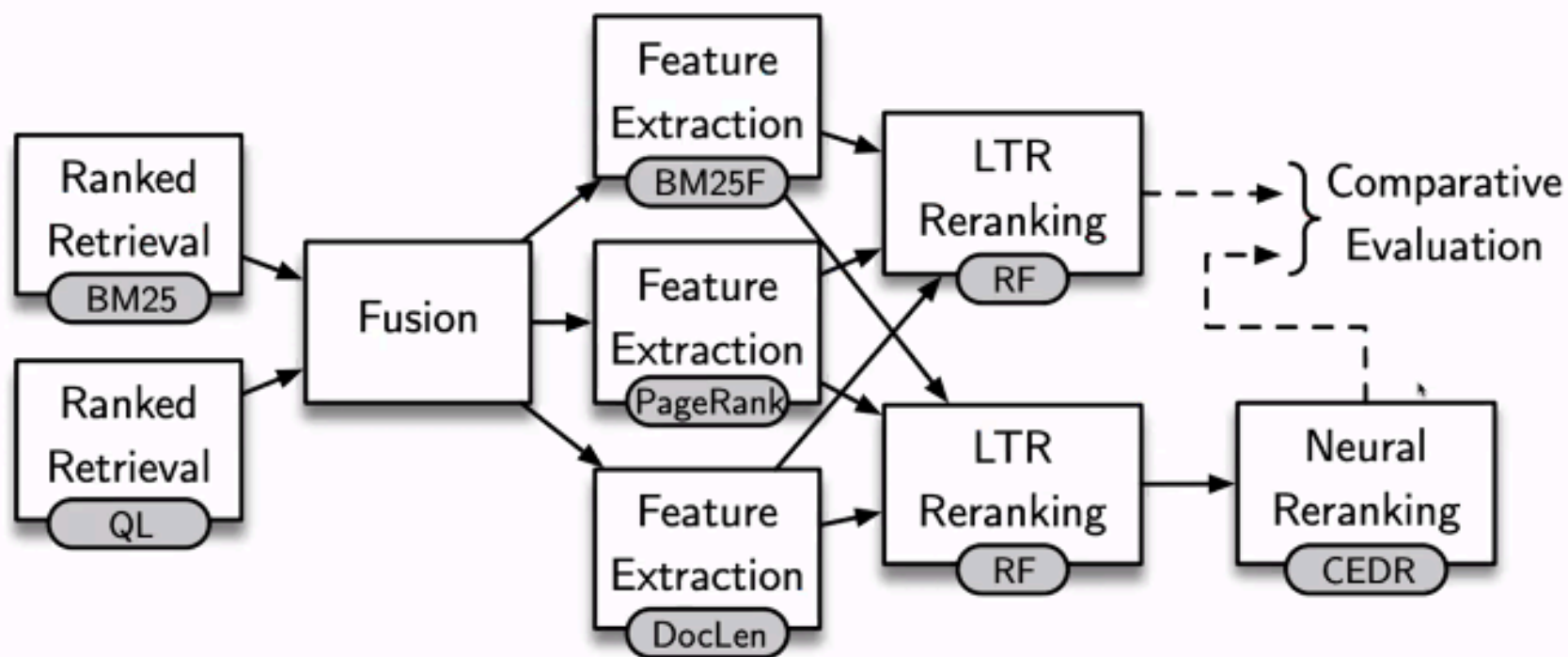
■ “Even a **100ms** latency has been shown to invoke negative user reactions”¹



[1] J Pederson. Query understanding at Bing. SIGIR 2010 Industry Day.

¹ Rosset, C., Jose, D., Ghosh, G., Mitra, B., & Tiwary, S. (2018). Optimizing Query Evaluations Using Reinforcement Learning for Web Search. SIGIR 2018. <https://doi.org/10.1145/3209978.3210127>

A More Complex IR Pipeline



Source: MacAvaney, S., McDonald, C., and Tonolotto, N., IR From Bag-of-words to BERT and Beyond through Practical Experiments, ECIR 2021.

Pseudo-Relevance Feedback

- **Relevance Feedback** is when users are shown a set of documents in response to a query, and tell the system which ones are relevant.
- The system can then use this information to re-run the search (with different term weights) and hopefully find more relevant documents that are similar to the ones the user has said are relevant.

Information Retrieval involves a "conversation" where you align your model of what you are searching for with the machine's model of what it thinks is relevant to your query....

Pseudo-Relevance Feedback

- **Relevance Feedback** is when users are shown a set of documents in response to a query, and tell the system which ones are relevant.
 - The system can then use this information to re-run the search (with different term weights) and hopefully find more relevant documents that are similar to the ones the user has said are relevant.
 - **However**, users don't like giving relevance feedback!
- **Pseudo-Relevance Feedback** simulates relevance feedback by *assuming* that the top k ranked documents are relevant, and then re-running the search in the same way.
 - **Why does this work?** “[T]erms that occur consistently in relevant documents also tend to appear consistently in top ranked documents.”¹
 - **But** it's not perfect: danger of **query drift** towards the topic of the top-ranked documents.

¹ Zhao, L., and Callan, J., Term Necessity Prediction, CIKM 2010. <https://doi.org/10.1145/1871437.1871474>

Pseudo-Relevance Feedback

What if I am interested in **Jaguars**?

ones are relevant.

- The system can then use this information to re-run the search (with different term weights) and hopefully find more relevant documents that are similar to the ones the user has said are relevant.
- **However**, users don't like giving relevance feedback!
- **Pseudo-Relevance Feedback** simulates relevance feedback by *assuming* that the top k ranked documents are relevant, and then re-running the search in the same way.
 - **Why does this work?** “[T]erms that occur consistently in relevant documents also tend to appear consistently in top ranked documents.”¹
 - **But** it's not perfect: danger of **query drift** towards the topic of the top-ranked documents.

¹ Zhao, L., and Callan, J., Term Necessity Prediction, CIKM 2010. <https://doi.org/10.1145/1871437.1871474>

Pseudo-Relevance Feedback

What if I am interested in **Jaguars**?

ones are relevant.

A search could return documents related to:



Jacksonville Jaguars
NFL Team



Jaguar
Cars



Jaguar
Cats

documents also tend to appear consistently in top ranked

Which type of Jaguar could I be interested in?

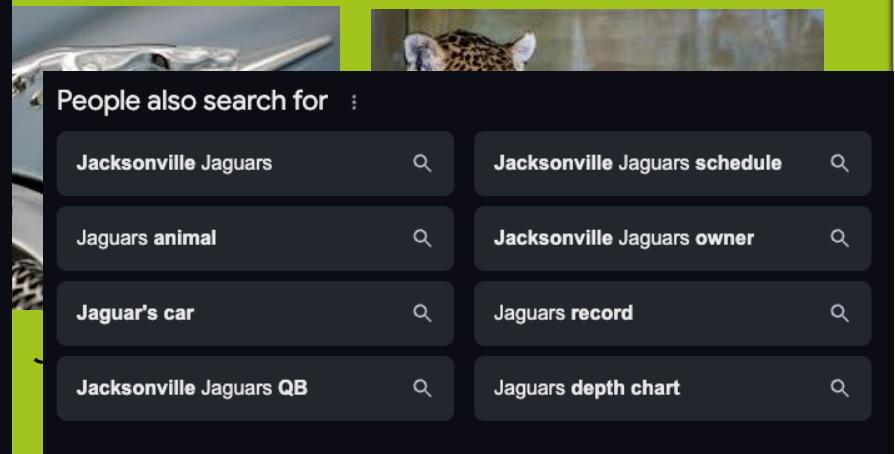
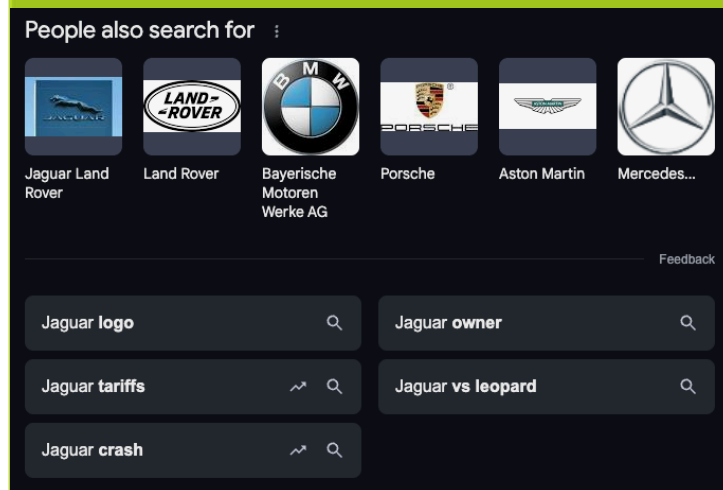
the top-ranked documents.

Pseudo-Relevance Feedback

What if I am interested in **Jaguars**?

ones are relevant.

A search could return documents related to:



Search: Jaguar

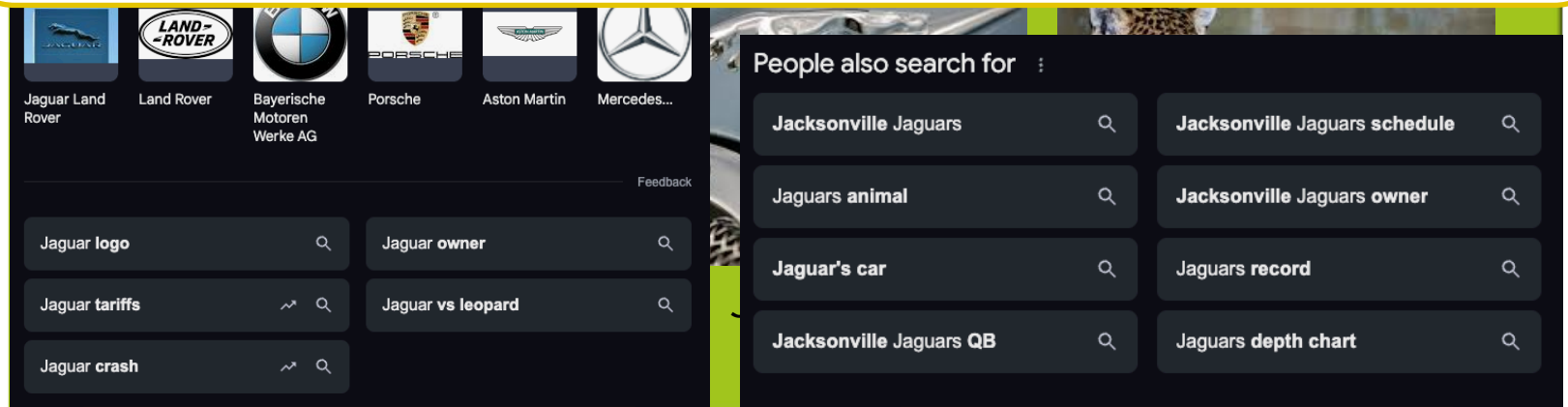
Search: Jaguars

¹ Zhao, L., and Callan, J., Term Necessity Prediction, CIKM 2010. <https://doi.org/10.1145/1871437.1871474>

Pseudo-Relevance Feedback

Key Point: Topic Drift

Depending on what I search for, pseudo-relevance feedback can make incorrect choices about the type of information I want...



Search: Jaguar

Search: Jaguars

Query Expansion

- Related terms are **added to the query** to increase the chance of matching relevant documents.
- Many sources of related terms:
 - A **manually-created thesaurus**, such as WordNet¹.
 - An **automatically-created thesaurus** from some external corpus like a web crawl, or Wikipedia².
 - Word embeddings**, where words are represented by vectors (e.g. word2vec, GloVe, ELMo and BERT-based embeddings).

The screenshot shows a Google search for "aircraft fuel". The search bar at the top contains the text "aircraft fuel". Below the search bar, the results show "About 685,000,000 results (0.69 seconds)". The first result is from Wikipedia, titled "Aviation fuel - Wikipedia". The snippet describes Jet fuel as a clear to straw-colored fuel based on kerosene or a naphtha-kerosene blend. Below this is another Wikipedia result titled "Jet fuel - Wikipedia", which provides more details about Jet fuel or aviation turbine fuel (ATF), including its boiling point, flash point, melting point, and density. The second result is from Shell Global, titled "Aviation Fuel & Gasoline | Aeroplane Fuel | Shell Global", with a snippet stating that today's kerosene jet fuels have been developed from illuminating kerosene used in early gas turbine engines.

¹ <https://wordnet.princeton.edu/>

² For example, through DBPedia (<https://wiki.dbpedia.org/>)

Query Expansion

- Many sources of related terms:
 - **Query Log** mining, where a search engine looks at the behaviour of previous users who have given the same query.
 - **Target-corpus based techniques**, based on the document collection being searched. Two categories:
 - *Distribution-based*: compare the distribution (frequency) of terms in the (pseudo-) relevant documents compared to the whole corpus.
 - *Association-based*: select terms based on their association (co-occurrence) with the query terms.

aircraft fuel

About 685,000,000 results (0.69 seconds)

en.wikipedia.org › wiki › Aviation_fuel ▾

Aviation fuel - Wikipedia

Jet fuel is a clear to straw-colored fuel, based on either an unleaded **kerosene** (**Jet A-1**), or a naphtha-kerosene blend (**Jet B**). Similar to diesel fuel, it can be used in either compression ignition engines or turbine engines.

Jet fuel · Avgas · Category:Aviation fuels · Aviation biofuel

en.wikipedia.org › wiki › Jet_fuel ▾

Jet fuel - Wikipedia

Jet fuel or aviation turbine fuel (ATF, also abbreviated avtur) is a type of **aviation fuel** designed for use in aircraft powered by gas-turbine engines. It is colorless to ...

Boiling point: 176 °C (349 °F; 449 K)	Flash point: 38 °C (100 °F; 311 K)
Melting point: -47 °C (-53 °F; 226 K)	Density: 775.0–840.0 g/L

Water in jet fuel · Military jet fuels · Synthetic jet fuel · USAF synthetic fuel trials

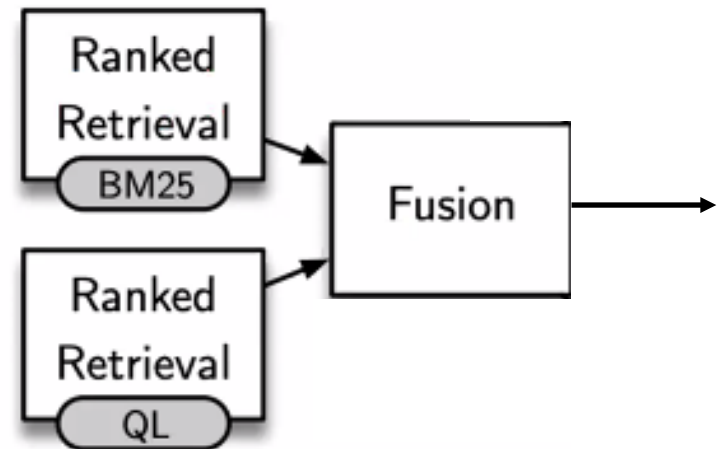
www.shell.com › Business customers › Aviation ▾

Aviation Fuel & Gasoline | Aeroplane Fuel | Shell Global

Today's kerosene jet **fuels** have been developed from the illuminating kerosene used in the early gas turbine engines. These engines needed a **fuel** with good ...

Fusion

- **Input:** a set of results (for one query) from several Ranked Retrieval systems.
- **Output:** a single, combined set of results that are (hopefully) better than the input results in isolation.
- Can be based on:
 - the **scores** calculated by the Ranked Retrieval systems;
 - the **rank** of the documents in each results list;
 - the **probability** of a document being relevant, based on past performance.

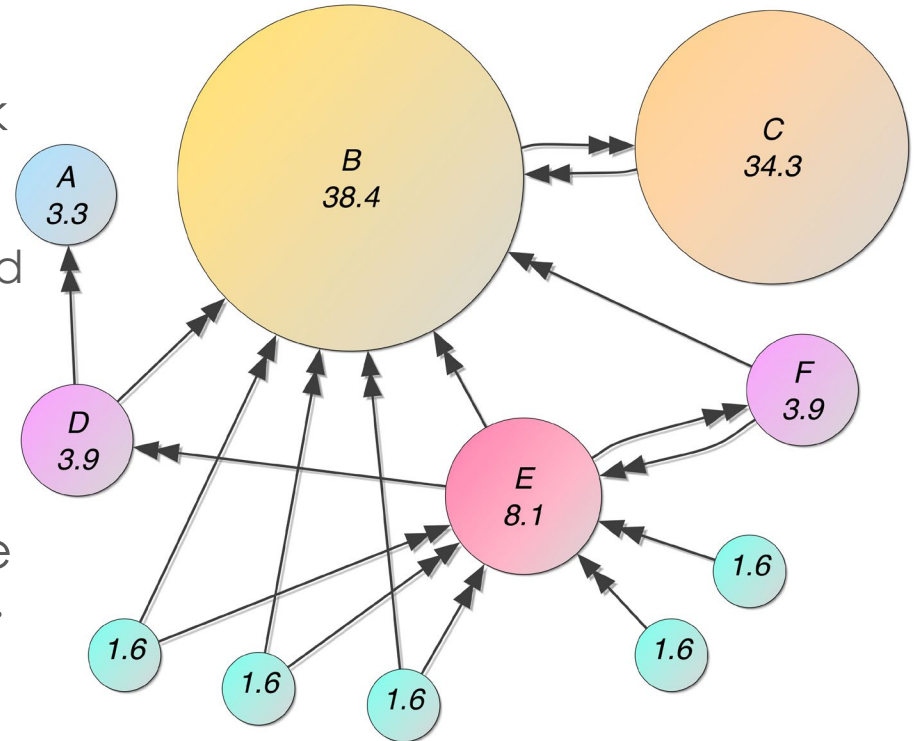


Learning to Rank

- After the initial search has been done by a classic ranked retrieval model (e.g. BM25, or a fused result from several rankers).
- Machine Learning used to re-rank the documents that have been retrieved.
 - **Aim:** improve the result by putting the most-relevant documents at the top of the ranked list of results.
- Takes into account much more information than just the initial ranking score.
 - Each piece of information used is known as a **feature** in Machine Learning.

PageRank: Document Importance

- One feature commonly used in Learning to Rank models is **PageRank**¹.
 - Developed by the founders of Google, and was a key part of Google's success.
- Measures the **importance** of a document based on the **link structure of the web**.
- A document is considered to be **important** if:
 - Many pages link to it; or
 - Other important pages link to it.



¹ Page, L., Brin, S., Motwani, R., and Winograd, T. *The PageRank Citation Ranking: Bringing Order to the Web*. Technical Report. Stanford InfoLab. 2009.

Learning to Rank: Features

- Some examples of features that can be used by Learning to Rank models¹
- **Textual Features:**
 - Term occurrence/non-occurrence
 - Term frequency
 - Inverse Document Frequency
 - Document Length
 - Term Proximity
- **Non-Textual Features:**
 - PageRank
 - URL Depth
 - Document Quality
 - Readability
 - Sentiment
 - Query Clarity

¹ Metzler, D., and Croft, W. B., Linear Feature-Based Models for Information Retrieval, 2007.
<https://doi.org/10.1007/s10791-006-9019-z>

Neural Language Models

- Bidirectional Encoder Representations from Transformers (BERT) is a deep-learning based Large Language Model (LLM) for Natural Language Processing.
 - Developed by Google and released in 2018¹.
 - In simple terms, it learns the relationships between words to aid document understanding
 - It has revolutionised NLP, and has had great success on many NLP tasks.
 - As at 2020, it is used for almost all English-language queries in Google's search engine².



¹ <https://github.com/google-research/bert>

² <https://searchengineland.com/google-bert-used-on-almost-every-english-query-342193>

Neural Language Models

BERT is concerned with **Natural Language Understanding (NLU)**

Its goal is to improve context better understanding the relationship between the tokens in a sentence.

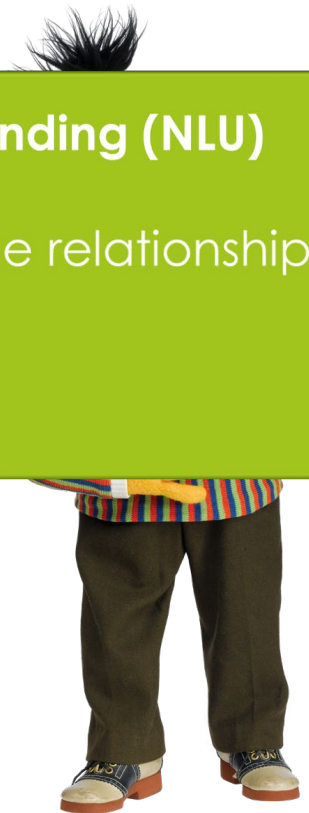
“A man was [MASK] on a river bank.”

between words to aid document understanding

- It has revolutionised NLP, and has had great success on many NLP tasks.
- As at 2020, it is used for almost all English-language queries in Google's search engine².

¹ <https://github.com/google-research/bert>

² <https://searchengineland.com/google-bert-used-on-almost-every-english-query-342193>



Neural Language Models

BERT is concerned with **Natural Language Understanding (NLU)**

Its goal is to improve context better understanding the relationship between the tokens in a sentence.

“A man was [MASK] on a river bank.”

between words to aid document

ChatGPT is concerned with **Natural Language Generation (NLG)**

Its goal is to predict the next word in the sentence.

“Write a poem about a man fishing on a river

bank. Upon a [NEXT TOKEN]”

Neural Language Models

BIDIRECTIONAL

BERT is concerned with **Natural Language Understanding (NLU)**

Its goal is to improve context better understanding the relationship between the tokens in a sentence.

← "A man was [MASK] on a river bank." →

between words to aid document

UNIDIRECTIONAL

ChatGPT is concerned with **Natural Language Generation (NLG)**

Its goal is to predict the next word in the sentence.

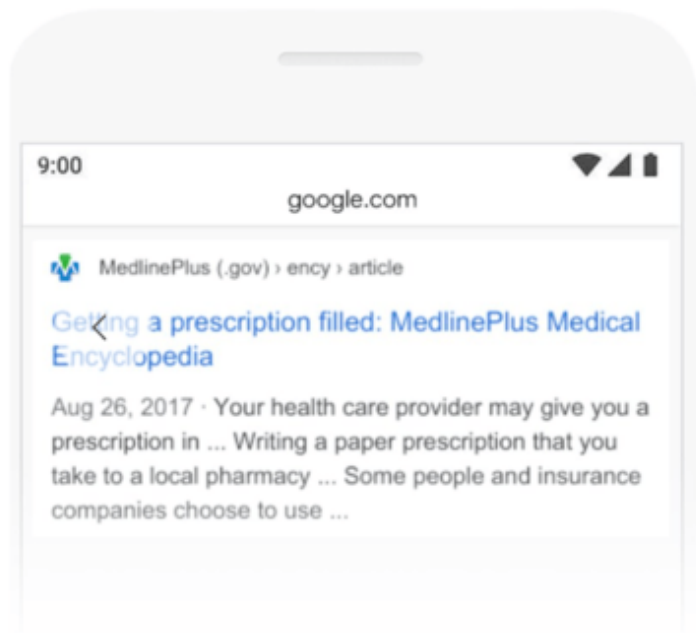
← "Write a poem about a man fishing on a river
bank. Upon a [NEXT TOKEN]"

Neural Language Models

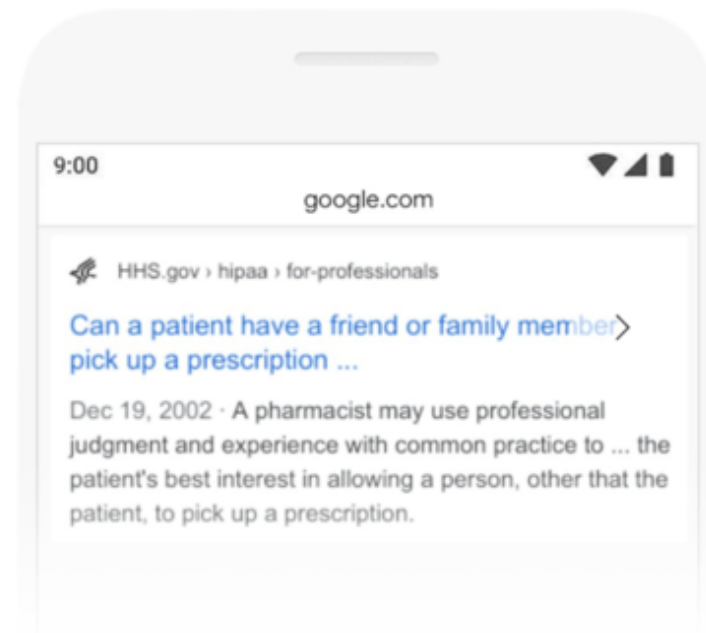


Can you get medicine for someone pharmacy

BEFORE



AFTER



<https://www.webfx.com/blog/internet/google-bert/>

What about ChatGPT?

- Probably the most famous LLM is now ChatGPT, released in 2022 based on GPT-3.5 by OpenAI.
 - Now ChatGPT-Plus is based on GPT-4, released in 2023.
 - Microsoft have integrated ChatGPT into Bing search engine (announced February 2023).
 - It will be interesting to see how users respond in the longer term, compared to traditional web search.
- LLMs are made by learning patterns and relationships between words/phrases from **enormous** quantities of text.
 - It is estimated to cost 27.5m RMB (4m USD) to train GPT-3, for example¹.
- How does ChatGPT work?²

¹ <https://www.cnbc.com/2023/03/13/chatgpt-and-generative-ai-are-booming-but-at-a-very-expensive-price.html>

² <https://www.zdnet.com/article/how-does-chatgpt-work/>

What about ChatGPT?

From a **sustainability** perspective Chat GPT is estimated to:

- generates around **16k tonnes of CO2 annually** (equivalent to around 400M cups of tea; 1 cup = 40g of CO2)
- uses around **1.7b gallons of water per year** (would fill more than 2,500 Olympic size swimming pools), and
- uses around **25MW of energy each time it is trained** (that is enough to power around 20k homes)

Each ChatGPT query uses 4.32g of CO2...

- 16 queries is the same as boiling a kettle
- 139 queries is like doing the laundry

■ It is estimated to cost 27.5m RMB (4m USD) to train GPT-3, for example¹.

■ How does ChatGPT work?²

¹ <https://www.cnbc.com/2023/03/13/chatgpt-and-generative-ai-are-booming-but-at-a-very-expensive-price.html>

² <https://www.zdnet.com/article/how-does-chatgpt-work/>

What about ChatGPT?

- In the context of IR, LLMs such as ChatGPT (and others: e.g. Bard by Google and LLaMa by Meta) have several serious problems.
 - “In simple terms, these models figure out what word is likely to come next, given a set of words or a phrase. In doing so, they are able to generate sentences, paragraphs and even pages that correspond to a query from a user.”¹
 - It is **not designed to provide knowledge** and so it is often **confidently wrong**. The output can be plausible and believable, but might still be incorrect.
 - It cannot **cite its sources**, so you cannot find where it got its information from. If you ask it to provide citations, it will provide something that seems to be correct, but these will be artificial, because it is trained to know **what a citation looks like**, and not to understand any knowledge about a citation.

¹ <https://theconversation.com/ai-information-retrieval-a-search-engine-researcher-explains-the-promise-and-peril-of-letting-chatgpt-and-its-cousins-search-the-web-for-you-200875>

Conclusion

- The Classical IR Pipeline has several limitations with regard to modern IR.
- Several techniques are available that can be used to help improve the quality of retrieval, but **response time is key** given the huge amounts of data involved.
- In the next few lectures, we will examine a few of these in more detail.

IR Pipelines and Modern IR

COMP3009J: Information Retrieval

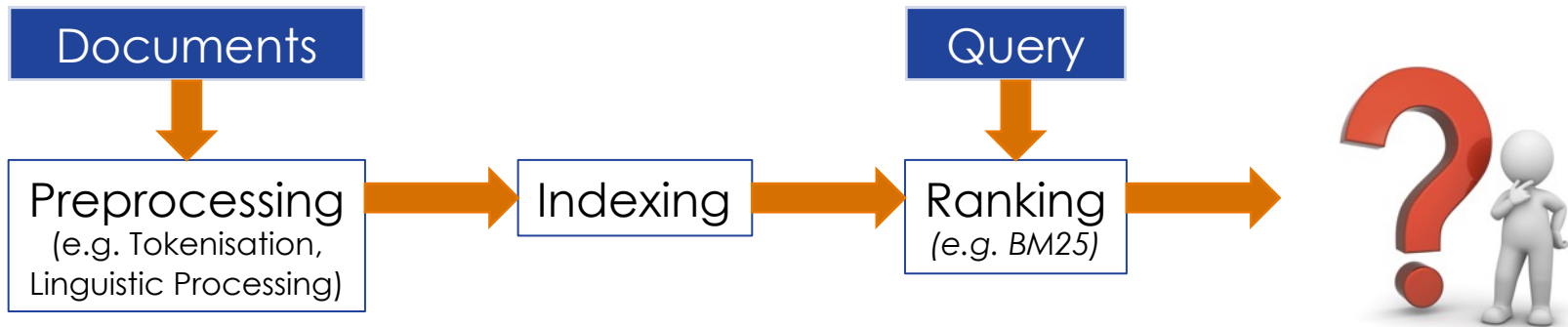
Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Recap: where are we now?

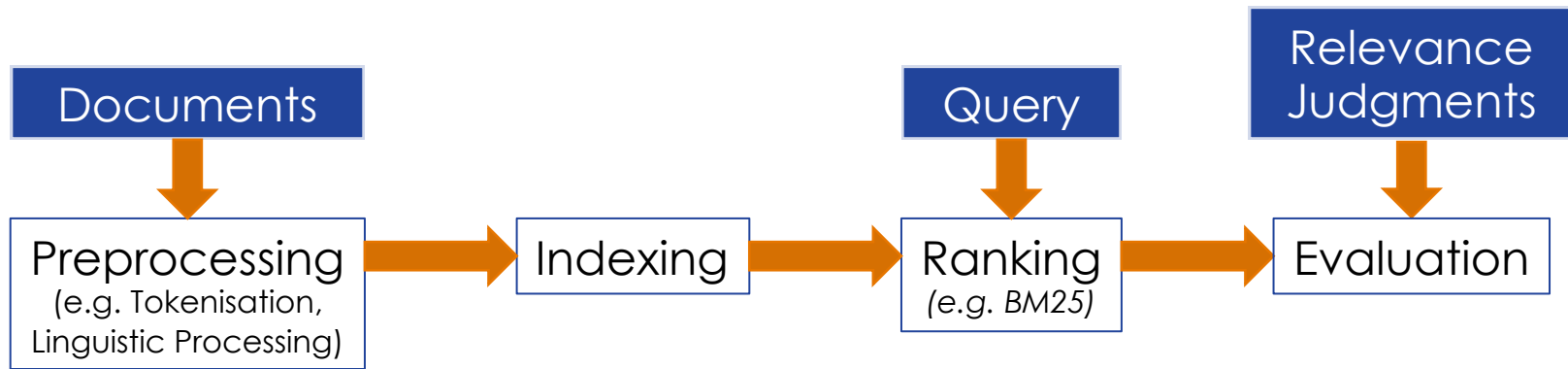
- So far we have looked at **classic** Information Retrieval, much of which is still relevant today. We know...
 - What IR is.
 - How an index can be built.
 - How several models can perform retrieval.
 - How evaluation can be done.
- Thus all the basic components are in place.
- Next we will look at how this process can be improved.

Classical IR Pipeline



1. Documents are **preprocessed** to prepare them for use in the IR system.
2. The **indexer** creates a suitable data structure so that retrieval can happen.
3. A **ranking** method matches queries against the index to produce a **ranked list of results** for the user (or to an evaluation step).

Classical IR Pipeline (Experimental)



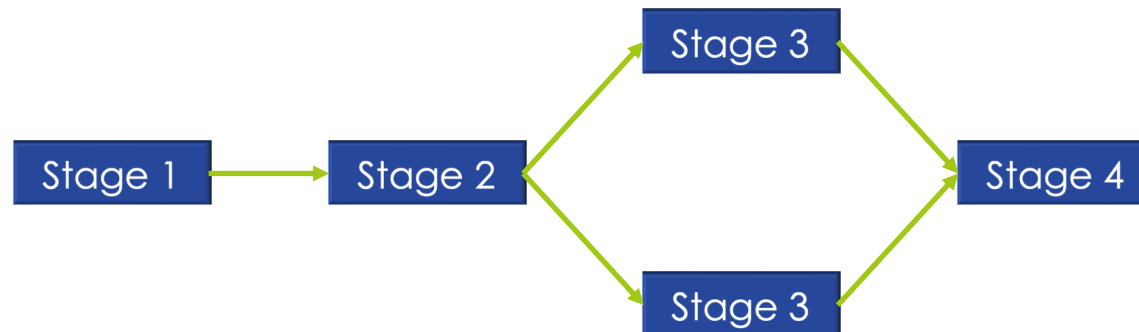
- Alternative pipeline with Evaluation component.
 - Used in experimentation – to evaluate the performance of the system that has been designed.
 - (this is evaluation in terms of the system's **effectiveness**)

Aside: Pipeline Design Pattern

- We call this design pattern a **pipeline** because the output of one process becomes the input of the next stage in the process.
- This is similar to a **linear pipeline**, where each stage happens sequentially.



- An alternative is a **non-linear** pipeline, where some stages can happen in parallel.



Issues with Classical Pipeline (so far)

- **Vocabulary mismatch** is a problem. The words chosen by the user may not match the words used in a relevant document.
- **polysemy**: the same word can have different meanings (e.g. “bank”).
- **synonymy**: two different words can have the same meaning (e.g. plane/airplane/aeroplane/aircraft).
- How can we be confident that our choice of **retrieval model** is the best for each query?
 - Could we use a combination of ranking models instead of just one?

Issues with Classical Pipeline (so far)

- Only the **document contents** are taken into account.
 - Particularly on the web, there is a lot more information available.
 - The **link structure** of the web:
 - Where does this page link to?
 - What other pages link to this page?
 - What text do other pages use in their links to this page (in their **anchor text**)?
 - The **URL** may contain useful information?
 - For a large search engines, they have **query logs** to record previous user's behaviour when they give the same query.

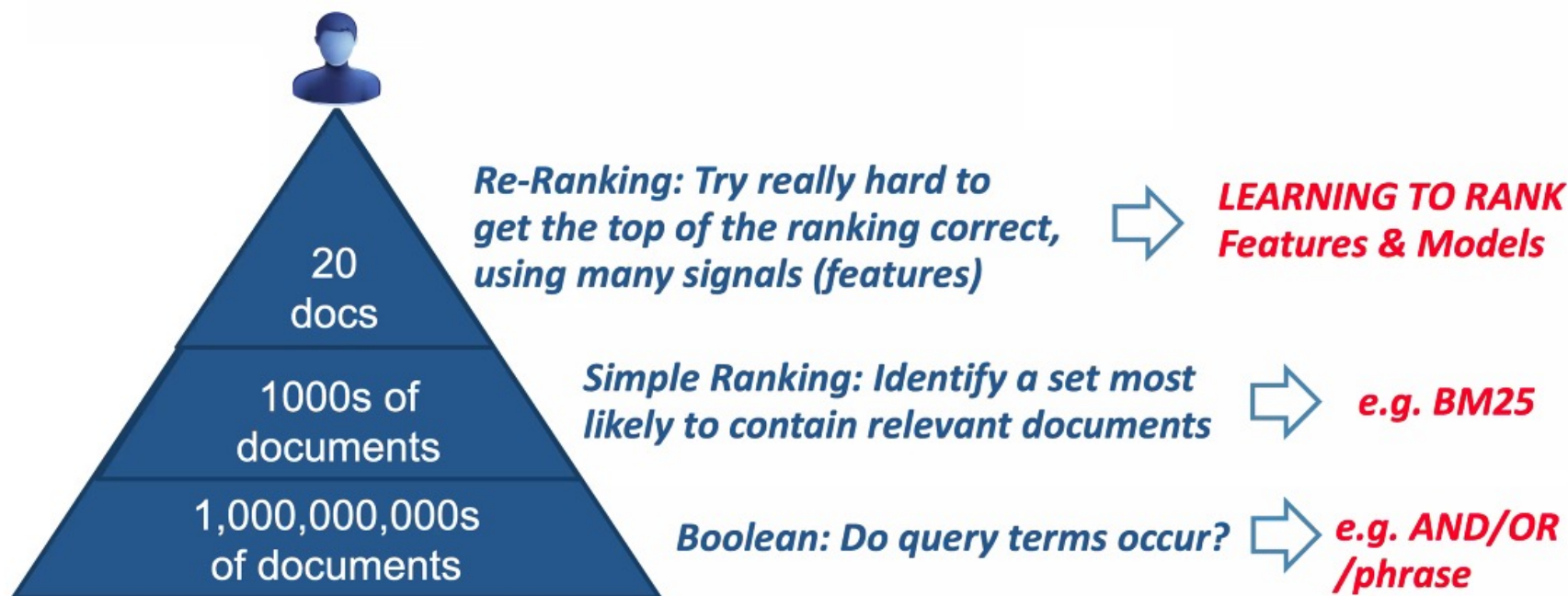
Issues with Classical Pipeline

- ❑ Doesn't make use of recent advances in **machine learning** (especially **deep learning**) and **Natural Language Processing** (NLP) and **Natural Language Understanding** (NLU).
- ❑ Machine Learning has transformed most data-processing and data-analysis tasks.
- ❑ Large Language Models (LLMs) such as **BERT** (Google, 2018), **T5** (Google, 2020), **GPT-4** (OpenAI, 2023) have made huge progress in a variety of NLP, NLU and Machine Translation tasks.
- ❑ **BUT...** Languages models are *sloooooooooow*...

Speed is of the essence...

■ **Speed is essential.** According to Microsoft:

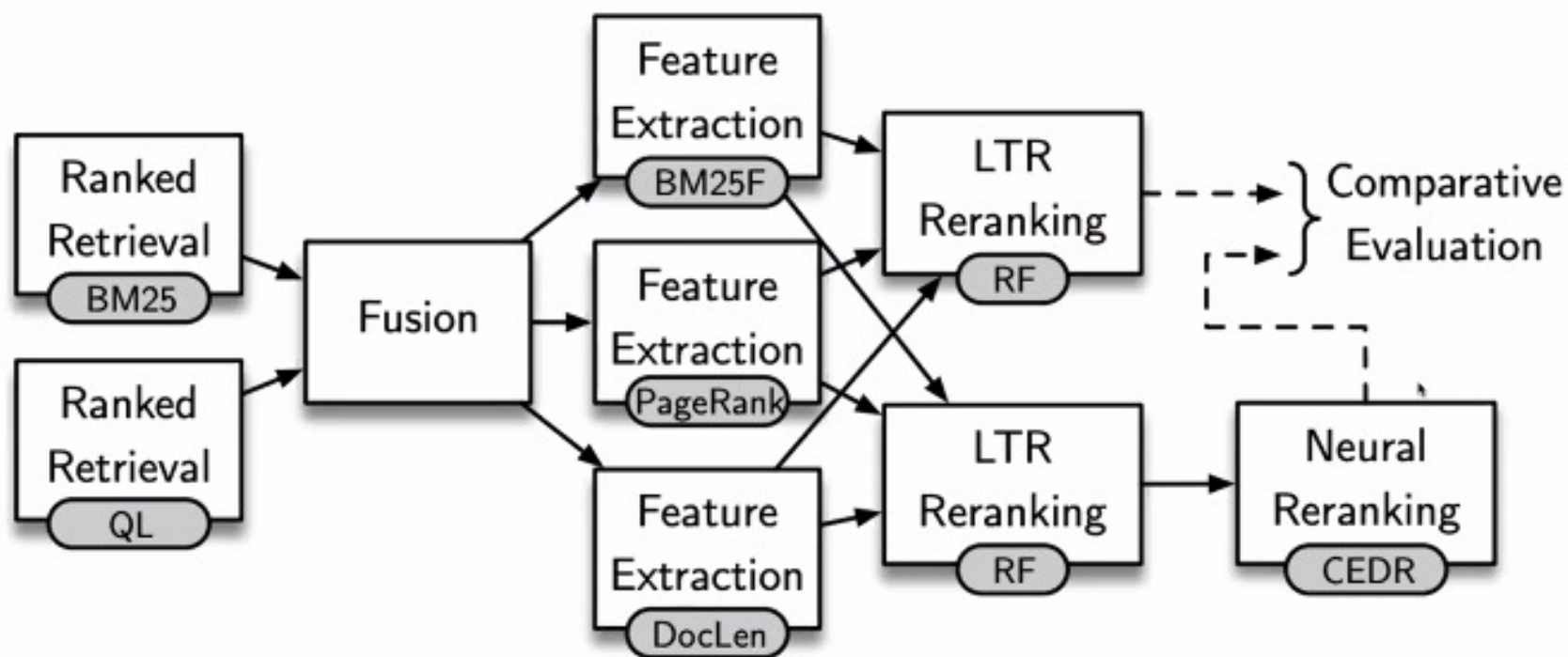
■ “Even a **100ms** latency has been shown to invoke negative user reactions”¹



[1] J Pederson. Query understanding at Bing. SIGIR 2010 Industry Day.

¹ Rosset, C., Jose, D., Ghosh, G., Mitra, B., & Tiwary, S. (2018). Optimizing Query Evaluations Using Reinforcement Learning for Web Search. SIGIR 2018. <https://doi.org/10.1145/3209978.3210127>

A More Complex IR Pipeline



Source: MacAvaney, S., McDonald, C., and Tonolotto, N., IR From Bag-of-words to BERT and Beyond through Practical Experiments, ECIR 2021.

Pseudo-Relevance Feedback

- **Relevance Feedback** is when users are shown a set of documents in response to a query, and tell the system which ones are relevant.
 - The system can then use this information to re-run the search (with different term weights) and hopefully find more relevant documents that are similar to the ones the user has said are relevant.
 - **However**, users don't like giving relevance feedback!
- **Pseudo-Relevance Feedback** simulates relevance feedback by *assuming* that the top k ranked documents are relevant, and then re-running the search in the same way.
 - **Why does this work?** “[T]erms that occur consistently in relevant documents also tend to appear consistently in top ranked documents.”¹
 - **But** it's not perfect: danger of **query drift** towards the topic of the top-ranked documents.

¹ Zhao, L., and Callan, J., Term Necessity Prediction, CIKM 2010. <https://doi.org/10.1145/1871437.1871474>

Query Expansion

- Related terms are **added to the query** to increase the chance of matching relevant documents.
- Many sources of related terms:
 - A **manually-created thesaurus**, such as WordNet¹.
 - An **automatically-created thesaurus** from some external corpus like a web crawl, or Wikipedia².
 - Word embeddings**, where words are represented by vectors (e.g. word2vec, GloVe, ELMo and BERT-based embeddings).

The screenshot shows a Google search for "aircraft fuel". The search bar at the top contains the text "aircraft fuel". Below the search bar, there are tabs for "All", "Images", "News", "Shopping", "Videos", and "More". The search results show "About 685,000,000 results (0.69 seconds)". The first result is from Wikipedia, titled "Aviation fuel - Wikipedia". The snippet describes aviation fuel as a clear to straw-colored fuel based on kerosene or a naphtha-kerosene blend. Below this is another Wikipedia result titled "Jet fuel - Wikipedia", which describes jet fuel as a type of aviation fuel designed for use in aircraft powered by gas-turbine engines. It lists physical properties: Boiling point (176 °C), Flash point (38 °C), Melting point (-47 °C), and Density (775.0-840.0 g/L). The third result is from Shell Global, titled "Aviation Fuel & Gasoline | Aeroplane Fuel | Shell Global", and mentions that today's kerosene jet fuels have been developed from illuminating kerosene used in early gas turbine engines.

¹ <https://wordnet.princeton.edu/>

² For example, through DBPedia (<https://wiki.dbpedia.org/>)

Query Expansion

- Many sources of related terms:
 - **Query Log** mining, where a search engine looks at the behaviour of previous users who have given the same query.
 - **Target-corpus based techniques**, based on the document collection being searched. Two categories:
 - *Distribution-based*: compare the distribution (frequency) of terms in the (pseudo-) relevant documents compared to the whole corpus.
 - *Association-based*: select terms based on their association (co-occurrence) with the query terms.

aircraft fuel

About 685,000,000 results (0.69 seconds)

en.wikipedia.org › wiki › Aviation_fuel ▾

Aviation fuel - Wikipedia

Jet fuel is a clear to straw-colored fuel, based on either an unleaded **kerosene** (**Jet A-1**), or a naphtha-kerosene blend (**Jet B**). Similar to diesel fuel, it can be used in either compression ignition engines or turbine engines.

Jet fuel · Avgas · Category:Aviation fuels · Aviation biofuel

en.wikipedia.org › wiki › Jet_fuel ▾

Jet fuel - Wikipedia

Jet fuel or aviation turbine fuel (ATF, also abbreviated avtur) is a type of **aviation fuel** designed for use in aircraft powered by gas-turbine engines. It is colorless to ...

Boiling point: 176 °C (349 °F; 449 K)	Flash point: 38 °C (100 °F; 311 K)
Melting point: -47 °C (-53 °F; 226 K)	Density: 775.0–840.0 g/L

Water in jet fuel · Military jet fuels · Synthetic jet fuel · USAF synthetic fuel trials

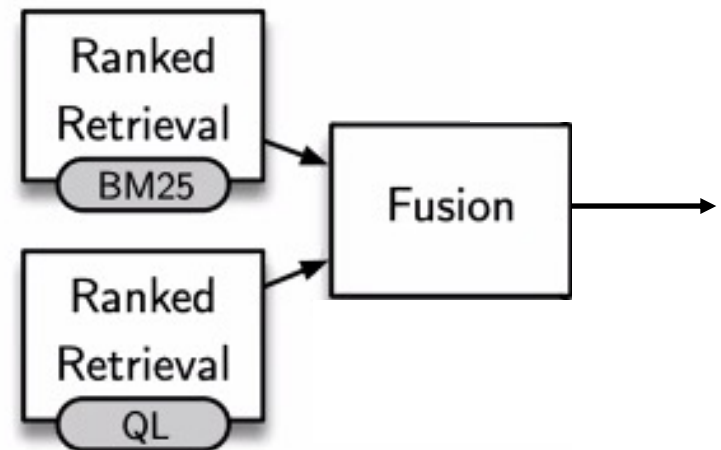
www.shell.com › Business customers › Aviation ▾

Aviation Fuel & Gasoline | Aeroplane Fuel | Shell Global

Today's kerosene jet **fuels** have been developed from the illuminating kerosene used in the early gas turbine engines. These engines needed a **fuel** with good ...

Fusion

- **Input:** a set of results (for one query) from several Ranked Retrieval systems.
- **Output:** a single, combined set of results that are (hopefully) better than the input results in isolation.
- Can be based on:
 - the **scores** calculated by the Ranked Retrieval systems;
 - the **rank** of the documents in each results list;
 - the **probability** of a document being relevant, based on past performance.

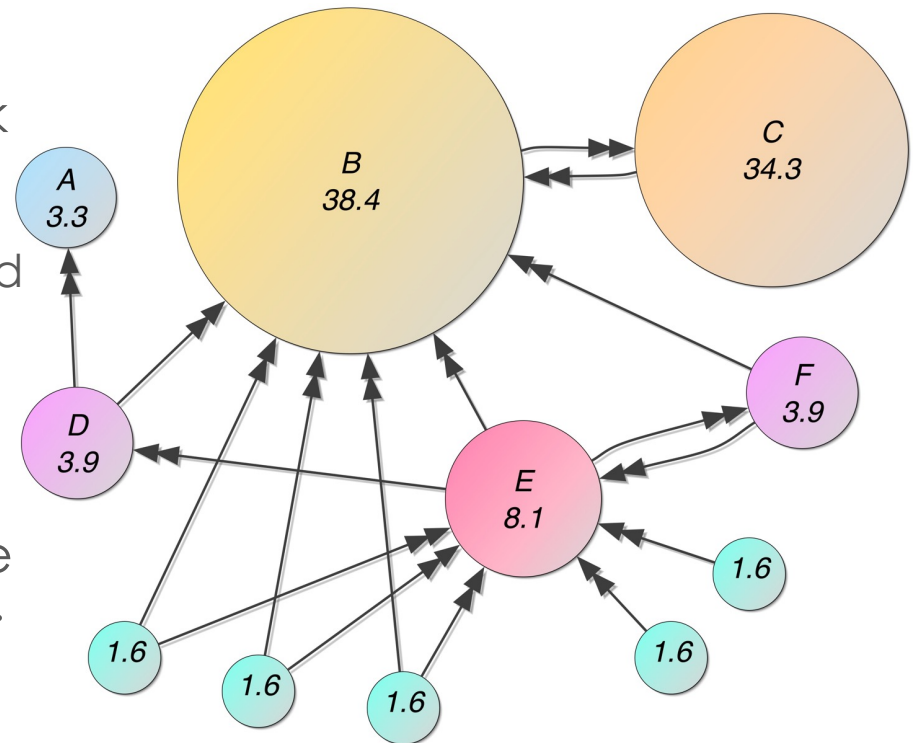


Learning to Rank

- After the initial search has been done by a classic ranked retrieval model (e.g. BM25, or a fused result from several rankers).
- Machine Learning used to re-rank the documents that have been retrieved.
 - **Aim:** improve the result by putting the most-relevant documents at the top of the ranked list of results.
- Takes into account much more information than just the initial ranking score.
 - Each piece of information used is known as a **feature** in Machine Learning.

PageRank: Document Importance

- One feature commonly used in Learning to Rank models is **PageRank**¹.
 - Developed by the founders of Google, and was a key part of Google's success.
- Measures the **importance** of a document based on the **link structure of the web**.
- A document is considered to be **important** if:
 - Many pages link to it; or
 - Other important pages link to it.



¹ Page, L., Brin, S., Motwani, R., and Winograd, T. *The PageRank Citation Ranking: Bringing Order to the Web*. Technical Report. Stanford InfoLab. 2009.

Learning to Rank: Features

- Some examples of features that can be used by Learning to Rank models¹
- **Textual Features:**
 - Term occurrence/non-occurrence
 - Term frequency
 - Inverse Document Frequency
 - Document Length
 - Term Proximity
- **Non-Textual Features:**
 - PageRank
 - URL Depth
 - Document Quality
 - Readability
 - Sentiment
 - Query Clarity

¹ Metzler, D. ,and Croft, W. B., Linear Feature-Based Models for Information Retrieval, 2007.
<https://doi.org/10.1007/s10791-006-9019-z>

Neural Language Models

- Bidirectional Encoder Representations from Transformers (BERT) is a deep-learning based Large Language Model (LLM) for Natural Language Processing.
 - Developed by Google and released in 2018¹.
 - In simple terms, it learns the relationships between words to aid document understanding
 - It has revolutionised NLP, and has had great success on many NLP tasks.
 - As at 2020, it is used for almost all English-language queries in Google's search engine².



¹ <https://github.com/google-research/bert>

² <https://searchengineland.com/google-bert-used-on-almost-every-english-query-342193>

What about ChatGPT?

- Probably the most famous LLM is now ChatGPT, released in 2022 based on GPT-3.5 by OpenAI.
 - Now ChatGPT-Plus is based on GPT-4, released in 2023.
 - Microsoft have integrated ChatGPT into Bing search engine (announced February 2023).
 - It will be interesting to see how users respond in the longer term, compared to traditional web search.
- LLMs are made by learning patterns and relationships between words/phrases from **enormous** quantities of text.
 - It is estimated to cost 27.5m RMB (4m USD) to train GPT-3, for example¹.
- How does ChatGPT it work?²

¹ <https://www.cnbc.com/2023/03/13/chatgpt-and-generative-ai-are-booming-but-at-a-very-expensive-price.html>

² <https://www.zdnet.com/article/how-does-chatgpt-work/>

What about ChatGPT?

- In the context of IR, LLMs such as ChatGPT (and others: e.g. Bard by Google and LLaMa by Meta) have several serious problems.
 - “In simple terms, these models figure out what word is likely to come next, given a set of words or a phrase. In doing so, they are able to generate sentences, paragraphs and even pages that correspond to a query from a user.”¹
 - It is **not designed to provide knowledge** and so it is often **confidently wrong**. The output can be plausible and believable, but might still be incorrect.
 - It cannot **cite its sources**, so you cannot find where it got its information from. If you ask it to provide citations, it will provide something that seems to be correct, but these will be artificial, because it is trained to know **what a citation looks like**, and not to understand any knowledge about a citation.

¹ <https://theconversation.com/ai-information-retrieval-a-search-engine-researcher-explains-the-promise-and-peril-of-letting-chatgpt-and-its-cousins-search-the-web-for-you-200875>

Conclusion

- The Classical IR Pipeline has several limitations with regard to modern IR.
- Several techniques are available that can be used to help improve the quality of retrieval, but **response time is key** given the huge amounts of data involved.
- In the next few lectures, we will examine a few of these in more detail.

Fusion: Rank-Based

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Rank-Based Fusion: Interleaving

- **Interleaving** is perhaps the simplest fusion algorithm of all*
- Here, we take one document from the top of each input set in a "**round robin**" fashion and add it to the fused result set.
 - The document chosen is the highest-ranked document that is not yet included in the fused result set.
- The effectiveness of this technique is, however, poor.
- There is an assumption that every result set is of equal quality, which can have the result that the better result sets are diluted by being merged with non-relevant documents from poorer systems.

* Voorhees, E. M., Gupta, N. K., & Johnson-Laird, B. (1994). The Collection Fusion Problem. In *Proceedings of the Third Text REtrieval Conference (TREC-3)* (pp. 95–104)

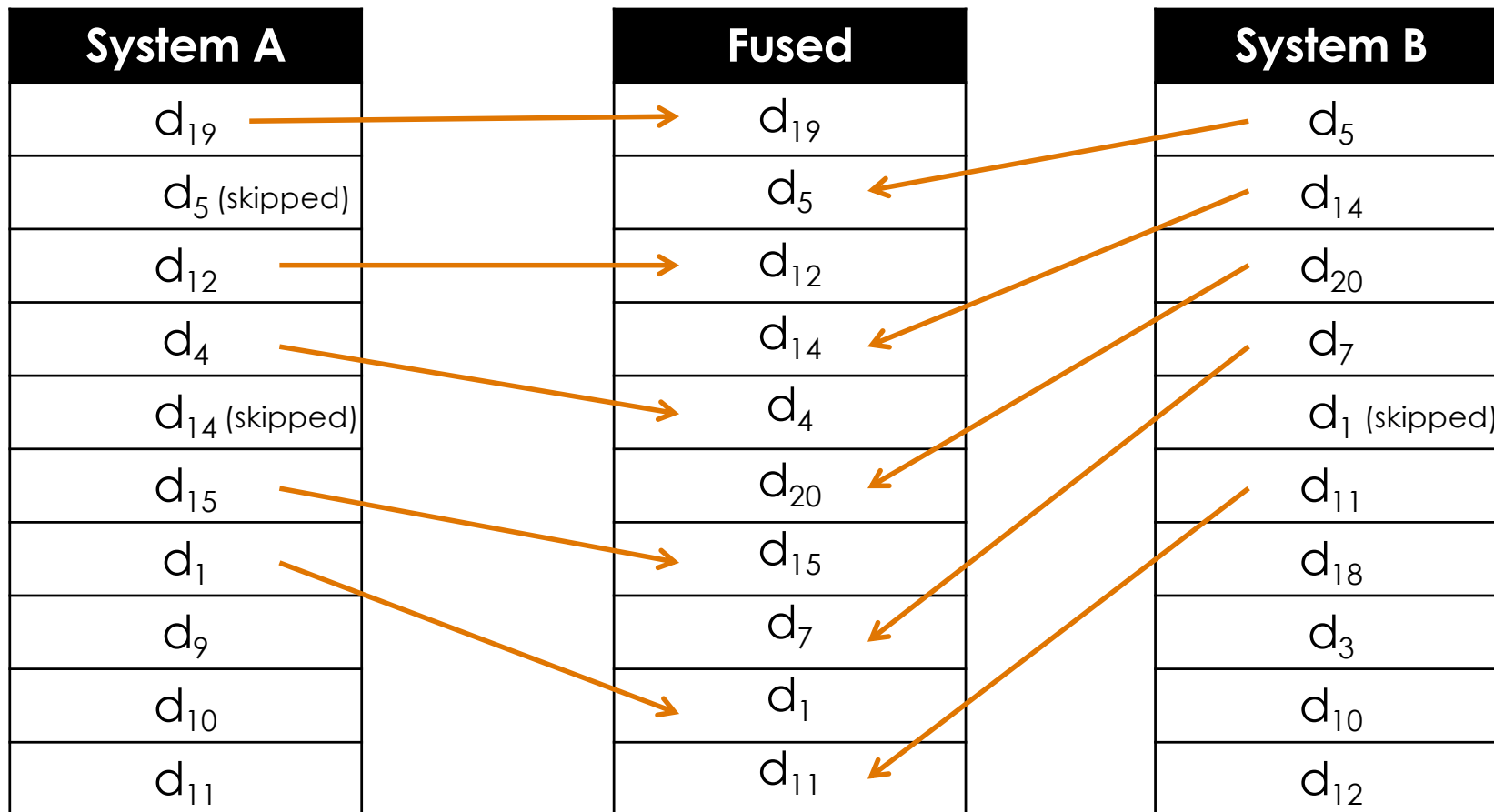
Rank-Based: Interleaving (Example)

System A
d_{19}
d_5
d_{12}
d_4
d_{14}
d_{15}
d_1
d_9
d_{10}
d_{11}

Fused

System B
d_5
d_{14}
d_{20}
d_7
d_1
d_{11}
d_{18}
d_3
d_{10}
d_{12}

Rank-Based: Interleaving (Example)



Rank-Based: Borda-Fuse

- **Borda-Fuse** is based on an election system for when a few voters(input systems) vote for many candidates (documents).
 - Each voter ranks a set of c candidates in order of preference.
 - For each voter, the top-ranked candidate is given c points, the second ranked is given $c-1$ points, etc.
 - If a candidate is not ranked by a voter, the voter's remaining points are divided evenly among unranked candidates.

* Aslam, J. A., & Montague, M. (2001). Models for metasearch. In *SIGIR '01: Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 276–284). New York, NY, USA.

Rank-Based: Borda-Fuse

- **Borda-Fuse** is based on an election system for when a few voters(input systems) vote for many candidates

Inventor: Jean-Charles de Borda

Born 4th May, 1773

French Mathematician, Physicist, Naval Officer

There is a moon crater named after him

Developed "Borda Count" in 1770 to elect members to the French Academy of Science.

Fought in the American Revolution

Helped to define the standard metre

Developed a technique for computing longitude in 1778



* Aslam, J. A., & Montague, M. (2001). Models for metasearch. In *SIGIR '01: Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 276–284). New York, NY, USA.

Rank-Based: Borda-Fuse

System A	Points
d_{19}	14
d_5	13
d_{12}	12
d_4	11
d_{14}	10
d_{15}	9
d_1	8
d_9	7
d_{10}	6
d_{11}	5

System B	Points
d_5	14
d_{14}	13
d_{20}	12
d_7	11
d_1	10
d_{11}	9
d_{18}	8
d_3	7

- 14 unique documents: $c = 14$
- System A gives 2.5 points to any documents it didn't choose:
 $(4+3+2+1)/4$.
- System B gives 3.5 points to documents it didn't choose:
 $(6+5+4+3+2+1)/6$

Rank-Based: Borda-Fuse

System A	Points
d ₁₉	14
d ₅	13
d ₁₂	12
d ₄	11
d ₁₄	10
d ₁₅	9
d ₁	8
d ₉	7
d ₁₀	6
d ₁₁	5

System B	Points
d ₅	14
d ₁₄	13
d ₂₀	12
d ₇	11
d ₁	10
d ₁₁	9
d ₁₈	8
d ₃	7

Fused	Points (A)	Points (B)	Points (Borda)
d ₅	13	14	27
d ₁₄	10	13	23
d ₁	8	10	18
d ₁₉	14	3.5	17.5
d ₁₂	12	3.5	15.5
d ₄	11	3.5	14.5
d ₂₀	2.5	12	14.5
d ₁₁	5	9	14
d ₇	2.5	11	13.5
d ₁₅	9	3.5	12.5
d ₁₈	2.5	8	10.5
d ₉	7	3.5	10.5
d ₁₀	6	3.5	9.5
d ₃	2.5	7	9.5

Comparison

System A	System B	Interleaving	Borda Fuse
d_{19}	d_5	d_{19}	d_5
d_5	d_{14}	d_5	d_{14}
d_{12}	d_{20}	d_{12}	d_1
d_4	d_7	d_{14}	d_{19}
d_{14}	d_1	d_4	d_{12}
d_{15}	d_{11}	d_{20}	d_4
d_1	d_{18}	d_{15}	d_{20}
d_9	d_3	d_7	d_{11}
d_{10}	d_{10}	d_1	d_7
d_{11}	d_{12}	d_{11}	d_{15}

Reciprocal Rank Fusion

- Reciprocal Rank Fusion is a simple rank-based method that has been shown to be effective in practice*.
- Given a set of documents D to be ranked, and a set of results R , the score for each document is calculated as follows:
 - $RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k+r(d)}$
 - where $r(d)$ is the rank of document d in result set r , and $k=60$ (set by experiment)

* Cormack, G. V., Clarke, C. L. A., Büttcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Models. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 758-759)

Useful Maths

- ▣ RRF uses $k=60$.
- ▣ If a document, d , is not ranked, it does not contribute to the RRF Score
 - ▣ i. e. the score for the term related to document d in the summation is 0.

Rank	Calculation	RRF Score
1	$1/61$	0.01639
2	$1/62$	0.01612
3	$1/63$	0.01587
4	$1/64$	0.01562
5	$1/65$	0.01538
6	$1/66$	0.01515
7	$1/67$	0.01492
8	$1/68$	0.01470
9	$1/69$	0.01449
10	$1/70$	0.01428
Not Ranked	0	0.0

Rank-Based: RRF

System A	System B
d ₁₉	d ₅
d ₅	d ₁₄
d ₁₂	d ₂₀
d ₄	d ₇
d ₁₄	d₁
d ₁₅	d ₁₁
d₁	d ₁₈
d ₉	d ₃
d ₁₀	d ₁₀
d ₁₁	d ₁₂

$$RRFscore(d_1) = \sum_{r \in R} \frac{1}{k+r(d)} = \frac{1}{67} + \frac{1}{65} = 0.0303$$

Rank-Based: RRF

System A	System B
d ₁₉	d ₅
d ₅	d ₁₄
d ₁₂	d ₂₀
d ₄	d ₇
d ₁₄	d ₁
d ₁₅	d ₁₁
d ₁	d ₁₈
d ₉	d₃
d ₁₀	d ₁₀
d ₁₁	d ₁₂

$$RRFscore(d_1) = \sum_{r \in R} \frac{1}{k+r(d)} = \frac{1}{67} + \frac{1}{65} = 0.0303$$

$$RRFscore(d_3) = \sum_{r \in R} \frac{1}{k+r(d)} = 0 + \frac{1}{68} = 0.0147$$

Rank-Based: RRF

System A	System B
d ₁₉	d ₅
d ₅	d ₁₄
d ₁₂	d ₂₀
d ₄	d ₇
d ₁₄	d ₁
d ₁₅	d ₁₁
d ₁	d ₁₈
d ₉	d ₃
d ₁₀	d ₁₀
d ₁₁	d ₁₂

Fused	RRF(A)	RRF(B)	RRF
d ₁	0.01492	0.01538	0.0303
d ₃		0.01470	0.0147
d ₄	0.01562		0.0156
d ₅	0.01612	0.01639	0.0325
d ₇		0.01562	0.0156
d ₉	0.01470		0.0147
d ₁₀	0.01449	0.01449	0.0290
d ₁₁	0.01428	0.01515	0.0294
d ₁₂	0.01587	0.01428	0.0302
d ₁₄	0.01538	0.01612	0.0315
d ₁₅	0.01515		0.0152
d ₁₈		0.01492	0.0149
d ₁₉	0.01639		0.0164
d ₂₀		0.01587	0.0159

Rank-Based: RRF

System A	System B
d ₁₉	d ₅
d ₅	d ₁₄
d ₁₂	d ₂₀
d ₄	d ₇
d ₁₄	d ₁
d ₁₅	d ₁₁
d ₁	d ₁₈
d ₉	d ₃
d ₁₀	d ₁₀
d ₁₁	d ₁₂

Fused	RRF(A)	RRF(B)	RRF
d ₅	0.01612	0.01639	0.0325
d ₁₄	0.01538	0.01612	0.0315
d ₁	0.01492	0.01538	0.0303
d ₁₂	0.01587	0.01428	0.0302
d ₁₁	0.01428	0.01515	0.0294
d ₁₀	0.01449	0.01449	0.0290
d ₁₉	0.01639		0.0164
d ₂₀		0.01587	0.0159
d ₄	0.01562		0.0156
d ₇		0.01562	0.0156
d ₁₅	0.01515		0.0152
d ₁₈		0.01492	0.0149
d ₉	0.01470		0.0147
d ₃		0.01470	0.0147

Comparison

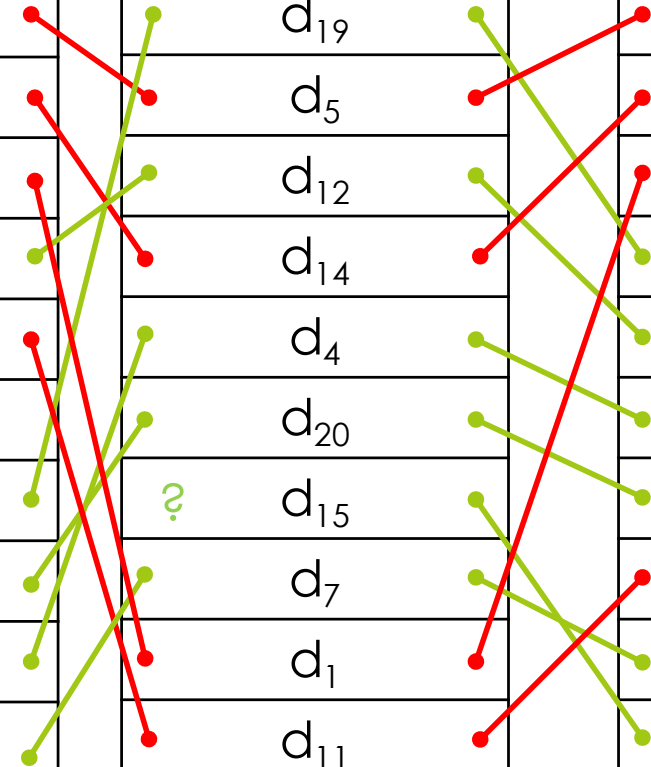
System A
d_{19}
d_5
d_{12}
d_4
d_{14}
d_{15}
d_1
d_9
d_{10}
d_{11}

System B
d_5
d_{14}
d_{20}
d_7
d_1
d_{11}
d_{18}
d_3
d_{10}
d_{12}

RRF
d_5
d_{14}
d_1
d_{12}
d_{11}
d_{10}
d_{19}
d_{20}
d_4
d_7

Interleaving
d_{19}
d_5
d_{12}
d_{14}
d_4
d_{20}
d_{15}
d_7
d_1
d_{11}

BF
d_5
d_{14}
d_1
d_{19}
d_{12}
d_4
d_{20}
d_{11}
d_7
d_{15}



Other Rank-Based Techniques

- A variation on interleaving is to use **historical data** to estimate which input system(s) tends to perform better.
- A weighted version of interleaving is then used so that **more documents are taken from the better systems.** *
- Another election-based approach from the same authors as Borda-Fuse is the *Condorcet-Fuse*** algorithm. A weighted version of Borda-Fuse is also proposed, where the points from each input system are multiplied by some weight.

* Voorhees et al. (1994)

** Montague, M., & Aslam, J. A. (2002). Condorcet fusion for improved retrieval. In *CIKM '02: Proceedings of the eleventh international conference on Information and knowledge management* (pp. 538–548). New York, NY, USA.

Class Question 1

System A
d_{10}
d_{18}
d_4
d_6
d_5
d_{17}
d_{11}
d_{14}

System B
d_{18}
d_6
d_1
d_2
d_{17}

System C
d_6
d_4
d_3
d_{18}
d_5
d_{10}
d_{15}
d_{19}

■ What is the fused ranking using:

1. Interleaving?
2. Borda Fuse?
3. Reciprocal Rank Fusion?

Fusion: Introduction

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Introduction

- You have already seen that there are many algorithms that can be used for Information Retrieval (IR):
 - Boolean Model
 - Vector Space Model
 - BM25
 - ... many more (Probabilistic Model, Query Likelihood, etc.)
- If there was one algorithm that worked better than all the others all the time, we could just use that.
- There isn't!

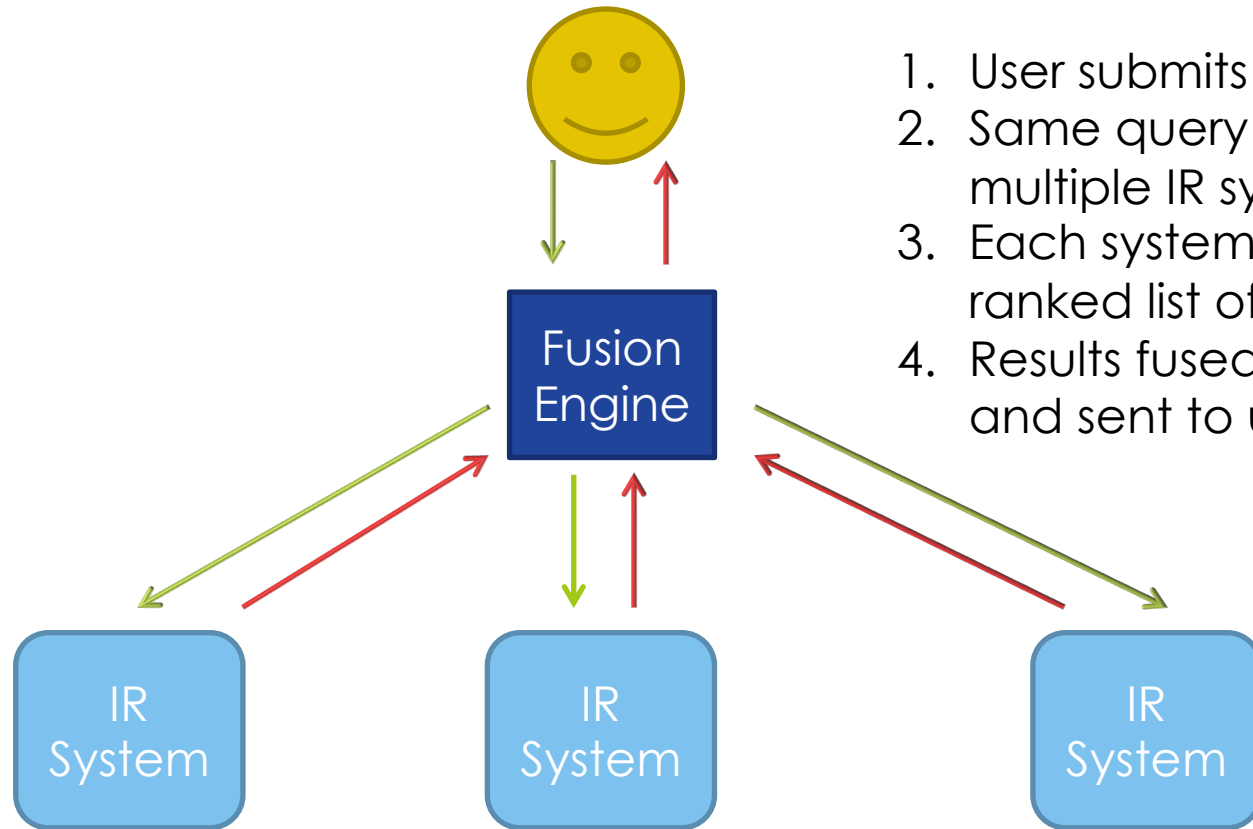
Introduction

- From the 1990s, an area of research called ***data fusion***, ***collection fusion***, or ***results aggregation*** became increasingly popular.
- **Basic definition:**
 - **Combining** the outputs of **multiple** different Information Retrieval systems/algorithms into a single ranked result set that can be shown to a user in response to a **query**.
- It is hoped that this combined result set will be of **better quality** than the individual input result sets.
 - Higher precision, higher recall (and other metrics).

Better Quality?

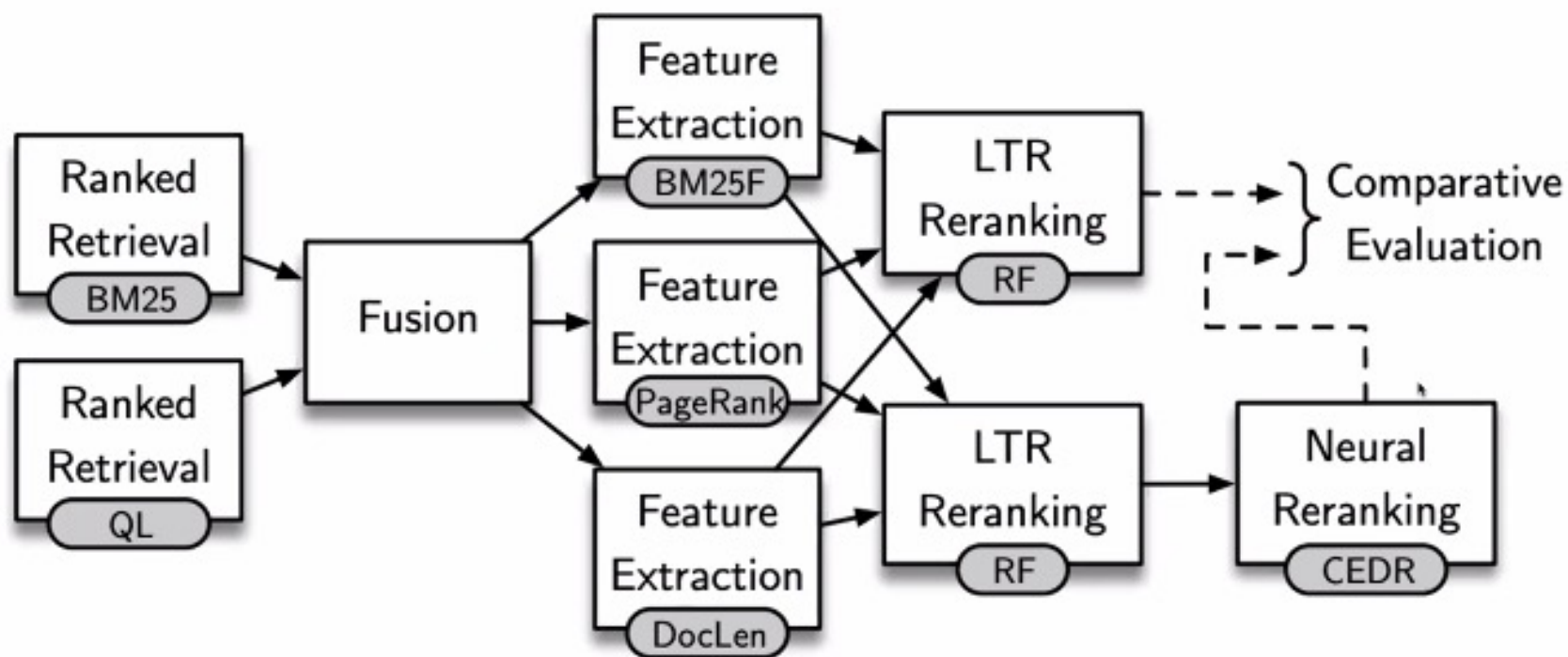
- But what do we mean by *better quality*?
 - Higher **Recall** – almost inevitable if we bring in an additional set of results (more of the available relevant documents are retrieved)
 - Higher **Precision** – more difficult. Need to make sure we introduce relevant documents in place of non-relevant documents.
 - For metrics (such as MAP) that reward high positions for relevant documents, we should ensure that our ranking puts more relevant documents first.

Anatomy of a Fusion System



1. User submits query
2. Same query sent to multiple IR systems
3. Each system returns a ranked list of results
4. Results fused/merged and sent to user.

A More Complex IR Pipeline



Source: MacAvaney, S., McDonald, C., and Tonolotto, N., IR From Bag-of-words to BERT and Beyond through Practical Experiments, ECIR 2021.

Anatomy of a Fusion System

- The final list returned to the user is typically ranked by calculating a **score** for **each document**: the document with the highest score goes at the beginning.
 - Document contents are not considered.
- Generally, each underlying IR system will contribute something towards the score of each document it returns.
 - Might depend on the **position/rank** of the document in the result, the **score** explicitly given by the search engine, **quality** of the search engine itself.
- Main difference between algorithms: what information they take into account when allocating scores.

Applications of Fusion

- **Metasearch:** This involves fusion of result sets returned by **autonomous, complete search engines** (e.g. Google, Bing).
- **Distributed Information Retrieval:** Numerous IR systems are **designed to co-operate** with one another, each working on a subset of the document collection.
- **Internal Metasearch:** Numerous algorithms perform searches on the **same document collection** (data fusion).

Corpus Overlap

- Before developing a fusion algorithm, it is important to consider how much the document collections (corpora) used by the systems we wish to use **overlap**.
- There are three different **levels of overlap** that can occur between document corpora.
- The level of overlap will have a significant effect on how we treat the result sets when fusing.

Corpus Overlap: Disjoint Databases

- Here, the input systems search separate, disjoint document collections that have **no documents in common**.
- A document **cannot be returned by more than one input system**, since it does not appear in more than one index.
- Distributed IR is typically implemented using disjoint databases.
- Fusion of disjoint corpora is frequently known as *Collection Fusion*.

Corpus Overlap: Identical Databases

- The input systems each apply their own IR algorithm to the **same set of documents**.
- Documents will frequently appear in multiple result sets.
- Appearing in multiple result sets is frequently interpreted as further **evidence of relevance** (as multiple systems agree that it is relevant).
- This has become known as **Data Fusion** and is our **main focus**.
- Internal Metasearch is generally a Data Fusion task.

Corpus Overlap: Overlapping Databases

- The document collections being used by the various input systems have some level of overlap, but are **not identical**.
- Documents may appear in multiple result sets.
- However, it is **difficult to draw reliable conclusions** about these documents that appear in multiple result sets.
- External Metasearch generally involves overlapping databases.

Corpus Overlap: Overlapping Databases

- A common feature of fusion algorithms is to give a **higher score** to documents appearing in **multiple result sets**:
 - Appears in every result set => every system considers it relevant.
 - Appears in no result sets => no system considers it relevant.
 - Appears in one result set but not another =>
 - One system does not consider it to be relevant **OR**
 - One system is not aware of the document.
- Difficult to decide how to treat the last situation, as it's often impossible to tell which possibility has occurred.

Three Fusion “Effects”

The Skimming Effect

- There are three "**effects**" that a fusion algorithm may try to leverage*
- In general, the most relevant documents in a result set appear at or near the top, when an effective IR algorithm is being used.
- The Skimming Effect argues that "**skimming**" the **top documents** from each result set and using these for fusion should give better performance.
- This principle is used for **all popular fusion algorithms**.

* Vogt, C. C., & Cottrell, G. W. (1999). Fusion Via a Linear Combination of Scores. *Information Retrieval*, 1(3), 151–173

The Chorus Effect

- If **multiple input systems agree** that a document is relevant, the Chorus Effect argues that this evidence of relevance should be taken into account and that document should be **highly ranked** in the fused result set.
- Whether this effect is applicable depends on the level of overlap between the corpora used by the input systems.
- For Data Fusion, the Chorus Effect tends to be an important consideration.
- For Collection Fusion, it is not a factor at all (documents cannot appear in multiple result sets).
- Difficult to gauge for partially overlapping corpora.

The Chorus Effect

Document d_5 is given a high rank by both systems: should probably be ranked highly in fused result set (Skimming & Chorus Effects)

System A

Rank	Document
1	d_{19}
2	d_5
3	d_{12}
4	d_4
5	d_{14}
6	d_{15}
7	d_1
8	d_9
9	d_{10}
10	d_{11}

System B

Rank	Document
1	d_5
2	d_{14}
3	d_{20}
4	d_7
5	d_1
6	d_{11}
7	d_{18}
8	d_3
9	d_{10}
10	d_{12}

The Chorus Effect

System A

Rank	Document
1	d ₁₉
2	d ₅
3	d ₁₂
4	d ₄
5	d ₁₄
6	d ₁₅
7	d ₁
8	d ₉
9	d ₁₀
10	d ₁₁

Document d₁₉ is given a high rank by one system but is not returned at all by System B: should be ranked below d₅ according to the Chorus Effect.

System B

Rank	Document
1	d ₅
2	d ₁₄
3	d ₂₀
4	d ₇
5	d ₁
6	d ₁₁
7	d ₁₈
8	d ₃
9	d ₁₀
10	d ₁₂

The Dark Horse Effect

- The Dark Horse Effect is where one input system returns results of a much **different quality** than the others.
- This may be as a result of returning either **unusually accurate** or **unusually inaccurate** results.
- This seems to **contradict the Chorus Effect**, as it would favour identifying a "dark horse" and just using its results, rather than fusing it with others.
- The Dark Horse Effect is very difficult to identify, and so is generally not used in fusion algorithms.

Categories of Data Fusion Techniques

- There are three principal categories of Data Fusion techniques that we will look at:
 - **Rank-Based Fusion:** These examine only the **rank/position** that each document occupies in the input result sets. Sometimes necessary if relevance scores are unavailable.
 - **Voting models** generally operate on the rank level also.
 - **Score-Based Fusion:** The **relevance scores** given to a document by an input system can be used as a measurement of how confident it is as to the document's relevance.
 - **Segment-Based Fusion:** Result sets are divided into groups of documents, rather than using individual ranks or scores.

Fusion: Rank-Based

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Rank-Based Fusion: Interleaving

- **Interleaving** is perhaps the simplest fusion algorithm of all*
- Here, we take one document from the top of each input set in a "**round robin**" fashion and add it to the fused result set.
 - The document chosen is the highest-ranked document that is not yet included in the fused result set.
- The effectiveness of this technique is, however, poor.
- There is an assumption that every result set is of equal quality, which can have the result that the better result sets are diluted by being merged with non-relevant documents from poorer systems.

* Voorhees, E. M., Gupta, N. K., & Johnson-Laird, B. (1994). The Collection Fusion Problem. In *Proceedings of the Third Text REtrieval Conference (TREC-3)* (pp. 95–104)

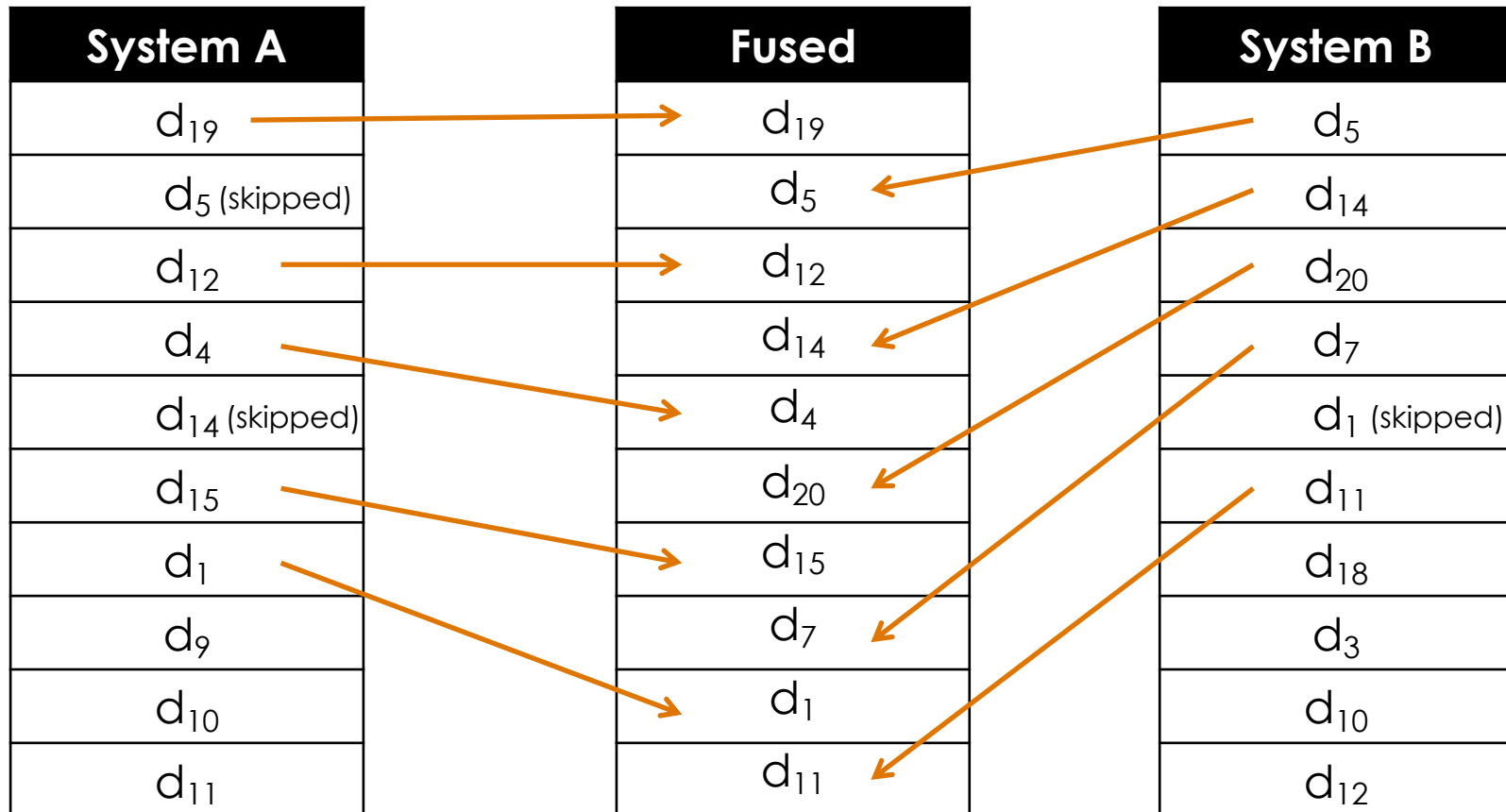
Rank-Based: Interleaving (Example)

System A
d_{19}
d_5
d_{12}
d_4
d_{14}
d_{15}
d_1
d_9
d_{10}
d_{11}

Fused

System B
d_5
d_{14}
d_{20}
d_7
d_1
d_{11}
d_{18}
d_3
d_{10}
d_{12}

Rank-Based: Interleaving (Example)



Rank-Based: Borda-Fuse

- **Borda-Fuse** is based on an election system for when a few voters(input systems) vote for many candidates (documents).
 - Each voter ranks a set of c candidates in order of preference.
 - For each voter, the top-ranked candidate is given c points, the second ranked is given $c-1$ points, etc.
 - If a candidate is not ranked by a voter, the voter's remaining points are divided evenly among unranked candidates.

* Aslam, J. A., & Montague, M. (2001). Models for metasearch. In *SIGIR '01: Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 276–284). New York, NY, USA.

Rank-Based: Borda Fuse

System A	Points
d_{19}	14
d_5	13
d_{12}	12
d_4	11
d_{14}	10
d_{15}	9
d_1	8
d_9	7
d_{10}	6
d_{11}	5

System B	Points
d_5	14
d_{14}	13
d_{20}	12
d_7	11
d_1	10
d_{11}	9
d_{18}	8
d_3	7

- 14 unique documents: $c = 14$
- System A gives 2.5 points to any documents it didn't choose:
 $(4+3+2+1)/4$.
- System B gives 3.5 points to documents it didn't choose:
 $(6+5+4+3+2+1)/6$

Rank-Based: Borda Fuse

System A	Points
d ₁₉	14
d ₅	13
d ₁₂	12
d ₄	11
d ₁₄	10
d ₁₅	9
d ₁	8
d ₉	7
d ₁₀	6
d ₁₁	5

System B	Points
d ₅	14
d ₁₄	13
d ₂₀	12
d ₇	11
d ₁	10
d ₁₁	9
d ₁₈	8
d ₃	7

Fused	Points (A)	Points (B)	Points (Borda)
d ₅	13	14	27
d ₁₄	10	13	23
d ₁	8	10	18
d ₁₉	14	3.5	17.5
d ₁₂	12	3.5	15.5
d ₄	11	3.5	14.5
d ₂₀	2.5	12	14.5
d ₁₁	5	9	14
d ₇	2.5	11	13.5
d ₁₅	9	3.5	12.5
d ₁₈	2.5	8	10.5
d ₉	7	3.5	10.5
d ₁₀	6	3.5	9.5
d ₃	2.5	7	9.5

Reciprocal Rank Fusion

- Reciprocal Rank Fusion is a simple rank-based method that has been shown to be effective in practice*.
- Given a set of documents D to be ranked, and a set of results R , the score for each document is calculated as follows:
 - $RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k+r(d)}$
 - where $r(d)$ is the rank of document d in result set r , and $k=60$ (set by experiment)

* Cormack, G. V., Clarke, C. L. A., Büttcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Models. In *Proceedings of the 32nd international ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 758-759)

Other Rank-Based Techniques

- A variation on interleaving is to use **historical data** to estimate which input system(s) tends to perform better.
- A weighted version of interleaving is then used so that **more documents are taken from the better systems.** *
- Another election-based approach from the same authors as Borda-Fuse is the *Condorcet-Fuse*** algorithm. A weighted version of Borda-Fuse is also proposed, where the points from each input system are multiplied by some weight.

* Voorhees et al. (1994)

** Montague, M., & Aslam, J. A. (2002). Condorcet fusion for improved retrieval. In *CIKM '02: Proceedings of the eleventh international conference on Information and knowledge management* (pp. 538–548). New York, NY, USA.

Fusion: Score-Based

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Score-Based Fusion: CombSUM

- CombSUM is a popular algorithm for performing fusion using the **relevance scores** that each document is assigned by the input IR system. It was proposed by Fox & Shaw in 1994*.
- The final score on which a document is ranked in the fused result set is calculated by **adding the individual scores** it was given in each of the input result sets.
- High scores in the input result set are carried to the fused result set, so the **Skimming Effect** is in use.
- Also, because the scores are added, documents appearing in multiple result sets will also receive a boost, exploiting the **Chorus Effect**.

* Fox, E. A., & Shaw, J. A. (1994). Combination of Multiple Searches. In *Proceedings of the 2nd Text REtrieval Conference (TREC-2)*, National Institute of Standards and Technology Special Publication 500-215 (pp. 243–252)

Score-Based: CombSUM Example

■ Example 1:

- Document #1 is given a score of 0.45 by System A
- Document #1 is given a score of 0.3 by System B
- Document #1 is given a score of 0.35 by System C
- Document #1's CombSUM score is $0.45 + 0.3 + 0.35 = 1.1$

■ Example 2:

- Document #2 is given a score of 0.55 by System A
- Document #2 is not returned by System B
- Document #2 is given a score of 0.65 by System C
- Document #2's CombSUM score is $0.55 + 0 + 0.65 = 1.2$

Score Normalisation

- There is a difficulty with simply adding raw relevance scores.
- Consider the situation where different IR systems may calculate scores in **different ranges**:
 - System A: Assigns scores between 0 and 1000
 - System B: Assigns scores between 0 and 1
- We often don't know the theoretical minimum and maximum scores a system could produce.
- To get around this, we need to perform **score normalisation** so that each document's score is in a **comparable range**.

Score Normalisation

- There are a number of approaches to normalising scores*.
- The most common is known as **standard normalisation** and is calculated by:

$$\text{normalised_score} = \frac{\text{unnormalised_score} - \text{min_score}}{\text{max_score} - \text{min_score}}$$

- The first-ranked document in each result set will have a normalised score of 1, and the lowest ranked document will receive a normalised score of 0.
- Once the score of every document has been normalised, we can then apply the CombSUM algorithm.

* Montague, M., & Aslam, J. A. (2001). Relevance score normalization for metasearch. In *CIKM '01: Proceedings of the Tenth International Conference on Information and Knowledge Management* (pp. 427–433). New York, NY, USA.

Score Normalisation: CombSUM

System A	
Document	Score
d ₁₉	0.90
d ₅	0.85
d ₁₂	0.82
d ₄	0.79
d ₁₄	0.77
d ₁₅	0.64
d ₁	0.44
d ₉	0.43
d ₁₀	0.41
d ₁₁	0.38

System B	
Document	Score
d ₅	943
d ₁₄	920
d ₂₀	901
d ₇	875
d ₁	862
d ₁₁	811
d ₁₈	795
d ₃	770
d ₁₀	732
d ₁₂	712

Fused	
Document	Score
d ₅	943.85
d ₁₄	920.77
d ₂₀	901.00
d ₇	875.00
d ₁	862.44
d ₁₁	811.38
d ₁₈	795.00
d ₃	770.00
d ₁₀	732.41
d ₁₂	712.82

Score Normalisation

System A		
Document	Score	Normalised
d ₁₉	0.9	1.00
d ₅	0.85	0.90
d ₁₂	0.82	0.85
d ₄	0.79	0.79
d ₁₄	0.77	0.75
d ₁₅	0.64	0.50
d ₁	0.44	0.12
d ₉	0.43	0.10
d ₁₀	0.41	0.06
d ₁₁	0.38	0.00

System B		
Document	Score	Normalised
d ₅	943	1.00
d ₁₄	920	0.90
d ₂₀	901	0.82
d ₇	875	0.71
d ₁	862	0.65
d ₁₁	811	0.43
d ₁₈	795	0.36
d ₃	770	0.25
d ₁₀	732	0.09
d ₁₂	712	0.00

Score Normalisation: CombSUM

System A		System B		Fused	
Document	Normalised	Document	Normalised	Document	Score
d ₁₉	1.00	d ₅	1.00	d ₅ * (*=in both)	1.90
d ₅	0.90	d ₁₄	0.90	d ₁₄ *	1.65
d ₁₂	0.85	d ₂₀	0.82	d ₁₉	1.00
d ₄	0.79	d ₇	0.71	d ₁₂ *	0.85
d ₁₄	0.75	d ₁	0.65	d ₂₀	0.82
d ₁₅	0.50	d ₁₁	0.43	d ₄	0.79
d ₁	0.12	d ₁₈	0.36	d ₁ *	0.77
d ₉	0.10	d ₃	0.25	d ₇	0.71
d ₁₀	0.06	d ₁₀	0.09	d ₁₅	0.50
d ₁₁	0.00	d ₁₂	0.00	d ₁₁ *	0.43

Score-Based: CombMNZ

- CombMNZ* is a variation of CombSUM that has been shown to give superior results in a variety of scientific studies.
- Although CombSUM makes use of the **Chorus Effect** by adding the normalised scores, CombMNZ **emphasises this even further**.
- It does this by **multiplying** each document's CombSUM score by the **number of result sets** it was contained in (i.e. the number of input systems that give it a non-zero score).
- The MNZ stands for "Multiply by Non Zero".

* Fox and Shaw (1994)

Score-Based: CombMNZ Example

■ Example 1:

- Document #1 is given a score of 0.45 by System A
- Document #1 is given a score of 0.3 by System B
- Document #1 is given a score of 0.35 by System C
- Document #1's CombMNZ score is $(0.45 + 0.3 + 0.35) \times 3 = 3.3$

■ Example 2:

- Document #2 is given a score of 0.55 by System A
- Document #2 is not returned by System B
- Document #2 is given a score of 0.65 by System C
- Document #2's CombMNZ score is $(0.55 + 0 + 0.65) \times 2 = 2.4$

Score Normalisation: CombMNZ

System A		System B		Fused	
Document	Normalised	Document	Normalised	Document	Score
d ₁₉	1.00	d ₅	1.00	d ₅ [*] (* = in both)	3.80
d ₅	0.90	d ₁₄	0.90	d ₁₄ [*]	3.30
d ₁₂	0.85	d ₂₀	0.82	d ₁₂ [*] (↑ ¹)	1.70
d ₄	0.79	d ₇	0.71	d ₁ [*] (↑ ³)	1.53
d ₁₄	0.75	d ₁	0.65	d ₁₉	1.00
d ₁₅	0.50	d ₁₁	0.43	d ₁₁ [*] (↑ ⁴)	0.86
d ₁	0.12	d ₁₈	0.36	d ₂₀	0.82
d ₉	0.10	d ₃	0.25	d ₄	0.79
d ₁₀	0.06	d ₁₀	0.09	d ₇	0.71
d ₁₁	0.00	d ₁₂	0.00	d ₁₅	0.5

Score-Based: CombMNZ

- As with CombSUM, CombMNZ should only be used for **data fusion tasks**, since it gives higher rankings to documents that appear in multiple result sets.
- CombMNZ is very simple to calculate and has been shown to be quite effective in practice.
- For this reason, it tends to be the most common baseline technique that researchers compare new algorithms against.

Score-Based: Linear Combination

- One difficulty with CombMNZ and CombSUM is that (like interleaving) every input result set is **assumed to be of equal quality**.
- The *Linear Combination Model* addresses this by apply a **weighting** to each input system*.
- The score calculation is similar to CombSUM, except that each document's normalised scores are multiplied by the weighting associated with the input system that returned it before they are added together.

* Vogt & Cottrell (1999)

Linear Combination Example

- Example 1 (System A, B and C have weights of 1, 2, 3 respectively):

- Document #1 is given a score of 0.45 by System A
- Document #1 is given a score of 0.3 by System B
- Document #1 is given a score of 0.35 by System C
- Document #1's overall score is:

$$(0.45 \times 1) + (0.3 \times 2) + (0.35 \times 3) = 2.1$$

- Example 2:

- Document #2 is given a score of 0.55 by System A
- Document #2 is not returned by System B
- Document #2 is given a score of 0.65 by System C
- Document #2's overall score is:

$$(0.55 \times 1) + (0 \times 2) + (0.65 \times 3) = 2.5$$

Linear Combination: Weights

- Many approaches have been used to calculate weights for a linear combination. Here are two notable ones.
 1. The *CORI* algorithm*, which relies on having access to **details about the document collection** each input system is using (generally used for Distributed IR).
 - **Document Frequency**: number of documents in the document collection that contains a term.
 - **Inverse Collection Frequency**: based on the number of collections that contain the term.
 2. *LMS*** (using result Length to calculate Merging Score): longer result set results in higher weight. Worked surprisingly well in practice.

* Callan, J. P., Lu, Z., & Croft, W. B. (1995). Searching distributed collections with inference networks. In *SIGIR '95: Proceedings of the 18th annual international ACM SIGIR conference on Research and development in information retrieval* (pp. 21–28). New York, NY, USA.

** Rasolofo, Y., Abbaci, F., & Savoy, J. (2001). Approaches to Collection Selection and Results Merging for Distributed Information Retrieval. In *CIKM '01: Proceedings of the Tenth International Conference on Information and Knowledge Management* (pp. 191–198). New York, NY, USA.

Fusion: Case Study: Probability-Based Fusion

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

ProbFuse - Motivation

- Using a single weight for each system only reflects that the **overall average performance** of one system is better (or worse) than another.
- This can miss some characteristics that we may wish to make use of.
 - An input system may tend to return a few relevant documents at the start of its results, so precision and recall may both be poor.
 - Another system could have good recall, but be poor at ranking its results so as to place relevant documents first.

ProbFuse - Motivation

- A better solution than a single weighting was sought.
- **Probabilistic** data fusion algorithms try to reflect the **positions/ranks** in the result sets where input systems **tend to return relevant documents**.
- **Past results** are analysed during a **training phase** to build a **series of weights**, depending on the position in a result set that a document appears in.

ProbFuse - Overview

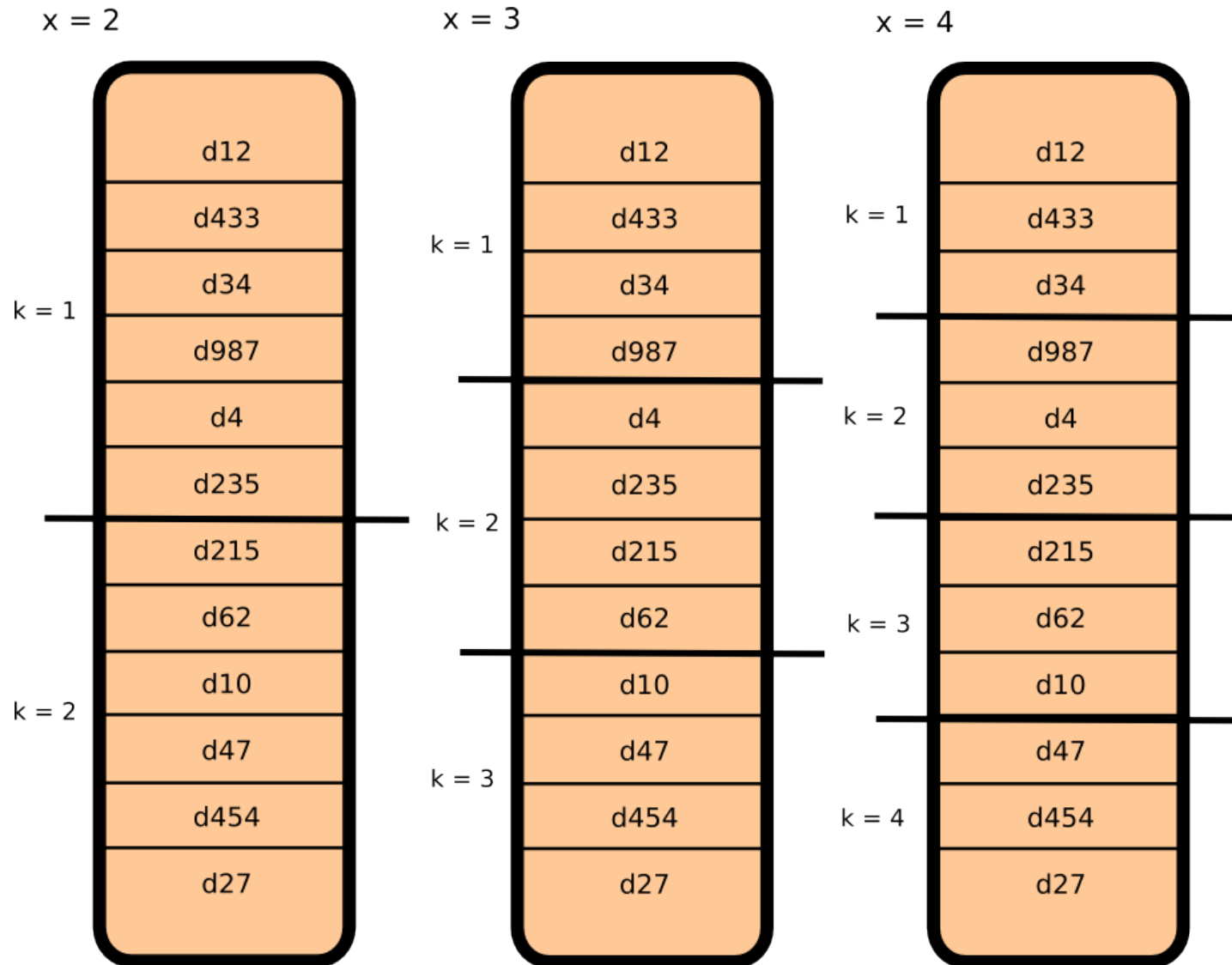
- *ProbFuse* was designed to take this information into account.*
- Each input result set is divided into x **equal-sized segments**.
- The score eventually used to rank a document is based on the **probability that the document is relevant**, given that it was returned by a particular input system in a particular segment.
- These probabilities are calculated by examining **historical data** from each input system.

* Lillis, D., Toolan, F., Collier, R., & Dunnion, J. (2006). ProbFuse: A Probabilistic Approach to Data Fusion. In *Proceedings of the 29th annual international ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '06)* (pp. 139–146). Seattle, WA, USA.

ProbFuse - Overview

- Using ProbFuse for fusion involves **two phases**:
 - **Training Phase**: Probabilities for each input system estimated, based on **historic queries** for which **relevance judgments are available**.
 - Variables:
 - x: how many segments to divide the result set into
 - k: segment number (k=1 is the first segment, k=2 the second, etc.)
 - **Fusion Phase**: Probabilities used to calculate ranking scores for each document. Used to rank final output result set.

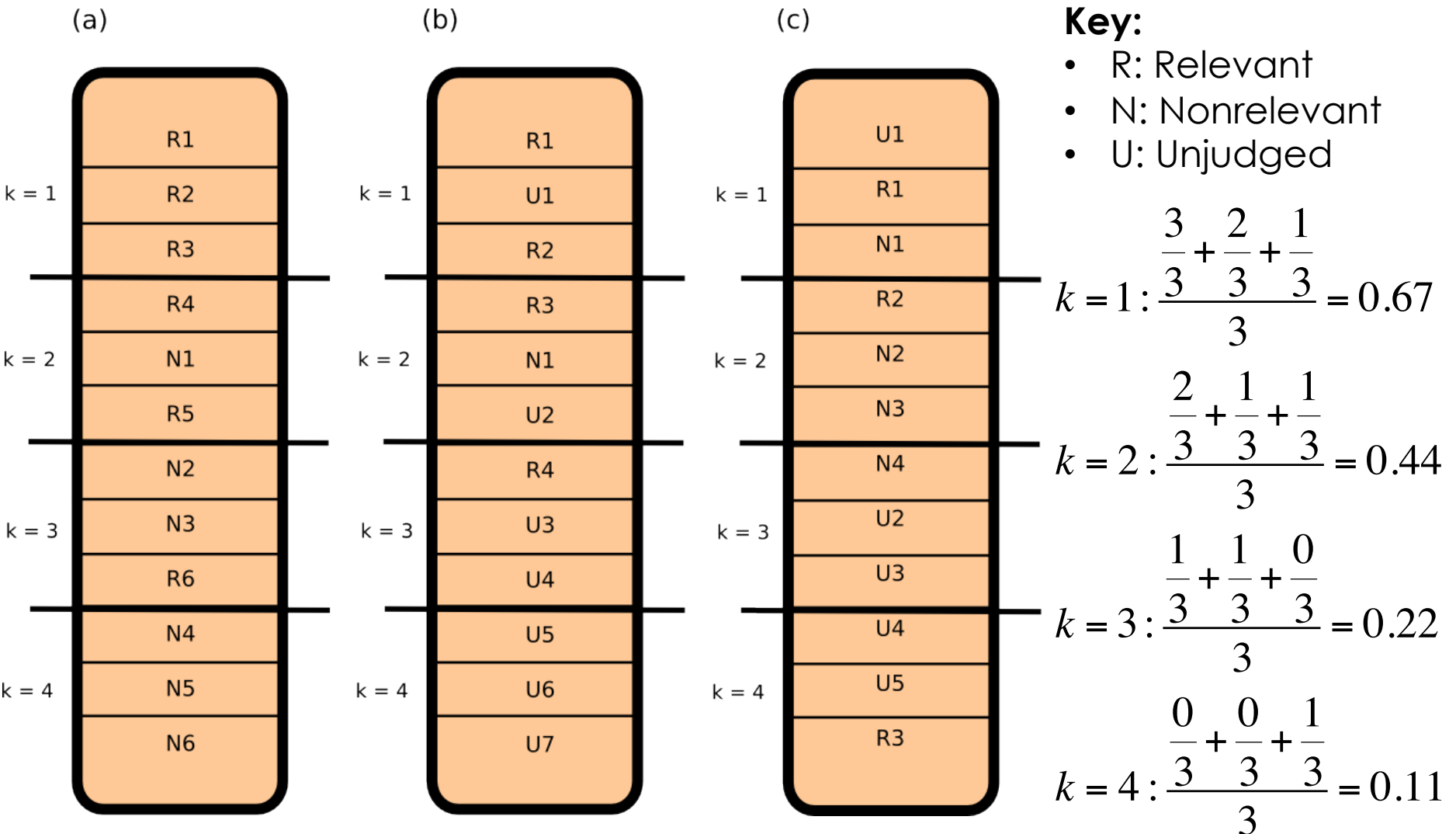
ProbFuse – What are "Segments"?



ProbFuse - Training

- A **set of probabilities** must be calculated for **every input system**.
- For this, a set of **training queries** is required, where relevance judgments are available.
- Calculate the probability that document d returned in segment k is relevant.
 1. Count all of the relevant documents in segment k
 2. Divide by total number of documents in segment k
 3. Average over all training queries

ProbFuse - Training



Result sets returned by the **same input system** in response to **different training queries**

ProbFuse - Fusion

- The score assigned to a document by each underlying input system is the **probability of relevance** associated with the **segment it is returned in**, divided by the **segment number**.
- Dividing by the segment number gives highly-ranked documents an additional boost, so as to **exploit the Skimming Effect**.
- The document's final ranking score is found by **adding each individual score**.
 - This exploits the **Chorus Effect** (higher score from being returned in multiple result sets)
- ProbFuse is therefore only suitable for **data fusion**.

SegFuse - Motivation

- Using ProbFuse, each segment is of **equal size**.
- Experiments on standard datasets showed that dividing result sets into **25 segments** resulted in **improvements over CombMNZ** in MAP and P@10 scores (bpref was inconclusive).
- The input result sets used were up to 1000 documents in length, meaning that **each segment contained 40 documents**.
- This means that document 40 would be treated the same as document 1.

SegFuse - Overview

- Shokouhi argues that this loses information, since relevant documents are more likely to be found in early positions.*
- Two major variations to ProbFuse proposed:
 - Instead of constant segment sizes, the size of the segments **increase exponentially** as we go down the result set.
 - **Normalised scores** used to boost the **Skimming Effect**, rather than dividing by the segment number.
 - For each document, the probability score of its segment was multiplied by its normalised score.

* Shokouhi, M. (2007). Segmentation of Search Engine Results for Effective Data-Fusion. *Advances in Information Retrieval*, 4425

SegFuse - Training

- When training SegFuse, the probability that a document in a given segment is relevant is calculated in **exactly the same way** as for ProbFuse.
- The only difference is that the **size of the segment** varies.
- It is given by: $Size_k = (10 \times 2^{k-1}) - 5$ where k is the segment number.

Segment Number	Size
1	5
2	15
3	35
4	75
5	155

SlideFuse - Motivation

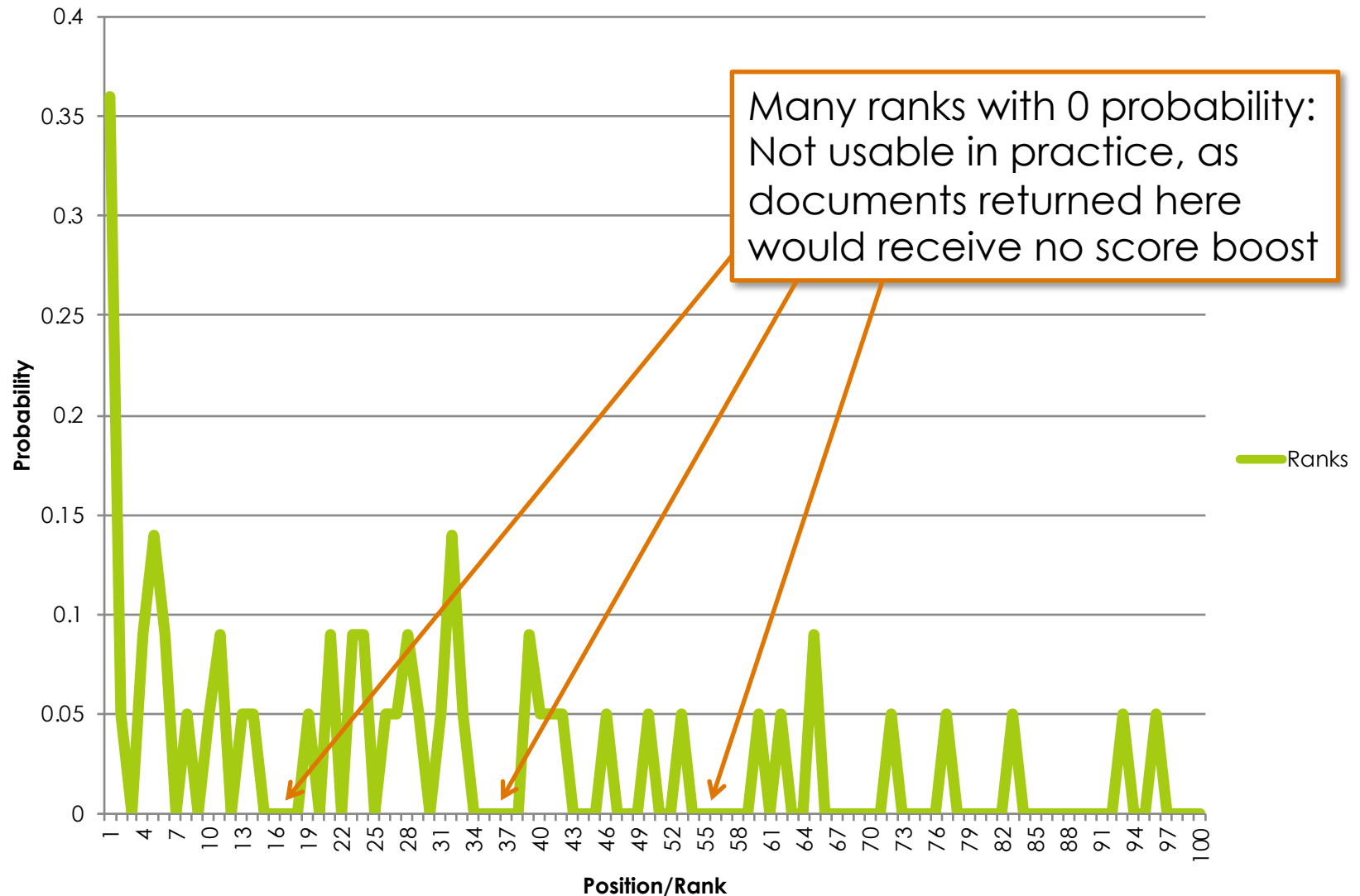
- SegFuse achieved **improvements over ProbFuse** in most of the experiments Shokouhi conducted, especially when measured using MAP.
- However, even with this improved method of dividing result sets into segments, there were still elements of concern.
- In particular, **adjacent documents** could be treated very differently, for instance under SegFuse:
 - Document 20 would be grouped with documents 6-20
 - Document 21 would be grouped with documents 21-55

SlideFuse - Overview

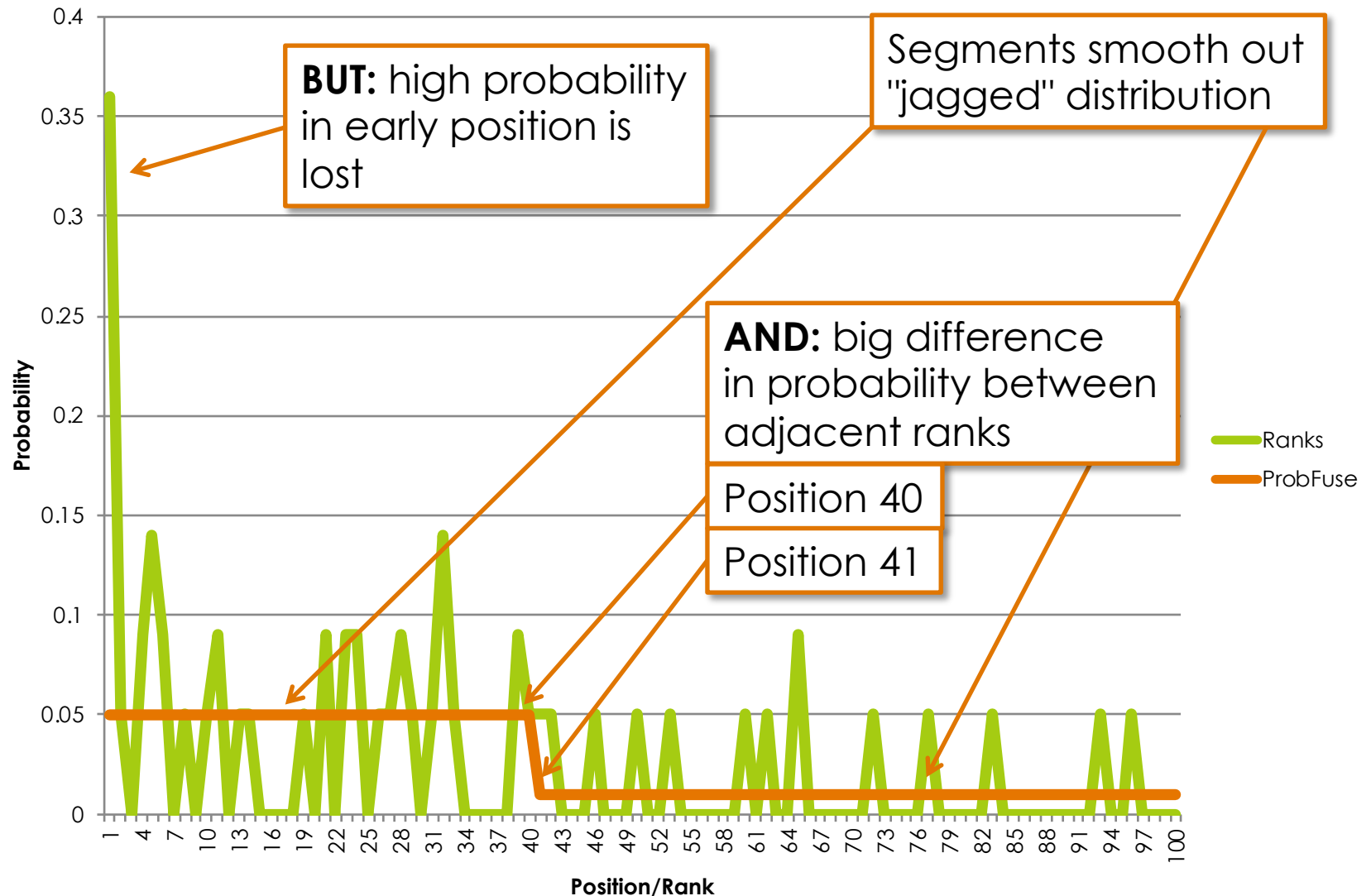
- This cutoff between segments means that documents that are side-by-side in a result set can be **treated very differently**.
- To deal with this, the **SlideFuse** algorithm was proposed.*
- **Aside:** Why not just use the probability at **each rank** in the result set?
- Especially for large-scale tasks, there may be few judged relevant documents available, but we don't want to give a probability of zero to a rank just because no relevant document was returned in that exact rank during training.

* Lillis, D., Toolan, F., Collier, R., & Dunnion, J. (2008). Extending Probabilistic Data Fusion Using Sliding Windows. In C. Macdonald, I. Ounis, V. Plachouras, I. Ruthven, & R. W. White (Eds.), *Advances in Information Retrieval. Proceedings of the 30th European Conference on Information Retrieval Research (ECIR 2008)* (Vol. 4956, pp. 358–369)

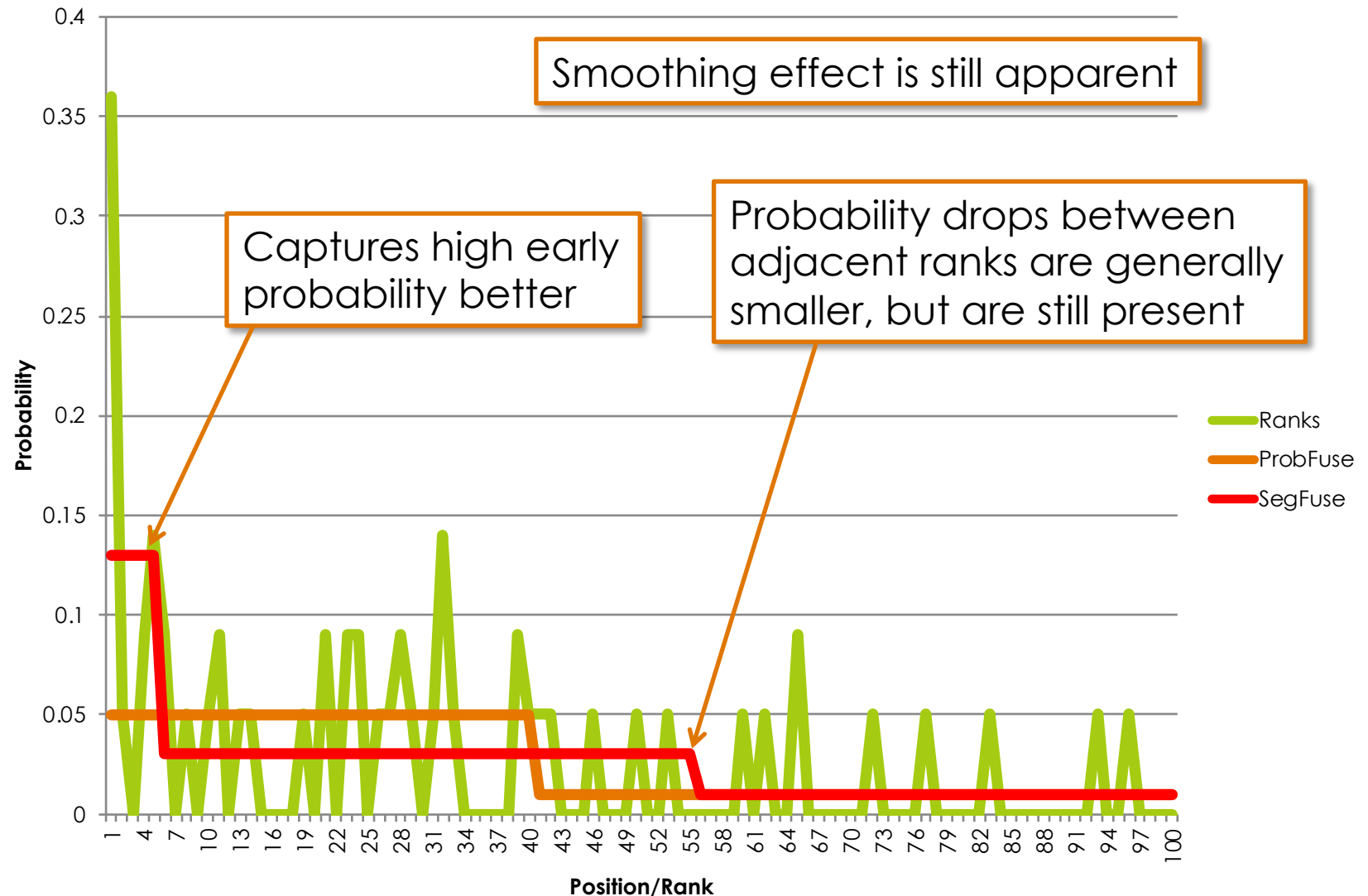
Distribution of Probabilities: Ranks



Distribution of Probabilities: ProbFuse



Distribution of Probabilities: SegFuse

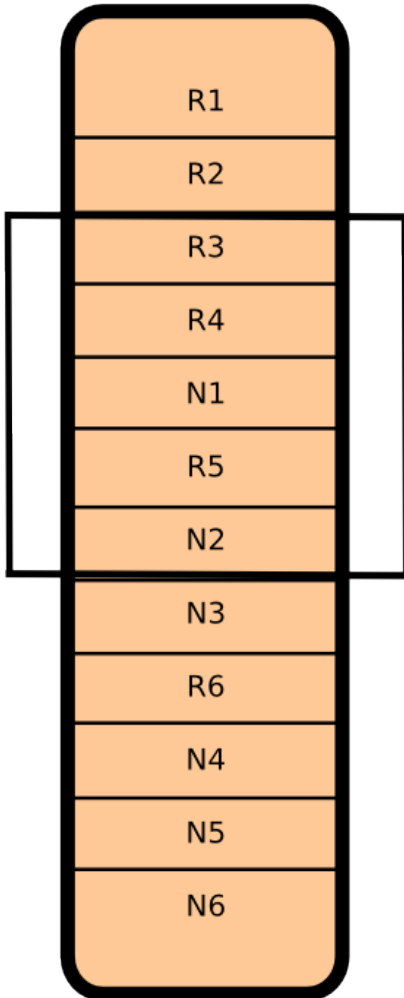


SlideFuse - Overview

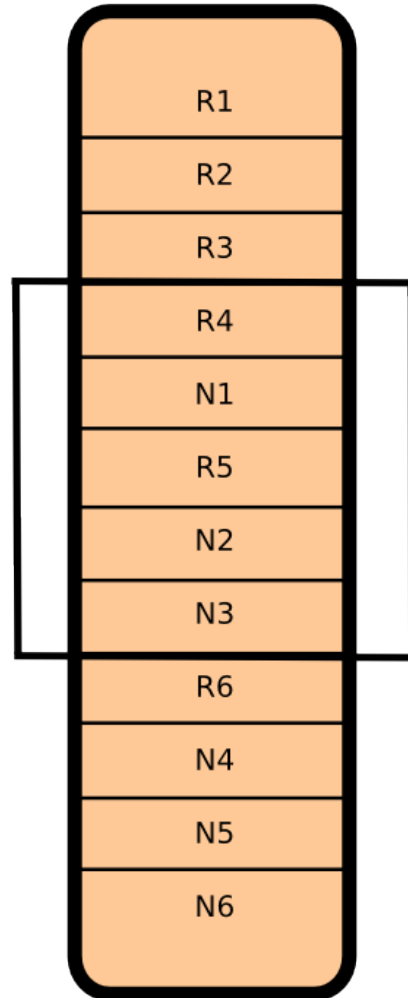
- Instead of dividing the result sets into segments, we instead use **each rank's neighbours** as evidence in estimating its probability.
- This will allow us to **smooth** the probability distribution graph to avoid the jagged peaks seen in the "ranks-only" distribution and also **avoid the sudden drops** in probability values seen with ProbFuse and SegFuse.
- We describe this as using "**sliding windows**", as the group of documents to be taken into account changes depending on which rank we are examining.

Example: Sliding Windows

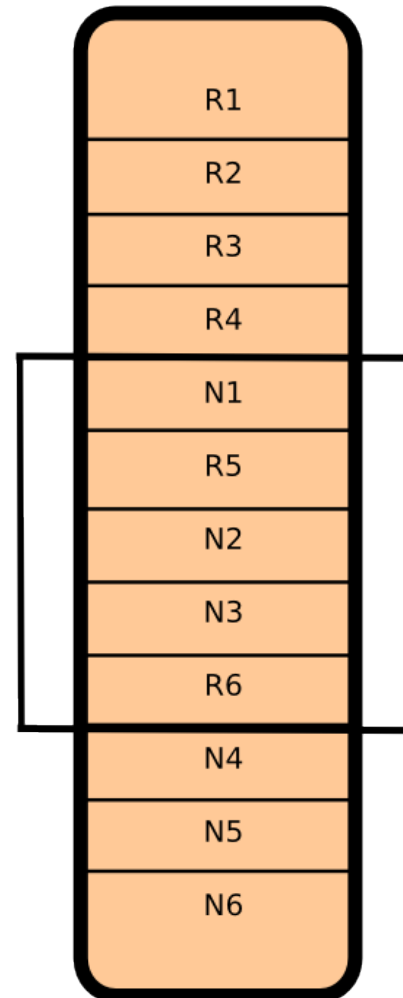
(a)



(b)



(c)



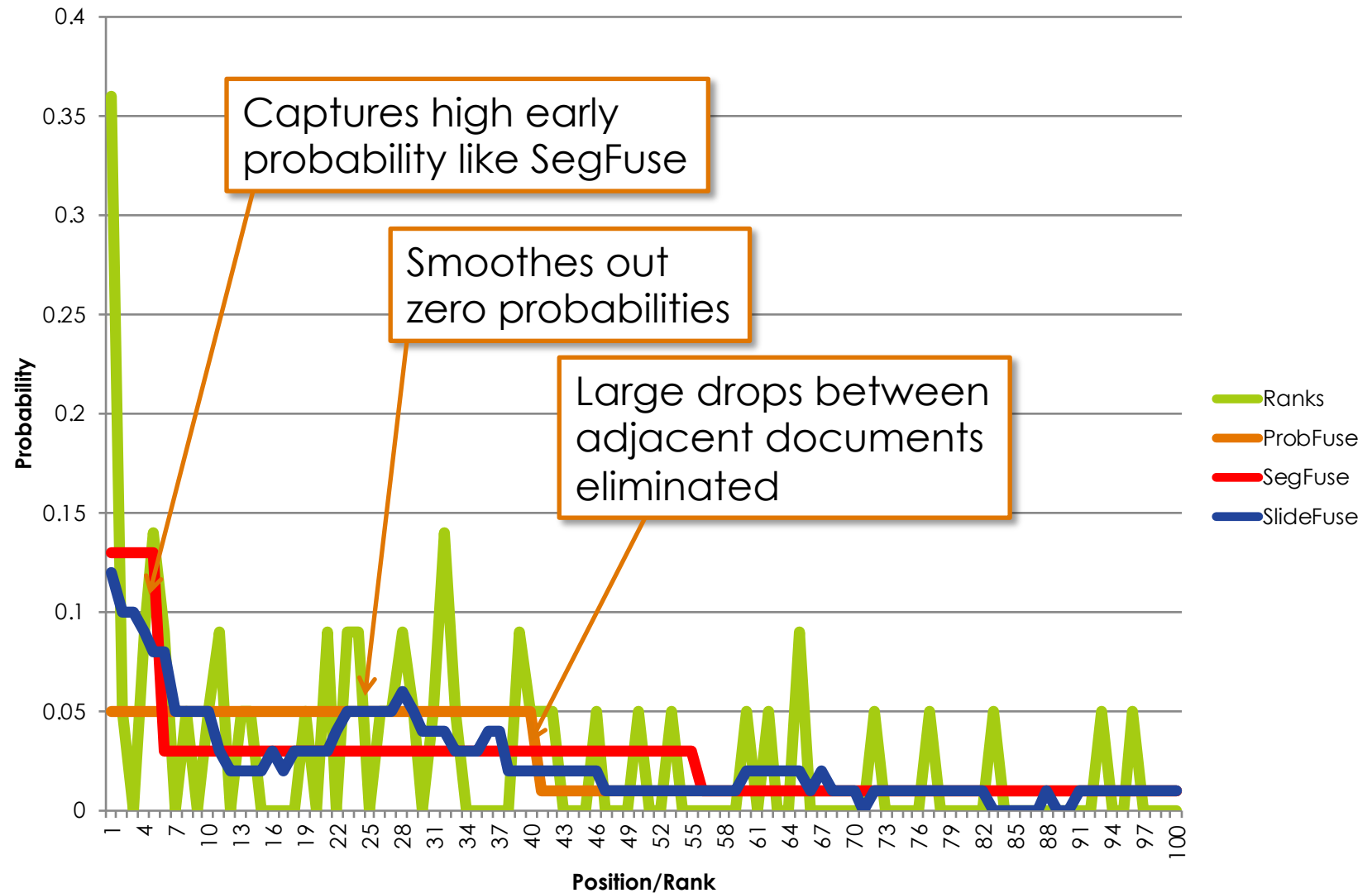
Result set for one training query returned by one system, showing sliding window for document:

- a) N1
- b) R5
- c) N2

SlideFuse - Training

- **Training phase:** for each input system, first calculate the probability of relevance at each rank r .
 - Number of relevant documents returned at rank r divided by number of training queries.
- **Fusion phase:** final score is the sum of the scores a document receives from the input systems.
 - For each input system, get the average probability in the sliding window that surrounds the document,.

Distribution of Probabilities: SlideFuse



Conclusions

- SlideFuse achieves superior results to CombMNZ, ProbFuse and SegFuse on the TREC-2004 web track data used.
- Statistically significant performance increase for:
 - 4/5 runs measured with MAP
 - 3/5 runs measured with bpref
 - 5/5 runs measured with P@10
- Average difference is $< 1\%$ in all other cases

PageRank

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Introduction

- Using the models we have seen so far (e.g. Vector Space, BM25), each document is treated completely **separately** to other documents.
- The ranking score is calculated based entirely on the **content** of the document itself.
- This does not take into account everything a human judge would use in deciding whether or not a document is suitable for returning in response to a particular query.
- **Learning to Rank** combines the ranking score with other **features** that might help to indicate the **relevance** of a document, or the **degree of relevance** of a document.
- One feature that can be used is a measure of the **importance** of a document.
 - A high ranking for important documents will be beneficial to users.
- It may seem that this is not an easy thing to accomplish...

Document Importance

- There are, however, some areas where it may be possible to estimate how **important** a document is.
- For instance, in academic writing a paper will generally **cite** other works that influenced it.
- Some organisations use “citation counting” to measure the influence of a piece of work.
- The intuition is that the more times your paper is cited by others, the more influence it must have had in its field.
- Also, if a paper appears in an influential journal, it is likely to be important.

REFERENCES

- Bloom, B. H. (1970). Space/Time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 13(7), 422–426.
- Breitinger, F., & Baier, H. (2012). Similarity Preserving Hashing: Eligible Properties and a New Algorithm MRSH-v2. In *International conference on digital forensics and cyber crime* (pp. 167–182). Springer.
- Breitinger, F., Baier, H., & White, D. (2014). On the Database Lookup Problem of Approximate Matching. *Digital Investigation*, 11, S1–S9. doi: 10.1016/j.diin.2014.03.001
- Breitinger, F., Guttman, B., McCarrin, M., Roussev, V., & White, D. (2014). Approximate Matching: Definition and Terminology. *NIST Special Publication*, 800, 168.
- Breitinger, F., Rathgeb, C., & Baier, H. (2014, sep). An Efficient Similarity Digests Database Lookup - A Logarithmic Divide & Conquer Approach. *Journal of Digital Forensics, Security and Law*, 9(2), 155–166.
- Broder, A. Z. (1997). On the Resemblance and Containment of Documents. In *Compression and complexity of sequences 1997. proceedings* (pp. 21–29). doi: 10.1109/SEQUEN.1997.666900
- Casey, E., Ferraro, M., & Nguyen, L. (2009). Investigation Delayed is Justice Denied: Proposals for Expediting Forensic Examinations of Digital Evidence. *Journal of forensic sciences*, 54(6), 1353–1364.
- de Braekt, R. I., Le-Khac, N. A., Farina, J., Scanlon, M., & Kechadi, T. (2016, April). Increasing Digital Investigator Availability Through Efficient Workflow Management and Automation. In *4th international symposium on digital forensic and security (isdfs)* (p. 68–73). doi: 10.1109/ISDFS.2016.7473520
- Gupta, J. N., Kalaimannan, E., & Yoo, S.-M. (2016). A Heuristic for Maximizing Investigation Effectiveness of Digital Forensic Cases Involving Multiple Investigators. *Computers & Operations Research*, 69, 1–9. doi: 10.1016/j.cor.2015.11.003
- Gupta, V., & Breitinger, F. (2015). How cuckoo filter can improve existing approximate matching techniques. In J. James & F. Breitinger (Eds.), *Digital forensics and cyber crime* (Vol. 157, p. 39–52). Springer International Publishing. doi: 10.1007/978-3-319-25512-5_4
- Harichandran, V. S., Breitinger, F., & Baggili, I. (2016). Byte-wise Approximate Matching: The Good, The Bad, and The Unknown. *The Journal of Digital Forensics, Security and Law: JDFSL*, 11(2), 59.
- James, J. I., & Gladyshev, P. (2015). Automated Inference of Past Action Instances in Digital Investigations. *International Journal of Information Security*, 14(3), 249–261. doi: 10.1007/s10207-014-0249-6
- Kornblum, J. (2006). Identifying Identical Files Using Context Triggered Piecewise Hashing. *Digital investigation*, 3, 91–97. doi: 10.1016/j.diin.2006.06.015
- Lillis, D., Becker, B., O’Sullivan, T., & Scanlon, M. (2016). Current Challenges and Future Research Areas for Digital Forensic Investigation. In *11th ADFSL Conference on Digital Forensics, Security and Law (CDFSL 2016)*. Daytona Beach, FL, USA: ADFSL. doi: 10.13140/RG.2.2.34898.76489
- Lillis, D., Breitinger, F., & Scanlon, M. (2017). Expediting MRSH-v2 Approximate Matching with Hierarchical Bloom Filter Trees. In *9th EAI International Conference on Digital Forensics and Cyber Crime (ICDF2C 2017)*. Prague, Czech Republic.
- Oliver, J., Cheng, C., & Chen, Y. (2013). TLSH – A Locality Sensitive Hash. In *Fourth Cybercrime and Trustworthy Computing Workshop (CTC), 2013* (pp. 7–13). doi: 10.1109/CTC.2013.9
- Quick, D., & Choo, K.-K. R. (2014). Impacts of Increasing Volume of Digital Forensic Data: A Survey and Future Research Challenges. *Digital Investigation*, 11(4), 273–294. doi: 10.1016/j.diin.2014.09.002
- Rogers, M. K., Goldman, J., Mislan, R., Wedge, T., & Debrota, S. (2006). Computer Foren-

Citation Counts and the Web

- The World Wide Web contains the largest collection of documents in existence, and is a most suitable forum for IR.
- Unlike in traditional IR systems, the documents available on the web are **not standalone**.
- Because of the presence of **hypertext** (i.e. links) in documents on the web, there is an **interconnection** between these documents.
- A number of researchers have attempted to apply principles similar to citation counts to web pages.
- This operates on the assumption that a page that is linked to from many other pages is important and this should be reflected in search results.

Citation Counts and the Web

- There are, however, a few notable differences between academic publishing and web publishing that must be taken into account:
- **Circular references** - in academic publishing, a paper can only cite a paper that has already been published. Later papers are not cited by earlier ones. On the web, two pages can link to one another.

Circular References

Republic of Ireland

From Wikipedia, the free encyclopedia

This article is about the sovereign state. For the revolutionary republic of 1919–1922, see [Irish Republic](#). For other uses, see [Ireland \(disambiguation\)](#).

Ireland (ⁱ/ˈɑːrələnd/; Irish: *Éire* [ˈeːɾʲə] (listen)), also known as the **Republic of Ireland** (*Poblacht na hÉireann*), is a [sovereign state](#) in north-western [Europe](#) occupying about five-sixths of the [island of Ireland](#). The capital and largest city is **Dublin**, which is located on the eastern part of the island, and whose metropolitan area is home to around a third of the country's 4.6 million inhabitants. The state shares its only land border with [Northern Ireland](#), a [part](#) of the [United Kingdom](#). It is otherwise surrounded by the Atlantic Ocean, with the [Celtic Sea](#) to the south, [Saint George's Channel](#) to the south-east and the [Irish Sea](#) to the east. It is a [unitary, parliamentary republic](#).^[9] The legislature, the *Oireachtas*, consists of a [lower house](#), *Dáil Éireann*, an [upper house](#), *Seanad Éireann*, and an elected [President](#) (*Uachtarán*) who serves as the largely ceremonial [head of state](#), but with some important powers and duties. The [head of government](#) is the *Taoiseach* (Prime Minister, literally 'Chief', a title not used in English), who is elected by the *Dáil* and appointed by the President, and appoints other government ministers.

Dublin

From Wikipedia, the free encyclopedia

This article is about the capital of Ireland. For other uses, see [Dublin \(disambiguation\)](#).

Dublin (ⁱ/dʌblɪn/, Irish: *Baile Átha Cliath* [ˈbˠalˠaːkˠlʲiəh]) is the capital and largest city of Ireland.^[9] Dublin is in the province of [Leinster](#) on Ireland's east coast, at the mouth of the [River Liffey](#). The city has an urban area population of 1,273,069.^[10] The population of the [Greater Dublin Area](#), as of 2011, was 1,801,040 persons.

Citation Counts and the Web

- **Quality control** - in academic publishing, a paper is peer-reviewed before it is approved for publication. Thus some quality is maintained in the papers that include citations. On the web, anybody can publish material and link to other documents. It would be a trivial task to write a program that would generate hundreds or thousands of pages containing links to somewhere else.

PageRank

- In 1998, Sergey Brin and Larry Page published the PageRank algorithm, which to measure the importance of web pages*.
- This later went on to be a core element in the success of the Google search engine.
- The algorithm itself has been modified since (secretly: Google rarely reveal anything about the search engine's inner workings anymore) to avoid situations where it was being exploited by malicious publishers.
- However, the core of how it functions has largely remained unchanged.

* S. Brin and L. Page, "The anatomy of a large-scale hypertextual Web search engine", *Computer networks and ISDN systems*, 30(1-7), 107-117. 1998.

PageRank

- Again, it is based on the premise that documents that are linked to by many other documents are important, and should receive a boost in search engine rankings as a result.
- A document will tend to have a high PageRank score if:
 - It is linked to by many documents and/or
 - It is linked to by documents that themselves have a high PageRank.

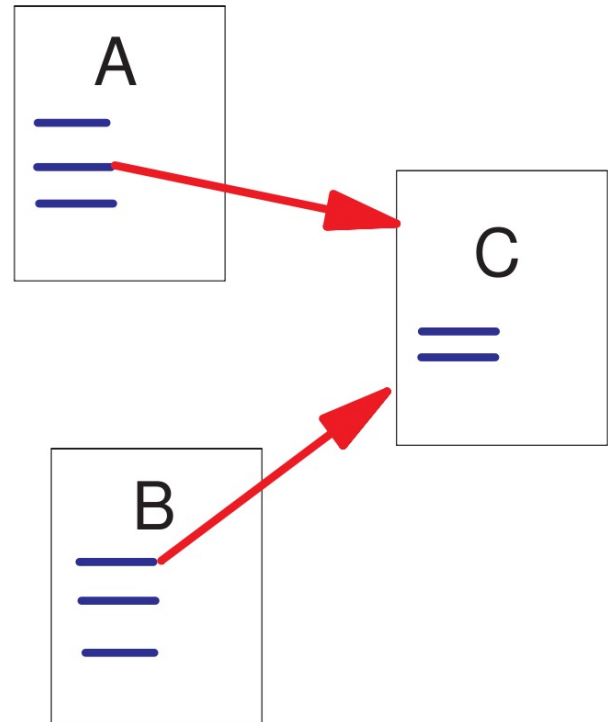
PageRank - Link Structure

The web is made up of HTML pages that are connected using hyperlinks.

We can think of this as a **directed graph**.

Pages A, B, C are vertices in the graph.

A and B have links (edges) to Page C.



PageRank - Link Structure

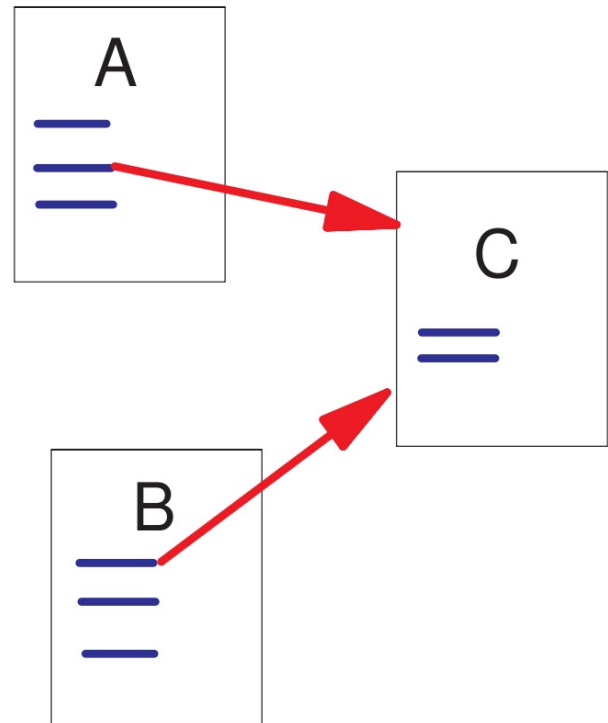
In their 1998 crawl, Brin and Page had 150 million vertices (pages) and 1.7 billion edges (links).

We say that A and B are **backlinks** of C.

We say that A and B have **outlinks** to C.

A document will have high PageRank if it has:

- Many backlinks
- Backlinks with high PageRank



PageRank: Simplified Version

- At a basic level, PageRank works using a formula similar to the following...
- $$R(u) = \sum_{v \in B_u} \frac{R(v)}{N_v}$$
 - $R(u)$ is the PageRank score for document u .
 - B_u is the set of all backlinks of document u .
 - $R(v)$ is the PageRank score for document v .
 - N_v is the number of outlinks in document v .

PageRank: Simplified Version

What does this mean?

A document contributes $\frac{R(v)}{N_v}$ to the PageRank of each document it links to.

That is, if a document links to 4 pages, its contribution to each of those pages is $\frac{1}{4}$ of its own PageRank.

- $R(u) = \sum_{v \in B_u} \frac{R(v)}{N_v}$
- $R(u)$ is the PageRank score for document u .
- B_u is the set of all backlinks of document u .
- $R(v)$ is the PageRank score for document v .
- N_v is the number of outgoing links in document v .

PageRank: Simplified Version

So if a backlink has high PageRank (and few outlinks), this will have a beneficial effect.

A document's final PageRank score is the sum of each of these contributions from backlinks.

The more backlinks a document has, the higher its PageRank will be.

- $R(u) = \sum_{v \in B_u} \frac{R(v)}{N_v}$
- $R(u)$ is the PageRank score for document u .
- B_u is the set of all backlinks of document u .
- $R(v)$ is the PageRank score for document v .
- N_v is the number of outlinks in document v .

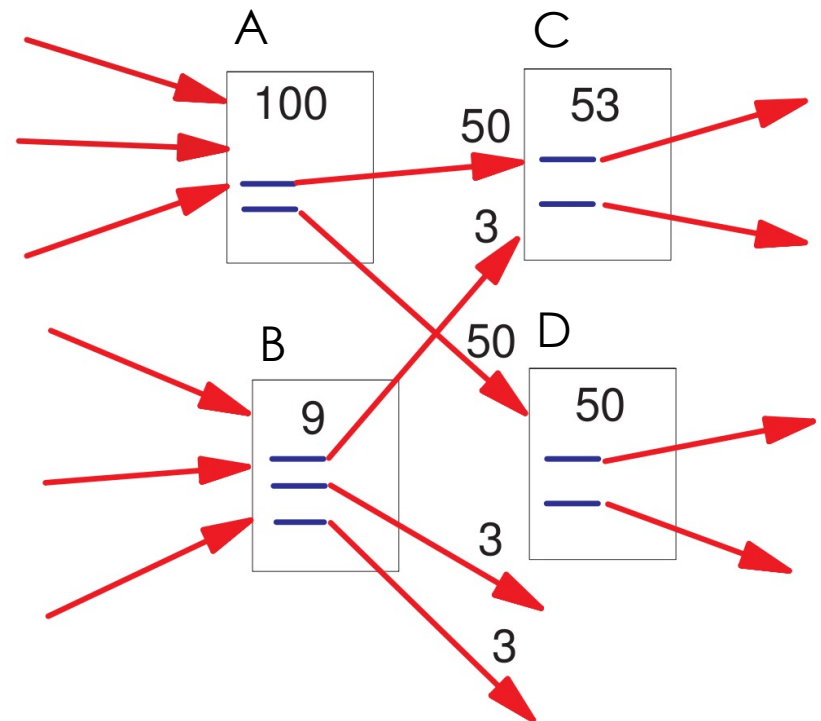
PageRank - Simplified Version

Document A has
PageRank of 100 and 2
outlinks:

- It sends 50 to C and 50 to D

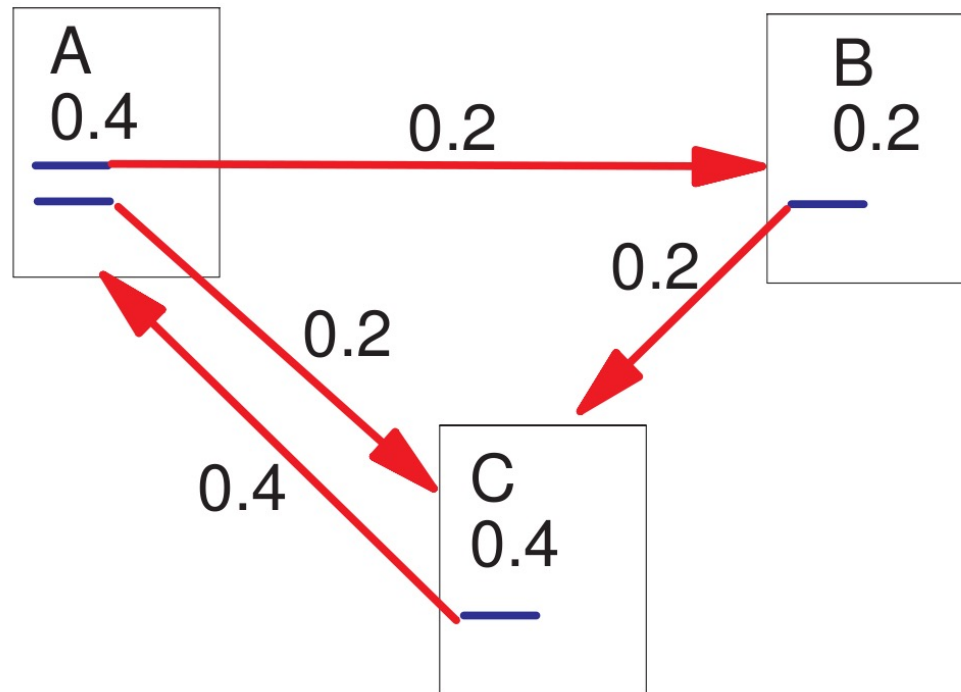
Document B has
PageRank of 9 and 3
outlinks:

- It contributes 3 to the
PageRank of each of its
outlinks (including C)



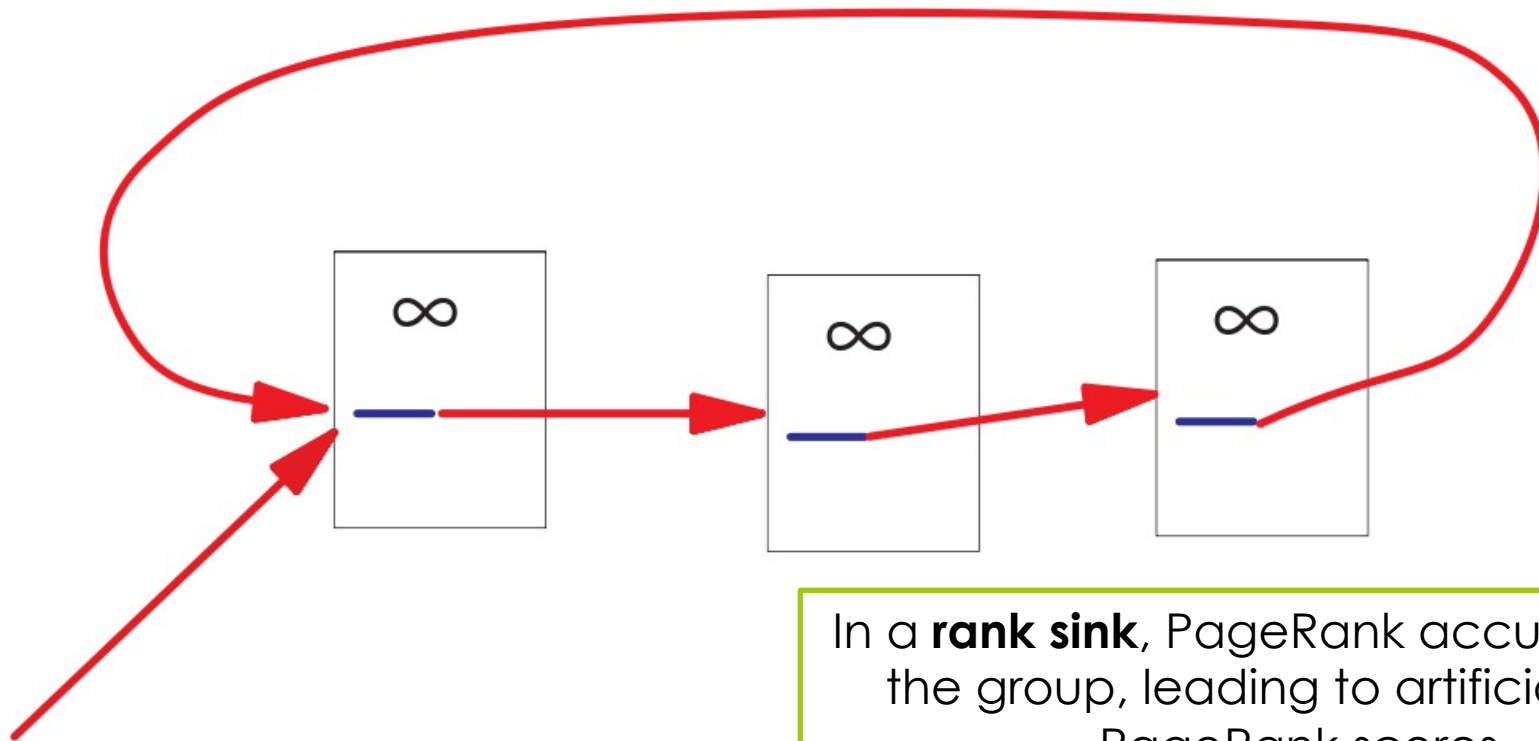
PageRank - Simplified Version

- Question: If we need PageRank to calculate PageRank, where does the initial PageRank come from?
 - At the beginning, we can give an **arbitrary score** to every document.
 - The formula we have seen can then be used to calculate new scores.
 - These **continue to be recalculated** until the scores **converge** (i.e. calculating again does not change the scores, or changes them very little).
 - The scores use as the inputs for each iteration of the algorithm are the scores from the previous iteration.



PageRank - Simplified Version

This image shows a stable state: no matter how many times PageRank is recalculated, the scores for A, B and C will always be the same.



In a **rank sink**, PageRank accumulates in the group, leading to artificially high PageRank scores.

PageRank - Problems

Although this simple example illustrates how PageRank works, it does not deal with certain situations very well.

One such situation is a **rank sink** which refers to a group of pages that have at least one backlink and link to one another, but do not link to anywhere else outside the group.

Combating Rank Sinks

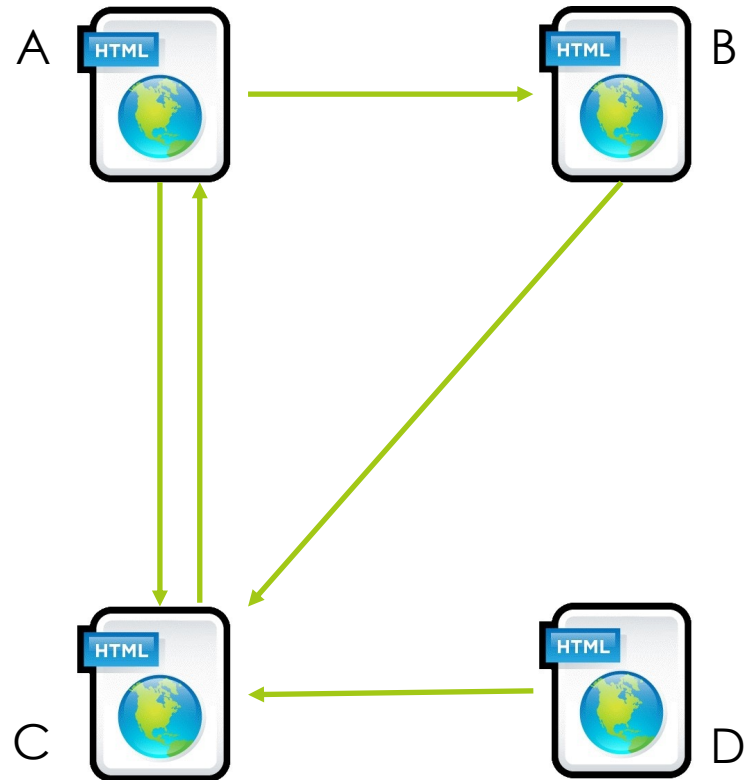
- To combat this type of situation, a new equation is used:
- $$R(u) = (1 - d) + d \times \sum_{v \in B_u} \frac{R(v)}{N_v}$$
- The formula is the very similar to the one we have seen before.
- The difference is the addition of a **damping factor** (d), which ensures that not all of a document's PageRank is passed on via its outlinks.

PageRank - No Backlinks

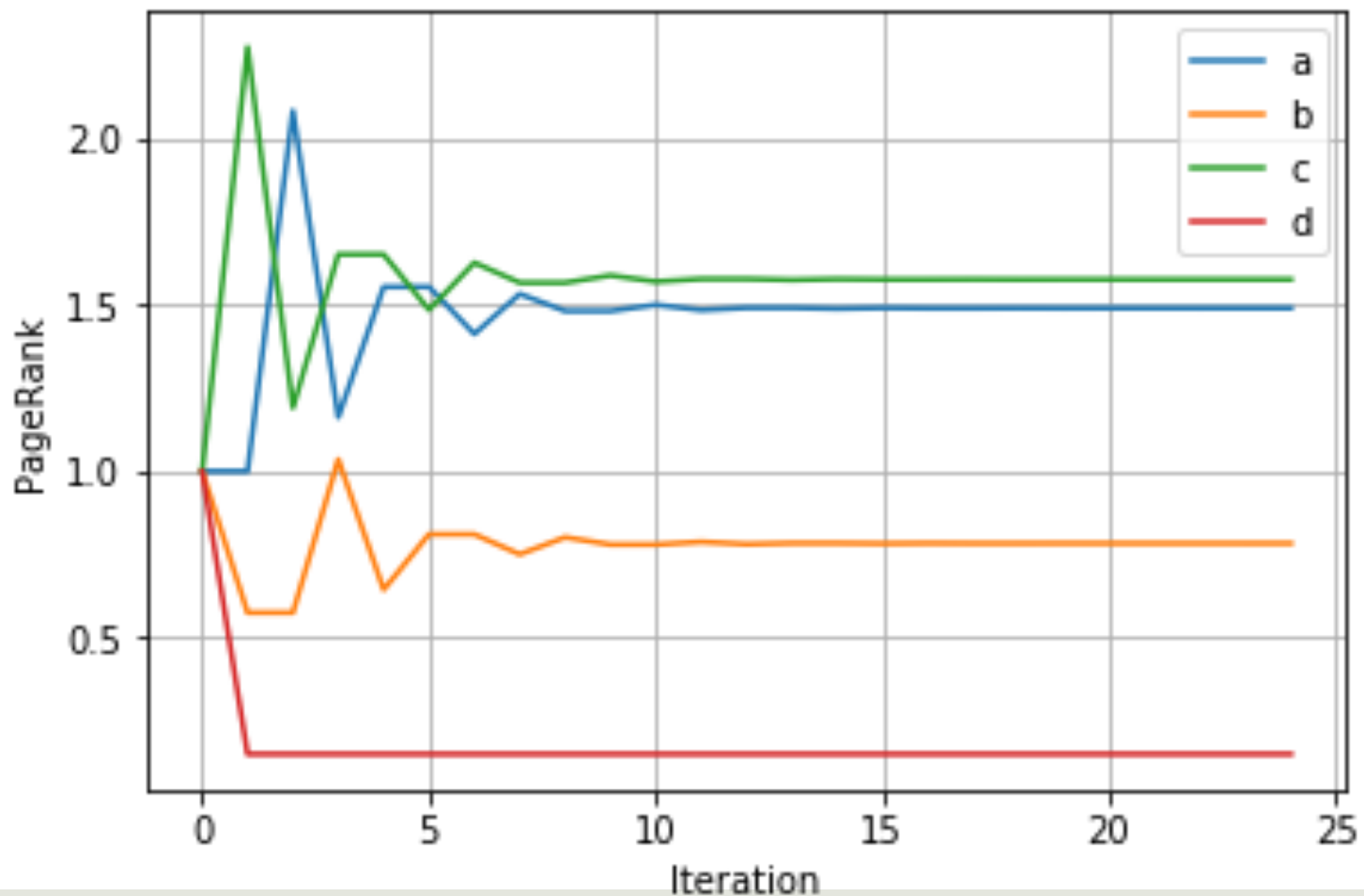
- In the original formula, the only source of PageRank for a document is from its backlinks.
- This meant that a document with no backlinks would have a PageRank of zero.
- This is perfectly acceptable when considering the importance of that document itself.
- However, this also means that it would not contribute anything to the PageRank of documents it links to.
- With the modified formula, a document with no backlinks has a PageRank of $(1-d)$ to contribute to the documents it links to.

PageRank - Example: 4 pages

- Consider the following simple page structure:
 - Page A: links to B and C
 - Page B: links to C
 - Page C: links to A
 - Page D: links to C
- Starting with an initial PageRank of 1 and using a damping factor of 0.85 (which Google appears to use), calculate the PageRank of each document.



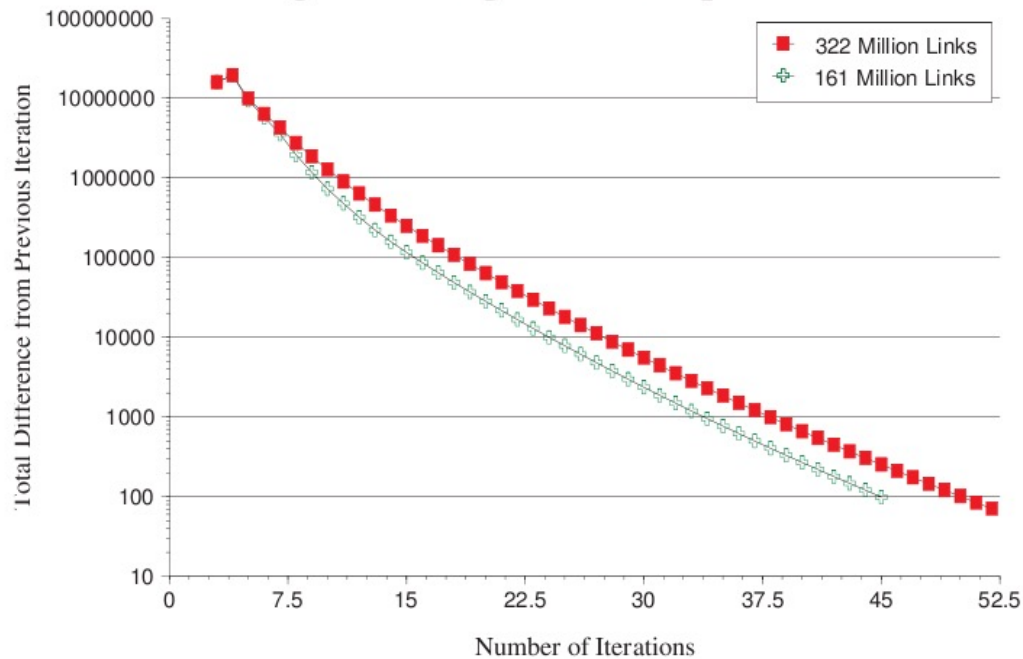
PageRank - Example: 4 Pages



PageRank - Example: 4 Pages

- With this simple system, the PageRank scores have **converged** perfectly after 20 iterations.
- Even after 8 or 9 iterations, the values are very similar to their ultimate values.
- Brin and Page were using a database of 322 million links and believed that the convergence had reached a reasonable level after 52 iterations.
- Calculations on half the data took 45 iterations to get to the same stage.
- This suggests that PageRank scales very well to very large-scale data collections.

Convergence of PageRank Computation



PageRank - Convergence

Doubling the size of the document collection does not double the time taken to converge.

In fact, the increase in the number of iterations required is very small (52 vs. 45): very efficient for larger collections.

Search

Showing Results From 0 to 10

74.79% <http://www.stanford.edu/>
- 3591923 - 01023927

65.78% <http://www.stanford.edu/home/administration/portfolio.html>
3K - 2591993 - 010397

73.26% 138 - 1333195 - 0101397

68.38% <http://www.indiana.edu/>
1k - 04228425 - 01035427

68.07% <http://www.uci.edu/>
36 - 1333396 - 0123927

62.05% <http://www.umn.edu/>
106 - 13376926 - 016928927

66.66% <http://www.iastate.edu>
 36 - 13/18/96 - 01/03/97

66.35% $fk = 32501223 = 011013023$

66.35% <http://www.msstate.edu/>
36 - 31501423 - 01603027

66.15% <http://www.nwu.edu/>
 * - 13/14926 - 0102597

<http://nptich.mcgowan.edu/> - size 1K - 16 Dec 96

<http://www.net.cmu.edu/> - size 4K - 19 Apr 95

<http://www.cis.wisc.edu/~csd> - size 3K - 15 Apr 96

<http://www.sfr.beiz.net/> - size 3K - 5 Feb 97

<http://www.chem.su.se/~su/-size/4K-25Feb97>

<http://www.mankato.mn.us.edu/> - size 3K - 27 Nov 96

<http://www.sau.edu/> - size 2K - 4 Feb 97

University of Washington ECSEL Projects

PageRank - Consequences

- Google's use of PageRank to help rank documents led to them dominating web search in the English-speaking world, which they continue to do today.
- Other IR techniques are also used (full-text search, title search, proximity search etc.) and a fusion process is used to merge the results of these different kinds of search.
- Specific details about how exactly Google does its searching are not generally available anymore, such is the competitive nature of the online search business.
 - Other search companies have their own version of PageRank.

Relevance Feedback (and Pseudo Relevance Feedback)

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Relevance Feedback

- **Relevance Feedback** is when the user is involved in the retrieval process to improve the final result set.
- Specifically the user gives feedback on the **relevance** of documents in the initial set of results, and this is used to re-process their query and provide an updated set of results.
- Although this is not typically used in most modern IR systems, the process is important because **pseudo relevance feedback** is common, which is a simulation of relevance feedback without the user being directly involved.

Relevance Feedback Process

■ Typical process:

1. User **provides a query** (usually short and simple)
2. The system returns an **initial set of results**.
3. The user marks some **relevant** and **nonrelevant** documents (in some situations the user only marks relevant results).
4. The system computes a **better representation of the information need** based on this feedback.
5. The system returns a **revised set of results**.
6. Possibly the above process is repeated.

Rocchio Algorithm: Background

- Probably the best-known algorithm for relevance feedback is the **Rocchio Algorithm**.
- Basic Idea:
 - Based on the idea that documents and queries are **represented as vectors** (e.g. using TF-IDF, although any vector representation will work).
 - Wants to **find a query vector** that maximises similarity with relevant documents and minimises similarity with nonrelevant documents.

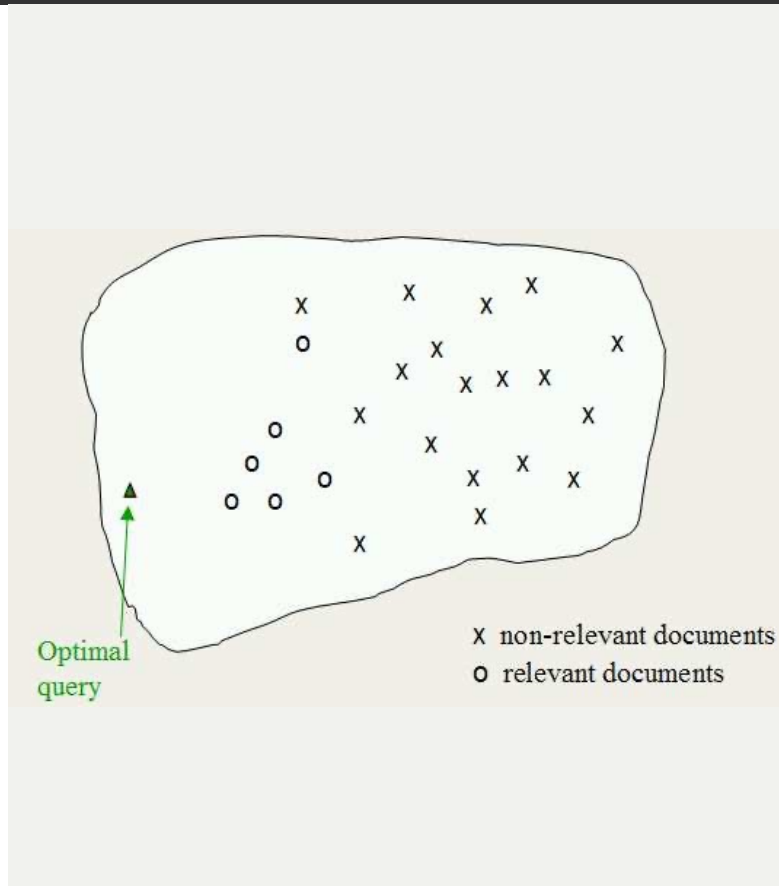
Rocchio Algorithm: Background

- Formally the aim is to find an optimal query vector $\vec{q}_{opt}$.
- The optimal query vector is the query vector with the largest difference between its similarity with the set of relevant documents and its similarity with the set of nonrelevant documents.

$$\vec{q}_{opt} = \arg \max_{\vec{q}} [sim(\vec{q}, C_r) - sim(\vec{q}, C_{nr})]$$

- where:
 - $sim(...)$ is a measure of similarity (e.g. cosine similarity)
 - C_r is the set of relevant documents
 - C_{nr} is the set of nonrelevant documents

Rocchio Algorithm: Background



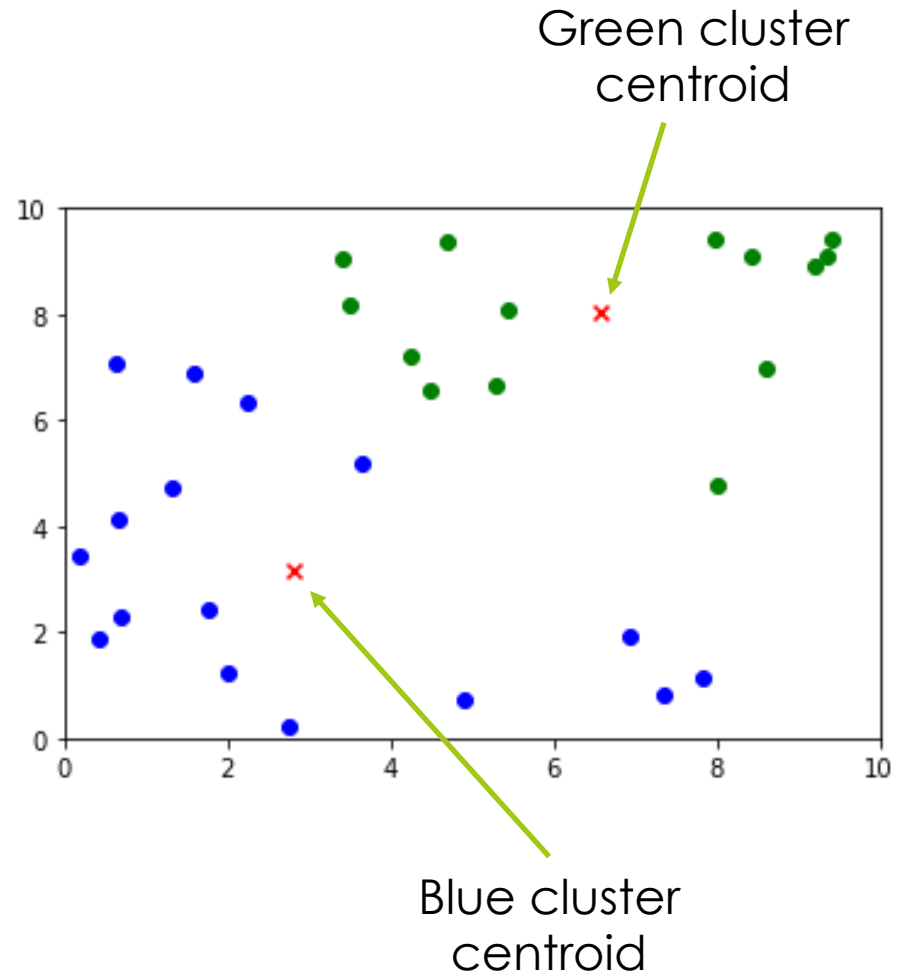
- This illustrates the ideal situation.
- Of course, the real vector space has far more than 2 dimensions: this is only an illustration.

Centroids

To further examine how the similarity between a query vector and a set of documents can be calculated, we need to consider the idea of a **centroid** of a set of vectors.

The centroid is the centre point of a set of vectors. It is itself a vector.

$$\textit{Centroid}(C) = \frac{1}{|C|} \sum_{\vec{d}_j \in C} \vec{d}_j$$



Rocchio Algorithm: Theory

With complete knowledge

- Using cosine similarity, the formula for calculating the optimal query vector is:

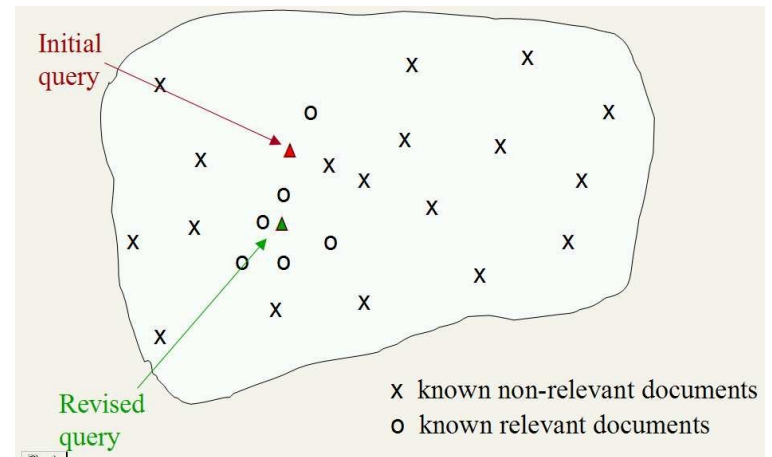
$$\vec{q}_{opt} = \frac{1}{|C_r|} \sum_{\vec{d}_j \in C_r} \vec{d}_j - \frac{1}{|C_{nr}|} \sum_{\vec{d}_j \in C_{nr}} \vec{d}_j$$

- Informally, this is the vector that is obtained by subtracting the centroid of the nonrelevant set from the centroid of the relevant set.
- **But:** In practice we don't yet know the relevant and nonrelevant sets!

Rocchio Algorithm in practice

In practice, in relevance feedback situations, we have access to a subset of the relevant and nonrelevant sets, which have been identified by the user.

The Rocchio algorithm takes this information and computes a **modified query vector** (or “revised query vector”), which is probably not be the optimal query vector, and re-runs retrieval.



Rocchio Algorithm: Formula

- The formula to calculate the **modified query vector** ($\vec{q}_m$) is as follows:

$$\vec{q}_m = \alpha \vec{q}_0 + \beta \frac{1}{|D_r|} \sum_{\vec{d}_j \in D_r} \vec{d}_j - \gamma \frac{1}{|D_{nr}|} \sum_{\vec{d}_j \in D_{nr}} \vec{d}_j$$

- Here:
- $\vec{q}_0$ is the original query provided by the user.
- D_r and D_{nr} are the documents that have been identified by the user as being relevant and nonrelevant respectively.
- α , β and γ are weights that can affect the behaviour of the formula.

Rocchio Algorithm: Weights

$$\vec{q}_m = \alpha \vec{q}_0 + \beta \frac{1}{|D_r|} \sum_{\vec{d}_j \in D_r} \vec{d}_j - \gamma \frac{1}{|D_{nr}|} \sum_{\vec{d}_j \in D_{nr}} \vec{d}_j$$

- From this formula, what is the effect of changing the weights α , β and γ ?
 - They control the balance between trusting the judged sets of relevant and nonrelevant documents and trusting the original query.
 - With many judged documents, β and γ might be high.
 - The modified query moves the original query some distance away from the centroid of the nonrelevant documents (γ) and some distance towards the centroid of the relevant documents (β).

Rocchio Algorithm in Practice

- In practice, relevance feedback is very helpful to increase recall in situations where that is important.
 - This is also the most likely situation where a user is happy to provide feedback.
 - Positive feedback (i.e. identifying relevant documents) is more useful than negative feedback (i.e. identifying non-relevant documents), so in most systems $\gamma < \beta$.
 - Reasonable values might be $\alpha = 1$, $\beta = 0.75$ and $\gamma = 0.15$.
 - Some systems only support positive feedback, which is effectively setting $\gamma = 0$.

Relevance Feedback Assumptions

- The user must have enough knowledge to make an **initial query that is close** to the relevant documents.
 - Without this, they will not be shown any relevant documents that they can give feedback on.
- Some problems that relevance feedback cannot solve by itself:
 - **Misspellings**: if the user's query does not spell terms correctly.
 - **Cross-language IR**: where documents in different languages will not be close in the vector space (although this depends on the vector representation).
 - **Vocabulary Mismatch**: where the user uses a different word/phrase to describe something (e.g. "laptop" versus "notebook computer").

Relevance Feedback Assumptions

- Relevant documents are assumed to be **close to each other** in a cluster.
- **But** sometimes relevant documents can be in several clusters:
 - Documents might use different vocabulary (e.g. astronaut vs. cosmonaut).
 - A query that combines two very different things (e.g. Football players who studied at UCD).
 - General concepts that combine many more specific subjects (e.g. "felines" includes both wild animals and domestic pets).

Pseudo Relevance Feedback

- In practice, users generally do not wish to provide feedback (known as **explicit** feedback).
- Instead, **pseudo relevance feedback** (also called **blind relevance feedback**) can be used: assume the top k ranked documents are relevant and then apply a relevance feedback algorithm.
- Quite successful in practice, although it can have problems in some situations.

Indirect Relevance Feedback

- If users do not provide explicit feedback by actively marking documents as relevant or nonrelevant, other sources of evidence can be gathered to get **implicit** feedback. For example:
 - Which documents does the user choose to click on?
 - How long did the user spend viewing a document?
 - How long did the user spend browsing the results?

Summary

- **Relevance feedback** allows the user to participate in retrieval by informing the system of results that are relevant and/or ones that are non-relevant.
 - **Example:** Rocchio algorithm based on vector representation.
- However, this is **not popular in practice**, so other ways are used to simulate feedback or get some feedback from other sources:
 - **Pseudo Relevance Feedback** (or “blind relevance feedback”: assume top k documents are relevant).
 - **Implicit Relevance Feedback** (evidence from other sources: clicks and user behaviour).

Query Expansion

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

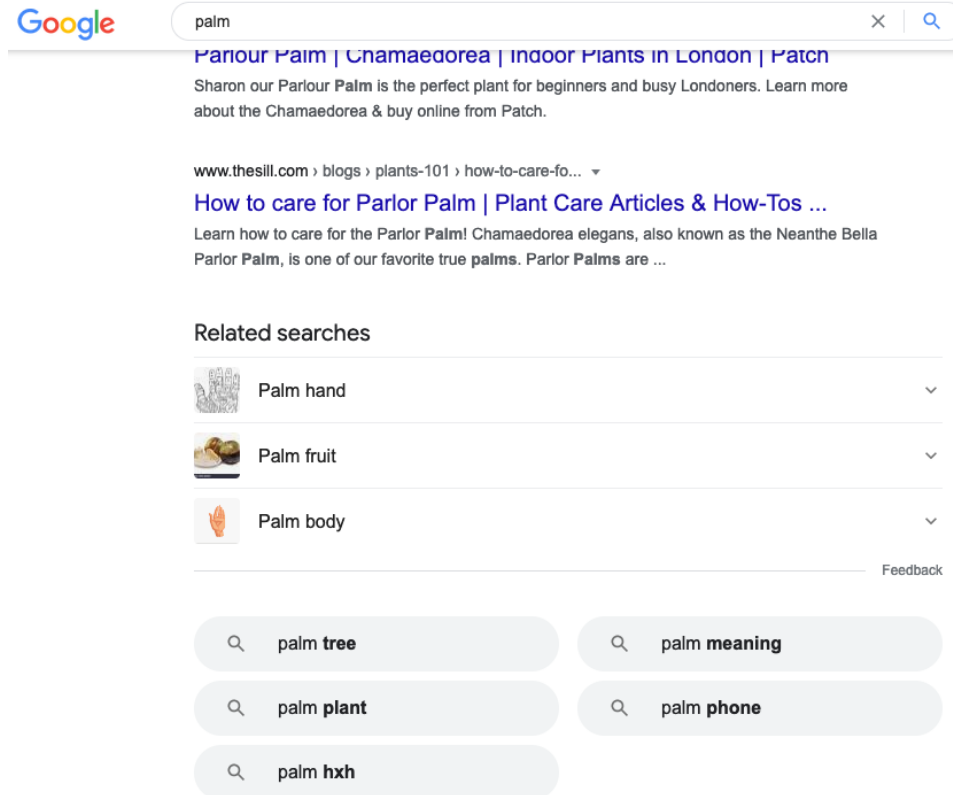
Query Expansion

- “**Query expansion** (QE) is a process in Information Retrieval which consists of selecting and adding terms to the user’s query with the goal of minimizing query-document mismatch and thereby improving retrieval performance.”*
- Generally works by increasing **recall** by matching more documents than the original query would match.
- Can be done **automatically** or **interactively** (with user involvement, similar to relevance feedback).

* Vechtomova, O. (2009). Query Expansion for Information Retrieval, https://doi.org/10.1007/978-0-387-39940-9_947

Interactive Query Expansion

- In results for the search query “palm”, Google has also suggested some query expansion options for the user to choose.



Automatic Query Expansion

- This query is shown from the medical search engine **PubMed**.
- The original query “cancer” has been automatically expanded with several related search terms.

History and Search Details

DownloadDelete

Search	Actions	Details	Query	Results	Time
#1	...	▼	Search: cancer "cancer s"[All Fields] OR "cancerated"[All Fields] OR "canceration"[All Fields] OR "cancerization"[All Fields] OR "cancerized"[All Fields] OR "cancerous"[All Fields] OR "neoplasms"[MeSH Terms] OR "neoplasms"[All Fields] OR "cancer"[All Fields] OR "cancers"[All Fields] Translations cancer: "cancer's"[All Fields] OR "cancerated"[All Fields] OR "canceration"[All Fields] OR "cancerization"[All Fields] OR "cancerized"[All Fields] OR "cancerous"[All Fields] OR "neoplasms"[MeSH Terms] OR "neoplasms"[All Fields] OR "cancer"[All Fields] OR "cancers"[All Fields]	4,355,195	10:41:52

Showing 1 to 1 of 1 entries

Resources for Query Expansion

- The main information used is **synonymy** (or near-synonymy), i.e. words that have the same or similar meanings.
- A **thesaurus** is a type of database that collects synonyms (or near-synonyms)
 - **Manual** thesaurus (maintained by human editors)
 - **Automatically derived** thesaurus
 - Using **word embeddings** as a similar effect, without using a thesaurus.
- **Query Log Mining**: based on queries previously submitted by other users.

Thesaurus-based Query Expansion

- For each term in the query, expand the query by adding words from the thesaurus that are similar to it.
 - e.g. "cancer" → "cancer, cancerous, neoplasms" etc.
- Generally increases recall, but may significantly reduce precision.
 - e.g. "interest rate" → "interest fascinate rate"
- Widely used in specialised search engines for science and engineering.
- Creating and maintaining a manual thesaurus is expensive, and is a lot of work!
 - Therefore, we seek automated methods to create a thesaurus.

Automatic Thesaurus Creation

- A thesaurus can be created automatically by analysing the **distribution of words in documents**.
- Attempt to find similarity between words.
- 2 possible definitions:
 1. Two words are similar if they co-occur with similar words (e.g. “car” and “truck” may be similar because they both occur with words like “road”, “driver”, “license”, etc.)
 2. Two words are similar if they occur in particular grammatical relations with the same words (e.g. you can “harvest”, “peel”, “eat”, “prepare” apples and pears, so apples and pears may be similar).

Automatically Generated Thesaurus (example)

Word	Nearest Neighbours
absolutely	absurd, whatsoever, totally, exactly, nothing
bottomed	dip, copper, drops, topped, slide, trimmed
captivating	shimmer, stunningly, superbly, plucky, witty
doghouse	dog, porch, crawling, beside, downstairs
makeup	repellent, lotion, glossy, sunscreen, skin, gel
mediating	reconciliation, negotiate, case, conciliation
keeping	hoping, bring, wiping, could, some, would
lithographs	drawings, Picasso, Dali, sculptures, Gauguin
pathogens	toxins, bacteria, organisms, bacterial, parasite
senses	grasp, psyche, truly, clumsy, naïve, innate

Word Embeddings for Query Expansion

- In recent times, researchers have used neural network approaches to create **word embeddings**: a representation of word as vectors in an n-dimensional space.
 - word2vec is probably the most famous: 300-dimension vectors.
 - Other, newer ones include GloVe, ELMo, or embeddings from language models like BERT.
 - Because words are represented as vectors, a **similarity** between words can easily be calculated.
 - It is assumed that words that are close to one another in the vector space have similar meaning.

* Diaz, F., Mitra, B., and Craswell, N. (2016) Query Expansion with Locally-Trained Word Embeddings. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*.

Search Engine Query Expansion (Query Log Mining)

- The **query log** stores details about previous queries and user behaviour.
- **Example:** After searching for “palm”, many users next search for “palm oil”.
- **Example:** Users searching for “football” and “soccer ball” often click on the same URL in the search results. Therefore, these are potential expansions of each other.

Summary

- Query Expansion is the process of adding terms to a query in order to **increase recall**.
- However, it is not a perfect process, and it can have negative effects also.
- A **manual thesaurus** is the most reliable method, but is expensive and difficult to maintain.
- Therefore, several **automatic** methods have been proposed.

Web Search: Web Crawling

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Introduction

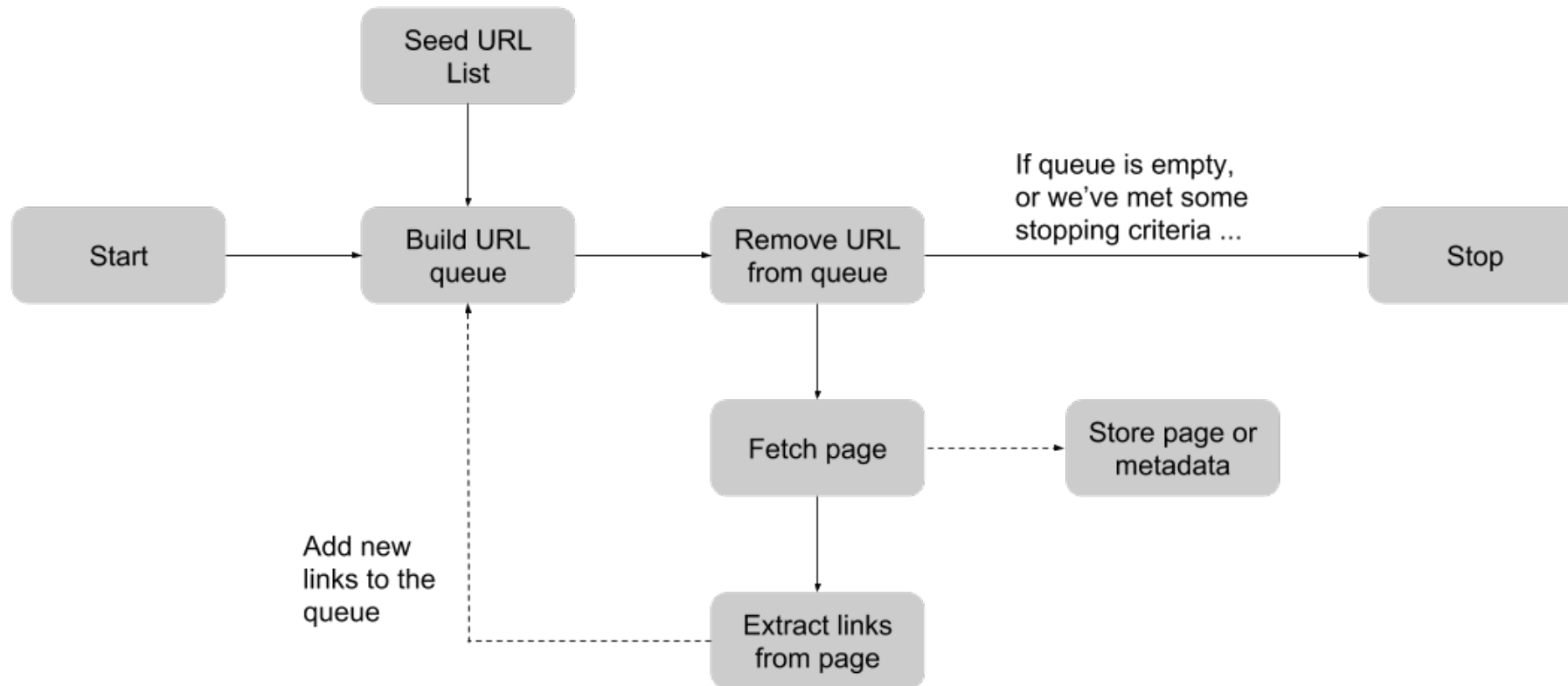
- Many of the techniques we have look at, both for performing and evaluating Information Retrieval were originally developed before the World Wide Web (WWW) became popular.
- Nowadays, the WWW is the biggest source of information in existence and web search has become an extremely important aspect of using the web.
- With the shift of IR towards this new environment, a number of significant challenges have presented themselves. We will look at a number of these.

How do we find information?

- **Web Crawlers** or **Web Spiders** are programs that automatically look for documents on the web that will be included in the index so that users can find them when searching.
- They do this by **extracting the hyperlinks** from each page they find, which will in turn point them to new pages to index.
- Even the decision on what policy to use for following hyperlinks is a subject of debate: one study claims that using a Breadth-First Search strategy will result in finding higher-quality pages (as measured by PageRank)*.
- It is also considered to be more polite to use a Breadth-First Search, as it reduces load on servers.

* M. Najork and J. Weiner, Breadth-First Search Crawling Yields High-Quality Pages, 2000

Typical Web Crawler Workflow



How do we find information?

- Breadth-first spider search:
 - Manually add a list of "seed" URLs to the spider's queue of pages to visit
 - While there are still URLs in the queue of pages to visit
 - Remove the first URL from the queue of pages to visit and download the page
 - Extract the links from the document
 - Add these links to the queue of pages to visit if they haven't already been visited and are not already in the queue.
- A depth-first search is the same, except you use a stack instead of a queue.
- Normally runs in **parallel**, so it does not waste time waiting for websites to respond.

Polite Crawlers

- It is easy to harm a website by using a web crawler, especially by overloading it with too many requests.
- Web Crawlers are expected to be “polite”, by:
 - Obeying robots.txt files and other methods to give instructions to crawlers.
 - Not overloading the website (obey Crawl-Delay).
 - Not consuming too much bandwidth.
 - Identifying who they are and who they belong to (“Googlebot”, “Baiduspider”).

Polite Crawlers: The Robots Exclusion Standard

- The Robots Exclusion Standard is a method of asking web crawlers not to index certain portions of a web site.
- It is a **voluntary standard**, so there are no technical barriers stopping crawlers that ignore the standard from accessing the pages in question.
- Whenever a well-behaved crawler visits a web server, it will firstly check the root directory for a file called `robots.txt` (i.e. `http://www.example.com/robots.txt`).
- This is a plain text file that contains a list of resources that the crawler should not visit.
- Different lists can be provided for different crawlers (e.g. if you wanted a particular page to be searchable on Yahoo! but not on Google).

Finding Information - the Robots Exclusion Standard

■ Robots Exclusion Rules

```
User-agent: *  
Disallow: /cgi-bin/  
Disallow: /tmp/  
Disallow: /~joe/
```

■ Allow one bot

```
User-agent: Googlebot  
Disallow:
```

```
User-agent: *  
Disallow: /
```

robots.txt

- There are a number of common mistakes made by web site owners when creating robots.txt files:
 - The robots.txt file must be in the root directory. A crawler is not required to check anywhere else.
 - Only one disallowed path should be specified on each line.
 - Missing forward-slash at the beginning of disallowed paths.
 - There are no wildcards in the “Disallow” section (though some crawlers support it)
 - Misspelling the name of a crawler you wish to exclude.
 - Trying to use the non-existent “Allow” command (though some crawlers support it)
- Some crawlers obey a “Crawl-Delay” instruction (how long to wait before the next request).

Polite Crawlers: noindex and nofollow

- Ignore pages that have `<meta name="robots" content="noindex" />`.
- Don't follow any links from pages that have `<meta name="robots" content="nofollow" />`.
- Individual links can be marked nofollow also:
`<a href="http://example.com">Anchor Text</a>`
`<a href="http://example.com" rel="nofollow">Anchor Text</a>`
- The exact treatment of this varies for different search engines.

Some Web-Specific Challenges

- In many cases, there may be several ways to find the same page, using different URLs. This can affect results because of duplication, but also it can affect PageRank results, for example.
- **Examples:**
 - For many domains, **www.example.com** redirects to **example.com**, or the other way around (easy to handle).
 - **Anchors** at the end of URLs only refer to sections inside the page and don't affect the page that is seen. **example.com/article?id=123#heading** refers to a section within the page that is named "heading" (using a `<a name="heading" />` tag).
 - **SEO-friendly URLs.** Websites will often use descriptive words in their page URLs to describe their contents to search engines. **example.com/article?id=123** and **example.com/ucd-appoints-new-president** might refer to the same page.
 - **Marketing tracking.** Often, query strings are used to dynamically select which page to load (e.g. **?id=123** in the above examples). However, tracking information is also often used in this way, so marketers can see where users came from (e.g. if they clicked in a link in an email, it might have **?id=123&campaign_id=234**). These URLs can become very complex.

Non-static documents

- Another issue is the fact that the web is a **dynamic environment**.
- New pages are **added constantly**, and many existing pages may **change frequently**.
- Even if a crawler has visited a large number of documents, it must still be **revisited** so the index does not get out-of-date.
 - For instance, news and social media sites will be constantly changing and are typically re-indexed on a regular basis.
- On the other hand, there are huge numbers of pages that are essentially static over long periods of time (mailing list archives, individual news articles, etc.)
- A policy must be implemented that decides which pages should be re-indexed on a regular basis and which need not be.

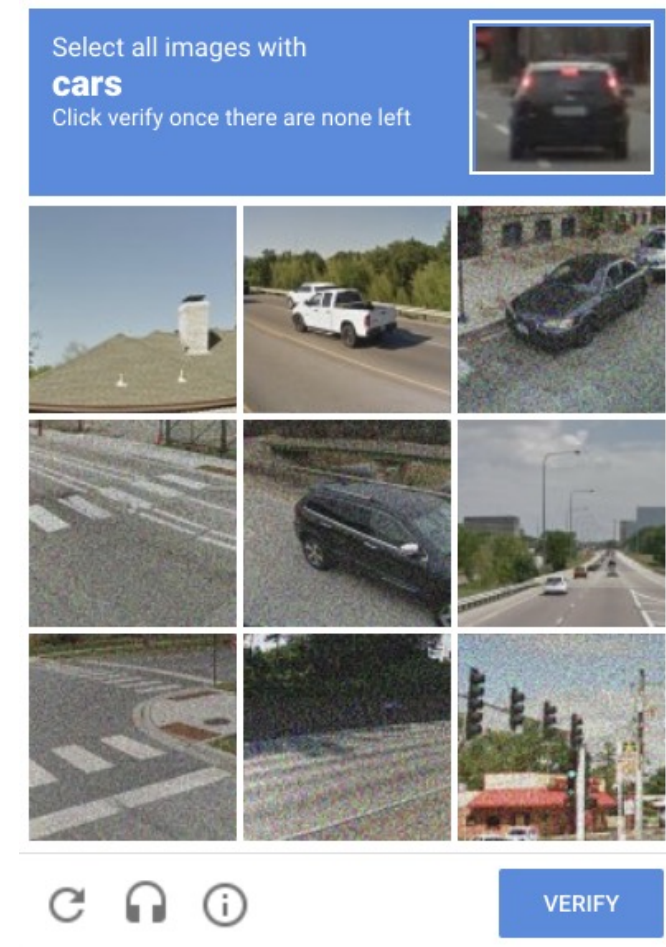
Finding information: challenges

Pages designed to frustrate attempts at automated access (e.g. through the use of CAPTCHAs).

A CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) is typically an image containing text that computers find difficult to read.

They are frequently to be seen when signing up for accounts with various web sites.

An automatic crawler can't access pages that are guarded by CAPTCHAs.



How big is the web?

- Google state that their index contains “hundreds of billions of webpages”.
- Technically, the size of the web is probably infinite, as dynamic pages give different results depending on the information they're given:
 - Calendar applications can typically show data for an indefinite amount of time into the future.
 - Search engines themselves will give a different page in response to every different query they receive.

* <http://www.google.com/insidesearch/howsearchworks/thestory/>

How big is the web?

- The **Deep Web*** (or invisible web or hidden web) refers to the fact that web crawlers cannot easily gain access to particular online resources.
 - Some pages or groups of pages may not have any links from elsewhere. Since web crawlers work by following links, it may not be possible to find these pages.
- Content served via AJAX/Asynchronous JavaScript can be hard for crawlers to index. AJAX content is fetched by JavaScript code running within a web page and is not accessible just using a URL like traditional content.

* M. K. Bergman, The Deep Web: Surfacing Hidden Value 2001

Web Search: Adversarial Information Retrieval

COMP3009J: Information Retrieval

Dr. David Lillis (david.lillis@ucd.ie)

UCD School of Computer Science
Beijing Dublin International College

Adversarial IR

- One key difference between traditional IR and Web Search is that traditionally, works were published without IR systems in mind.
- An author writing a book or a newspaper, magazine or technical article would only concentrate on the work they were producing.
- The publishing of web pages is very different. An entire industry known as **Search Engine Optimisation (SEO)** has emerged.
- SEO involves taking steps to get your page high up in search engine rankings.
 - Motivation: Higher rankings → more clicks → more money.
- Case study on how Google's search algorithms can impact businesses:
<https://www.bbc.com/future/article/20240524-how-googles-new-algorithm-will-shape-your-internet>
 - Housefresh.com went from thousands of daily visitors and 15 employees to hundreds of visitors and having to let go most of its staff because of a change in Google's ranking algorithms.

Search Engine Optimisation (SEO)

- To improve search engine ranking, many techniques are available, and can be divided into two categories:
 - "White Hat" SEO: operating ethically and following search engine rules and guidelines to get higher rankings.
 - Optimising the website for relevant keywords.
 - Creating mobile-friendly versions of websites.
 - Satisfying users' information needs.
 - Descriptive and relevant URLs, titles and meta descriptions.
 - Relevant, descriptive text in links.
 - etc.
 - "Black Hat" SEO: violating search engine guidelines and rules to try and get an artificial advantage: Adversarial IR.

Adversarial IR: Meta Tags

- HTML allows for page authors to describe the contents of their pages by using <meta> tags.
- These were designed to allow such information as a page's description, keywords and author to be specified in a standardised way.

```
<head>
  <title>Stamp Collecting World</title>
  <meta name="description" content="Everything you wanted to
    know about stamps, from prices to history." />
  <meta name="keywords" content="stamps, stamp
    collecting, stamp history, prices, stamps for sale" />
</head>
```

Adversarial IR: Meta Tags

- Some early search engines did not have the processing power to perform full-text search.
- Even for many that did, they initially gave a high weight to terms appearing in the description and keywords of a page.
- Theoretically, this would succinctly describe the key contents of their web sites.
- In reality, this was open to abuse by dishonest webmaster that would attempt to target certain search queries.
- A common example was people including the word “free” amongst their keywords, as many searchers would use this when they began searching for a particular service.
- Because of this behaviour, meta keyword tags are mostly ignored by search engines nowadays. Meta descriptions are still used to some degree.

Adversarial IR: Hidden Text

- Consider this example:

A site for buying computer components

```
<p>A site for buying computer components</p>  
<p><font color="#ffffff"> computer parts,  
computer hardware, memory </font></p>
```

Adversarial IR: Hidden Text

- Consider this example:

A site for buying computer components

computer parts, computer hardware, memory, CPUs, central processing units, RAM, HDD, hard drives, flash memory, USB, keyboard, mouse, mice, cases, monitors, displays, peripherals, cheap, best

```
<p>A site for buying computer components</p>
<p><font color="#ffffff"> computer parts,
computer hardware, memory </font></p>
```

Adversarial IR: Hidden Text

- ▣ This is an example of manipulation using **hidden text**.
- ▣ The second paragraph in the previous slide will not be visible to the human eye, as the text is the same colour as the background.
- ▣ This is a method of changing the contents of the page without negatively affecting what a user sees.
- ▣ It has two main functions:
 1. Add extra keywords for a search engine to pick up that aren't already contained in the content of the page (known as "keyword stuffing").
 2. Increase the term frequency of important search terms.

Adversarial IR: Hidden Text

- Consider this example:

A site for buying computer components

computer parts, computer hardware, memory, CPUs, central processing units, RAM, HDD, hard drives, flash memory, USB, keyboard, mouse, mice, cases, monitors, displays, peripherals, cheap, best

```
<p>A site for buying computer components</p>
<p><font color="#ffffffe"> computer parts,
computer hardware, memory </font></p>
```

Another example.

```
<p class="class1">A site for buying computer components</p>
```

```
<p class="class2">computer parts, computer hardware, memory</p>
```

... with the following CSS:

```
.class1 {  
    font-family: century gothic;  
}
```

```
.class2 {  
    color: white;  
}
```

Adversarial IR: Hidden Text

- Here, the manipulation is not as obvious, as you'd have to check the Cascading Style Sheet class "class2" to see if there are any color changes defined there.
- You would also have to check any other elements that the second paragraph is contained in.
- This is not a trivial task for a crawler to carry out (remember that a crawler needs to operate very quickly to build up a sufficiently large index).
- Many search engines will remove your site from its index if this type of activity is found.

Adversarial IR: Cloaking

- **Cloaking** is the term given to the practice of serving different content to web crawlers than to users.
- A well-behaved crawler will identify itself to a web server when it visits, by way of a “user agent string”.
- A server (or a script running on it) can easily be configured to give different content based on this user agent.
 - This technique is often used legitimately to show a mobile-optimised site for phone users.
- The idea is to serve highly-targeted text to the crawler, so as to gain a high ranking on particular searches.
 - Regular users would be served the normal content.
- This can also be done using redirect (redirecting users and crawlers to different URLs).

Adversarial IR: Sneaky JavaScript

- Web crawlers (and old browsers) are normally not capable of executing JavaScript code.
- In this situation, there is a `<noscript>` tag that allows you to specify text content for users who are unable to run JavaScript (this is not so common anymore) or those who are unwilling to run it.
- This text is what the web crawler will see as the content of the page.
- As a result, this can be exploited in the same way as cloaking: web crawlers index the contents of the “noscript” tag, whereas real users will get redirected (via JavaScript) to a different page without them noticing.

Adversarial IR: Exploiting PageRank

- Since a high PageRank (or similar) results in high rankings in search results, there is a commercial benefit to having a high PageRank score.
- This has led to the creation of “link farms”: sites (or networks of sites) with the sole purpose of linking to other sites so as to increase their PageRank.
- Each page has a small contribution to overall PageRank, so it is easy to create huge numbers of pages consisting solely of links.
- Additionally, pages that benefited from this PageRank would link back to the farm so as to boost the PageRank of the farm.
- Often, this link would be hidden by being the same colour as the rest of the page.
- Some owners of sites with high PageRank built up in this way would charge a fee from anybody who wanted to be linked to from the farm.

Adversarial IR: Exploiting PageRank

- Another approach is “comment spam”, where pages that allow third-party commenting are abused to try and boost PageRank.
 - Advertising links are posted as comments in hundreds or thousands of places, to gain PageRank from each.
- This is why the “nofollow” attribute for links was created. This would mean that search engines do not take the link into account when calculating PageRank. This allows site owners to have more control over which links from their site add to the PageRank of another page.
- This still needs to be implemented correctly. Bots will search for sites that do not use these attributes and continue to try and exploit this where possible.

Adversarial IR: Doorway Pages

- Rather than attempting to include all of the desired search queries on a single page (which will affect the term frequencies of the individual terms), sometimes individual “doorway pages” are created.
- These are pages that are optimised for a particular search query, each of which then funnels a user to the same destination.
- Again, this is a method of creating content that is solely for the benefit of search engines so as to gain a high ranking.

Adversarial IR: Summary

- Some website owners can violate search engine's rules and guidelines to get higher search engine rankings ("Black hat SEO").
- This can be difficult to detect, but search engines will penalise or ban sites who are found to break the rules.
 - A guide to Google Penalties (<https://www.searchenginejournal.com/the-complete-list-of-google-penalties-and-how-to-recover/201510/>)
 - 10 Big Brands that were Penalized by Google (<https://marketingland.com/10-big-brands-that-were-penalized-by-google-69646>).
 - 5 Times Google Penalized Itself for Breaking Its Own SEO Rules (<https://searchengineland.com/google-penalized-breaking-seo-rules-184098>).
- The job of the search engine creator is to try to make sure that web pages using these tactics are not included in the index, because they are unlikely to satisfy users' information needs.